UNITEC INSTITUTE OF TECHNOLOGY

MASTER THESIS

Comparative Research on Code Vulnerability Detection:

Open-Source vs. Proprietary Large Language Models and LSTM Neural Network

Author:

Ravihansa Geekiyanage Geekiyanage Don

Supervisor:

Dr. Iman Ardekani

Co-supervisor:

Dr. Jamie Bell

A thesis submitted in partial fulfillment of the requirements

for the degree of Master of Applied Technologies

in the

School of Computing, Electrical & Applied Technology

December 23, 2024

© Mr. Ravihansa Geekiyanage

ABSTRACT

The reliance of industries such as banking, e-commerce, logistics, transportation, energy, and healthcare on computer systems has escalated the threat of cyberattacks. With the rapid development of the online ecosystem and the growth of sensitive data, cybercriminals have developed the ability to attack software systems. Safeguarding software has become an urgent yet challenging task despite advancements in technology. The contributions of this thesis show the incorporation of security into the Software Development Lifecycle through a focus on Static Code Analysis as a proactive approach for the discovery and mitigation of security flaws in the development cycle.

This research focuses on enhancing vulnerability detection in source code through advanced machine learning techniques, including open-source fine-tuned and proprietary large language models. It compares the models, including CodeGen2, LLaMA 2, and GPT, developed by OpenAI to quantitative and qualitative how suitable they are for detecting vulnerabilities, their accuracy and efficiency. Using a zero-shot classification-based approach, the study examines these models' capabilities in detecting security risks and compares them to Word2Vec+LSTM neural networks.

The findings reveal that CodeGen2 emerges as the most reliable model for vulnerability detection, achieving near-perfect precision and balanced recall, leading to superior F1 and AUC scores. LLaMA2-7b delivers reasonable performance, particularly in precision, but falls short in recall. Conversely, GPT-4 Assistant excels in recall but suffers from a high false positive rate, limiting its effectiveness. Classical neural networks, such as Word2Vec+LSTM, demonstrate moderate capability but lag modern LLMs in precision and recall.

Through this comparative analysis, the thesis underscores the importance of selecting tools aligned with organizational needs and constraints. The insights gained contribute to developing secure software by integrating machine learning into the SDLC, inspiring further progress in vulnerability detection and mitigating risks in an increasingly digital world.

ACKNOWLEDGEMENT

I want to take this opportunity to express my special thanks to all those who supported and directed me throughout my study and research to complete this thesis. Thank you for encouraging and helping me to make it possible.

I want to extend my sincere thanks to my supervisors, Dr. Iman Ardekani and Dr. Jamie Bell, for the support and guidance they provided to me. Their expertise, knowledge and feedback have been invaluable.

Many thanks to my family for their patience, love and understanding. I have had your constant support as an anchor and without you, I couldn't have done this.

Thank you all, not only for being here but also for helping make this journey a success.

Thank You.

Contents

Abstract iii

Acknowledgement iv

List of Abbreviation viii

List of Figures ix

List of Tables xi

CHAPTER – 1 1

Introduction 1

1.1. Overview 1

1.2. Background and Problem Definition 2

1.3. Motivation 4

1.4. Project Goal 5

1.5. Contribution 7

1.6. Existing Challengers 8

1.7. Organization of Thesis 9

CHAPTER – 2 12

Background Theory 12

2.1. Overview 12

2.2. Evolution of Secure Software Development 12

2.3. Security Integration in SDLC 13

2.4. Vulnerability Detection Techniques 16

2.5. Manual Code Review 16

2.6. Static code analysis 17

2.7. Dynamic Code Analysis 18

2.8. Strengths and Limitations 19

2.9. Collaborative Approach 19

2.10. Future Directions 19

CHAPTER – 3 22
Related Work.....22
3.1. Literature Review 22
3.2. Traditional AI Approaches 22
3.3. Large Language Models 26
3.4. Research Gap 29
3.5. Selected Programming Language 30
3.6. Selected Dataset 31
CHAPTER – 4 33
Methodology 33
4.1. Research Design 33
4.2. Rationale Behind Choosing the Models 33
4.3. Comparative Analysis Goals.....34
4.4. Models Selection and Setup.....34
4.4.1. LLaMA2 Model.....34
4.4.2. CodeGen2 Model 37
4.4.3. Finetune LLMs 40
4.4.4. GPT Models 43
4.4.5. Neural Network.....45
4.5. Performance Analysis 48
CHAPTER – 5 52
Implementation 52
5.1. Overview.....52
5.2. Dataset Preparation 52
5.2.1. Dataset Clean 53
5.2.2. Dataset Splitting.....53
5.3. Open Source LLMs Fine Tune 54
5.3.1. Model Access and Quantization 54
5.3.2. Hyperparameters 56

5.4. Open Source LLMs Validation 57
5.5. Open Source LLMs Testing 58
5.5.1. Lama2 Performance Analysis 59
5.5.2. CodeGen2 Performance Analysis 62
5.6. GPT Models Testing 64
5.7. GPT Models Performance 66
5.7.1. GPT-3.5 – Chat Completion Mode 66
5.7.2. GPT-3.5 – Assistant Mode 67
5.7.3. GPT-4 – Chat Completion Mode 67
5.7.4. GPT-4 – Assistant Mode 68
5.8. Neural Network Classifier 69
5.8.1. Hyperparameter Tuning 70
CHAPTER – 6 74
Comparative Analysis 74
6.1. Overview 74
6.2. Opensource LLMs 74
6.3. Proprietary LLMs 76
6.4. Neural Network 77
6.5. Overall Comparison 78
6.5.1. Qualitative Comparison 78
6.5.2. Quantitative Comparison 83
CHAPTER – 7 86
Conclusion and Future Work 86
7.1. Conclusion 86
7.2. Real-World Integration 87
7.3. Future Work 88
Bibliography 89
Appendix A 102
Source Code Repository 102

List of Abbreviation

CVE	Common Vulnerabilities and Exposures
LLMs	Large Language Models
AI	Artificial Intelligence
SDLC	Software Development Life Cycle
ML	Machine Learning
LSTM	Long Short-Term Memory Networks
SSDLC	Secure Software Development Lifecycle
CI	Continuous Integration
CD	Continuous Delivery
CLASP	Comprehensive Lightweight Application Security Process
SCA	Static code analysis
DCA	Dynamic Code Analysis
SVM	Support Vector Machine
RNN	Recurrent Neural Network
ANN	Artificial Neural Network
DNN	Deep Neural Network
CNN	Convolutional Neural Network
SAST	Static Application Security Testing
PEFT	Parameter-Efficient Fine-Tuning Technique
AST	Abstract Syntax Tree
LoRA	Low-Rank Adaptation
QLoRA	Quantized Low-Rank Adaptation
NLP	Natural Language Processing
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve

List of Figures

Figure 1.1: Various vulnerabilities have been exploited. 2
Figure 2.1: Secure Software Development Lifecycle. 14
Figure 4.1: Rotary Position Embedding 35
Figure 4.2: Model architecture of LLaMA2 36
Figure 4.3: Full Fine-tuning, LoRA and QLoRa Fine-tuning Methods 42
Figure 4.4: Original Data and Quantized Data 43
Figure 4.5: GPT Models Architecture 44
Figure 4.6: Word2Vec + LSTM Neural Network 46
Figure 4.7: Confusion Matrix 49
Figure 4.8: AUC-ROC 50
Figure 5.1. Result of the training loss of the fine-tuned LLaMA2 model. 59
Figure 5.2. Result of the learning rate of the fine-tuned LLaMA2 model. 59
Figure 5.3. Result of the validation loss of the fine-tuned LLaMA2 model 60
Figure 5.4. Confusion Matrix of the LLaMA2 Model 61
Figure 5.5. Result of the training loss of the fine-tuned CodeGen2 model. 62
Figure 5.6. Result of the validation loss of the fine-tuned CodeGen2 model 63
Figure 5.7. Confusion matrix of the fine-tuned CodeGen2 model 64
Figure 5.8. GPT-3.5 Chat Completion Evaluation Metrics for Three Different Prompts 66
Figure 5.9. GPT-3.5 Assistant Evaluation Metrics for Three Different Prompts 67
Figure 5.10. GPT-4 Chat Completion Evaluation Metrics for Three Different Prompts 68
Figure 5.11. GPT-4 Assistant Evaluation Metrics for Three Different Prompts 69
Figure 5.12. Word2Vec+LSTM Plane Code Input Hyper Parameter Tuning Result 71
Figure 5.13. Word2Vec+LSTM Plane Code Input of Training and Validation Accuracy ... 71
Figure 5.14. Word2Vec+LSTM AST Code Input Hyper Parameter Tuning Result. 72
Figure 5.15. Word2Vec+LSTM AST Code Input of Training and Validation Accuracy 73
Figure 6.1. LLaMA2 and CodeGen2 Models Performance. 75

Figure 6.2. GPT Models Best Performance Prompts Result 76

Figure 6.3. Neural Network Two Types of Input Performance Result 77

Figure 6.4. LLaMA2 GPU Power Usage & CPU Usage..... 81

Figure 6.5. CodeGen2 GPU Power Usage & CPU Usage..... 81

Figure 6.6. Neural Network CPU Usage & Memory Usage. 82

List of Tables

Table 3.1: Summary of Studies on Source Code Vulnerability Detection Using LLMs 28

Table 4.1: The input sentence "The sky clear" is an example of an attention mask 35

Table 4.2: CodeGen2 Model Details 40

Table 4.3: GPT models 44

Table 5.1: Hyperparameters for LLaMA2 and CodeGen2 models..... 56

Table 5.2: GPT-3.5 – Chat Completion Evaluation Metrics for Three Different Prompts .. 66

Table 5.3: GPT-3.5 – Assistant Evaluation Metrics for Three Different Prompts 67

Table 5.4: GPT-4 – Chat Completion Evaluation Metrics for Three Different Prompts 67

Table 5.5: GPTT-4 – Assistant Evaluation Metrics for Different Three Prompts..... 68

Table 6.1: Performance Metrics of All Models 83

CHAPTER – 1

Introduction

Today, software is everywhere, powering systems and services that affect our lives in big and small ways, from mobile devices and household appliances to complex industrial setups and essential public services. The constant innovation and evolution of software development processes have made developing and deploying secure, effective, and stable software, and development is becoming very challenging [1][2]. The highest risk is software vulnerabilities, flaws or weaknesses in software that can be exploited by an attacker and result in a significant loss to stakeholders or the public [3]. Yet, as software sophistication and utility increased in the modern environment, more problems occurred concerning the existing methodologies for developing software.

Defects or weaknesses in a system's design, implementation or internal controls, which, if exploited by a threat source, would result in a threat to software, and these are the kinds of software faults that can be misused. Vulnerabilities are usually hidden and go unnoticed by developers, testers and users. In contrast, typical software faults manifest as errors or crashes during everyday use or testing. Their hidden nature, however, makes them especially dangerous because they can allow for unauthorized access, data breaches, service interruptions and more [4].

1.1. Overview

The ever-growing use of software systems in healthcare, transport, finance, and all other service risks implies more critical implications of software being used in these areas. Even minimal imperfection results in significant losses, and the case of irregular risks, such as incidents like WannaCry ransomware [5], Heartbleed [6] and others in the OpenSSL cryptographic library, shows that even the most minor things can be fatal. They've cost billions of dollars, damaged its image, and even closed core business operations. For instance, the WannaCry attack impacted many countries' healthcare and business sectors, closing their operations with significant money and time lost. Attacks like these clearly illustrate that software security problems are a technical challenge and an essential global societal issue.

Over the years, the software development community has ensured that most identified risks are avoided during the Software Development Life Cycle (SDLC). This has been done through methods like Static code analysis (SCA), Dynamic analysis, and Formal verification, which seek to identify weaknesses in the source code before the code reaches the end user. However, equivalent attempts have not prevented the formation of novel risks. For instance, the Common Vulnerabilities and Exposures (CVE) list is the most significant worldwide list of publicly exposed vulnerabilities, comprising over 125000 records as of 2023 and adding. This trend

raises the need to consider the urgent need for more effective tools and methodologies to detect and mitigate software vulnerabilities [7].

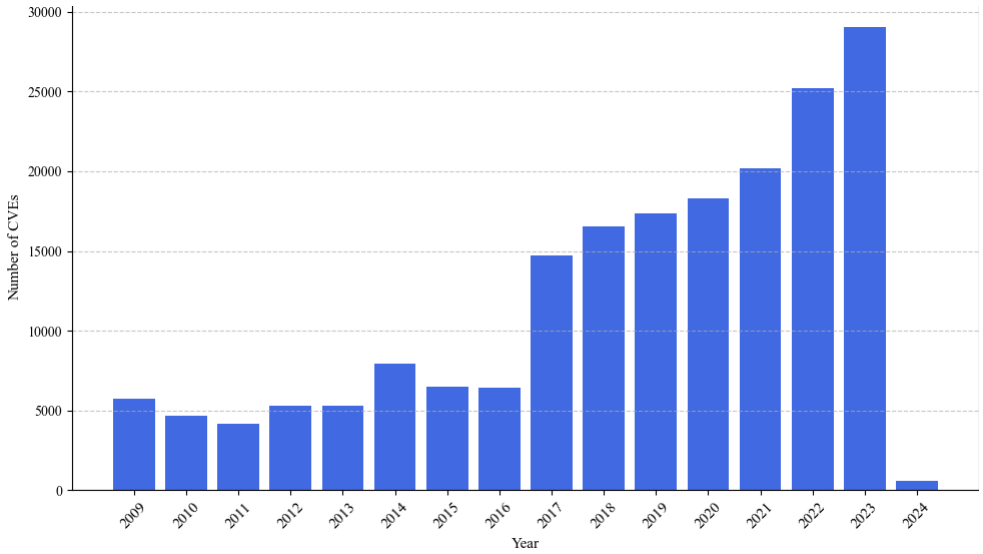


Figure 1.1: Various vulnerabilities have been exploited [7].

The problems get even worse regarding the modern complex software systems that include different languages, integrated libraries, and cloud services. Thus, developers must be able to think about these subtleties and simultaneously possess the proper attitude, tools and methods to build secure software. Currently, there are many standard regulations and codes of practice that address issues of secure software development, but what stands out is the weak implementation and compliance with the guidelines, thus making application interfaces insecure and can easily be exploited.

An interesting observation of the vulnerability types in various product types. More than one-third of the high-risk vulnerabilities found might be exploited remotely. More than 70% of the vulnerabilities detected were of the five most common types [8]. This highlights the necessity for a comprehensive vulnerability management strategy incorporating remote scanning capabilities rather than depending primarily on agent-based methods.

However, not all vulnerabilities provide a high risk, with just a tiny portion (less than 1%) contributing to the highest risk. These significant vulnerabilities have been weaponized for exploitation, are actively exploited by ransomware, threat actors, and malware, or have been confirmed to be abused in the wild [9].

1.2. Background and Problem Definition

The endless development in the information technologies sector has caused the rise of software systems in different fields in human activity and health care, business and finance, transport,

defense, etc. This is why relying on such systems is increasingly becoming important, and the security and validity of these systems must be given adequate consideration. These weaknesses, if exploited in software, can cause catastrophic outcomes, including data leakage, loss of money, company image, and even threaten national security. As software systems become more complicated and threats become more diverse, detecting and preventing weaknesses has become more complex and vital [4].

The cybersecurity environment is dynamic and appears to be shaped by new threats, new technologies, new business structures and a need for more skilled professionals [11]. Properly financed cybercriminal groups are emerging, and their attacks are more complex than can be dealt with by our security solutions. For example, there is an increase in ransomware attacks, the increasing use of open-source software, and the widening supply chain vulnerabilities. Not only are they exposing software to increased dangers, but they're magnifying those dangers. These sophisticated threats are usually beyond the purview of standard security architecture and practice, thus making a more proactive and flexible vulnerability management approach necessary [12].

New technologies must be involved to enhance organizational innovativeness and business growth, but these new technologies introduce poorly understood risks and expand the attack surface. For example, the existing cloud-computing frameworks ensure scalability and flexibility but introduce significant security risks that various organizations need help addressing. The decentralized drive for application modernization has generated several applications within the network, consequently increasing risk areas. These factors are made awful by a generally disaggregated system of infrastructural and development efforts. Many rising vulnerabilities are attributed to late modifications to legacy systems, weak integration of new applications, and no coordination between development and security teams.

Furthermore, rapid organizational changes, such as mergers, acquisitions, and shifts in work environments (e.g., remote work, cloud migrations), create additional attack surfaces. These changes can introduce new vulnerabilities and make existing ones more challenging to detect and address. The lack of comprehensive security policies and realignment of organizational priorities often results in delayed responses to potential threats, exposing systems [13].

The need for more qualified personnel is a critical factor threatening the cybersecurity industry. As the focus on information security increases, the global shortage of experts and the number of open vacancies continue to increase. This talent shortage leads to staff overworking, staff fatigue, and critical high turnover rates among current employees, further challenging vulnerability management initiatives. Due to this, they end up with repeat or cyclical risks, open flaws, and higher error gradients in structuring software development and management. Outsourcing development work instead of the opposite increases the potential for lower quality and security of the result since third parties, as a rule, need to be made aware of some specifics of the organizational environment and its demands in terms of security.

Modern approaches to developing software, like the application of AI code tools and new testing techniques, have brought about both positives and negatives. On the same note, even with AI technologies, one is bound to realize that there are disadvantages due to AI's inability to develop

unique developmental solutions and overdependence on standard working procedures, rules, and structures [14]. If not validated correctly, these tools may even introduce some security loopholes in the source code. Furthermore, establishing who is to blame for a security vulnerability stemming from an AI-generated code must be more well-defined and ever-changing, making things somewhat unpredictable for companies.

Since there is an increase in the number of risks and changes in the threat environment, it is essential to create new ways to prevent and identify software vulnerabilities or enhance the methods used. This research addresses these challenges using novel approaches to automated vulnerability detection in software code that utilizes complex method comparison.

1.3. Motivation

The motivation for this thesis comes from the current lack of proper security in software and an attempt to improve security by implementing complex ML techniques, particularly in the context of Large Language Models (LLMs) [15]. Even as we experience an exponential increase in the total number of software vulnerabilities and the specific complexity of software vulnerability, the traditional methods of vulnerability identification have manifested significant inadequacies. This thesis evolved because open-source and proprietary LLMs can produce better solutions for finding and eliminating the risks in software systems and neural network models.

One of the primary motivations for this research is the growing sophistication of cyber threats. Today's hackers are more organized and well-equipped, using techniques that neutralize most traditional security tools. There are new types of attacks, like ransomware, supply chain attacks, threats relating to third parties, and other related challenges when protecting organizational systems. This study compares various methods of LLMs and neural network models to define the best to use in identifying vulnerabilities at an early stage of the software development processes.

The trends toward constantly growing implementation of relatively new technologies such as cloud computing or AI and the development of the Internet of Things drive this thesis. Although these promising technologies bring innovation opportunities, they also bring new, unexpected, and unknown security threats [16]. Increased exposure and range of these technologies suggest that current vulnerability assessment approaches need to be revised. The significant advantage of open-source LLMs is that their usage allows the application of a more individualistic approach to detecting vulnerabilities. At the same time, one must note that the main advantage of using proprietary LLMs is their ability to optimize performance and the usage of the newest technologies in the domain. This research aims to analyze the possibilities and inadequacies of LLMs and neural networks in providing solutions to the security threats caused by modern technologies.

A significant aspect of this thesis is addressing the gaps in current development practices. Modern software development involves integrating legacy systems, new applications, and third-party components, all of which can introduce vulnerabilities. The lack of collaboration between development and security teams often results in vulnerabilities being detected too late in the development process. The motivation here is to leverage LLMs and neural network models to enable a shift-left approach in software development, embedding vulnerability detection and mitigation from the early stages of development [17]. This approach aims to create a seamless integration between development and security, reducing the time and resources needed to address vulnerabilities.

The cybersecurity talent shortage is evidence of a need for more automated solutions and a gap between the supply and demand of cybersecurity specialists, which makes companies need help attracting and keeping talent to deal with the rising complexity of cyber risks [18]. Such a state of the system results in burnout in both the individuals involved, high turnover rates, and more vulnerabilities exposed because of the issues that have not been patched. In this work, to decrease the usage of the manual approach and overall efforts of the existing cybersecurity teams and to minimize human factors by utilizing open-source and proprietary LLMs with neural network models, find the optimized solution for reducing the manual process and increase the automated process and novelty the optimized solution for code vulnerability detection methodology.

Furthermore, the need for more effectiveness of the current vulnerability detection software, including those that AI enhances, is another drive for this thesis. Automated code tools interacting with AI can increase the speed of software development but usually cannot develop new solutions or interpret context to point at more intricate risks. In addition, using AI-derived code can bring in new kinds of vulnerabilities if it is not tested. In other words, this research aims to arrive at a more elaborate and refined approach to identify the vulnerabilities with the strengths of AI models strength.

In conclusion, this thesis explores the comparative effectiveness of open-source and proprietary LLMs and other neural network models in enhancing vulnerability detection and mitigation. By identifying the best-fit models and techniques, this research aims to provide actionable insights for developing automated tools that can proactively detect, categorize, and address software vulnerabilities. This approach will ultimately contribute to more secure software development practices, reducing the risks of deploying complex systems in an increasingly interconnected digital landscape.

1.4. Project Goal

This research aims to determine the best performance method for vulnerability detection based on AI, applicable to C# source code. This will use an open-source data set of C# codes, some vulnerable while others are not vulnerable. This dataset will help train machine learning (ML)

models that will learn quickly how to identify patterns and characteristics of security vulnerability from the code. The following steps will be taken to achieve this goal.

i). Review of Current Research and Techniques

There needs to be more vulnerability detection techniques in C# programming language using ML models. Hence, a detailed review of the existing knowledge on vulnerability detection will be performed. This will include researching general tools, approaches, and AI models available for source code analysis and vulnerability identification.

ii). Dataset Analysis

Conduct exploratory data analysis on various variables of a massive, easily accessible C# source code. This data set will contain vulnerable and non-vulnerable code samples, so there will be multiple cases when training the ML model.

iii). Data Preprocessing

Transform the given C# source code in the dataset into a form that could be fed into a ML model. This will include primary data preprocessing, for example, in the next steps of tokenization, normalization, and conversion to feature representations that the models can recognize.

iv). Model Training

Prepare the data set in a way that can then be fed into different ML algorithms and different types of neural networks. The models will train the concept of understanding which portions of code are 'Vulnerable' and which are 'non-vulnerable' by learning patterns and signs inherent to particular security vulnerabilities.

v). Vulnerability Prediction

Apply the trained models to identify novels, test C# code, and classify the given code as containing possible security vulnerabilities. This step will be taken to deploy the models into a usable tool or valuable schema for developers and information assurance operatives to investigate vulnerabilities in automated systems.

vi). Performance Evaluation

Examine the trained models' performance using particular measures such as the evaluated model's F1-score, accuracy, precision, recall, and AUC-ROC. This evaluation will be necessary in determining the models' strengths and weaknesses and the areas that require improvement.

All these steps are the long-term objectives of this project, which seeks to enhance the software security field with an adequate AI-based approach for automatic source code vulnerability identification. The best practice is to improve the effectiveness of the vulnerability assessment activities and offer developers tools to aid in developing better-secured software systems.

1.5. Contribution

This thesis makes significant, valuable, and unique contributions to AI power code vulnerability detection by analyzing the classification strengths of open-source, proprietary LLMs and neural networks. Indeed, it can reveal the behavior of LLMs and models' neural network capabilities to understand source code vulnerabilities better. The research proposes a relatively unexplored yet complete comparison and integration approach by leveraging the unique strengths of each model type. This provides a novel understanding of utilizing several LLMs effectively within vulnerability discovery tasks. The contribution includes significant overall processing of datasets containing both vulnerable and non-vulnerable source code. This dataset attempts to provide a balanced, complete training set for ML architectures to uncover vulnerability patterns from within the code. The first set of processing procedures transforms the dataset to a representation that complements the models' learning capabilities and is critical for enabling high performance of vulnerability detection.

Our research includes using and optimizing several ML algorithms and one of the latest LLMs. The C# dataset was preprocessed to match the highest performing architectural models and hyperparameters so that they can attain the best results from models that will be trained on it. It also details how finetune LLMs can be improved Low-Rank Adaptation (LoRA)and reviews the open source and proprietary LLMs used in code vulnerability detection. The thesis presents an extensive comparative study of these models in terms of their status of benefits and drawbacks and discusses their applicability to vulnerability detection tasks. This effort generates an essential resource for researchers and specialists wanting to realize the features and limitations of the different LLMs and enable the exploitation of LLMs' potential in identifying vulnerabilities in source code.

Based on these capabilities and findings, Engineers or developers can develop automated tools to identify vulnerabilities in the C# code. These tools will use the learned models to identify real-time code analysis and feedback on this information to the developers before going to the software systems production. It can provide integration with the current development environments, which makes the tool more practical and feasible to use in actual software development.

Additionally, the evaluation of the model is presented with all of the F1-score, recall, accuracy, and precision factors of the model. It also provides the specifics of the vulnerabilities that are hard to identify. Apart from proving the efficiency of the suggested approach and pointing out

the areas that require more research and development, this analysis reveals trends for further study in the sector, especially LLMs and other AI models.

Finally, the thesis offers guidelines and recommendations for future research in automated vulnerability detection. These recommendations are based on the lessons learned during the study and provide a roadmap for exploring new methodologies, improving model performance, and extending the framework to other programming languages and vulnerabilities. By doing so, this thesis lays a strong foundation for future studies and advancements in creating more secure software systems.

1.6. Existing Challengers

Code vulnerability identification remains one of the most critical tasks and, simultaneously, the most challenging one in the SDLC. One of the primary challenges is the constant and detailed inspection of code to identify issues that may be required because of multiple entry points that can lead to possible vulnerabilities [19]. In the case of software, the vulnerabilities are also dynamic due to the constant updates and other changes made to the code. Each modification, even minor changes, could introduce a new code that makes the code susceptible to new threats. While code analysis and review are essential to development, traditional manual code review is time-consuming. It demands extensive knowledge of IT security principles, and the programming language used for building the software. There are many reasons for this, and it has become even more complicated with the growing complexity and integration of software systems nowadays, where vulnerabilities may be hidden in what appears to be a simple piece of code. Such vulnerabilities are then identified based on the knowledge of the programming language and the ability to look at the code from an attacker's perspective, something that not all developers have. It needs more experienced developers or engineers.

Another critical problem is hidden vulnerabilities that generally cannot be visible in the system unnoticed for a long time because attackers can use these weaknesses before the bug is discovered and fixed. Open sourcing of software, reuse of code, and even everyday activities such as forking on repositories such as GitHub [74] ensure that the difficulty of vulnerable code is spread extremely fast, thus making it almost impossible to track and mitigate a vulnerability once it is out of there [20]. More than this, the adage in cybersecurity is that the defender must secure everything. At the same time, the attacker must crack only one point, which is a strategy that poses severe challenges for developers. In cybersecurity, defenders face the tough job of protecting every possible weakness, while attackers only need to find and exploit a single vulnerability. This imbalance creates significant challenges for developers trying to build secure systems.

There is also no systematic and comprehensive method of defining and identifying all the vulnerabilities. Typically, vulnerability identification has relied on the experiences of specialists who determine the features of vulnerability. However, this approach is inherently subjective,

problematically qualitative, and likely inconsistent. While it results differently from another expert, one may focus on various aspects of the same idea, which leads to getting these people to contribute, which takes a lot of workforce and planning. Additionally, static analysis tools commonly used today, such as FindBugs [21], Splint [22], Flawfinder [111], and PyT [112], follow a rule-based approach and often suffer from high false positive rates. This functionality is limited due to overload. Most of these are false alarms whenever a developer gets results and needs more confidence in these mechanisms.

Considering all these challenges, there has been increased interest in using data-driven approaches focusing on ML & AI-based techniques. Compared to traditional methods that apply previously defined decision-making rules and experts' knowledge, the pattern is obtained by ML-based approaches based on large data sets containing vulnerable and non-vulnerable codes. This helps this system to identify vulnerabilities more accurately and without the need for influencing expert knowledge. In addition, ML models can be retrained repeatedly any time new types of vulnerability or exploitation are recognized, therefore possessing higher adaptability to new threats. Automating vulnerability detection through ML reduces reliance on human expertise and allows for early detection in the software development lifecycle, significantly lowering the cost and effort required to fix vulnerabilities.

However, some research shows that the usage of ML and AI-based methods still needs to be improved. Some research funded by IBM indicated that even though various automation techniques can effectively reduce such flaws, most organizations apply them insignificantly or not [113]. This suggests a significant difference between the capabilities of such technologies and the extent of their current adoption in the industry. To improve this issue, it is imperative to create better, more efficient, accessible ML models and tools for detecting vulnerabilities. These innovations could revolutionize security processes in organizations, better defending them against the multiplied number of cyber threats.

1.7. Organization of Thesis

The remaining sections of the thesis are organized as follows.

Chapter 2 explores the background theory surrounding secure software development, tracing its evolution and significance in modern software engineering. This chapter addresses the objective of establishing foundational knowledge for secure software practices and the importance of integrating security into the Software Development Lifecycle (SDLC) from initial design to deployment. Various vulnerability detection techniques are examined, including manual code reviews, SCA, and dynamic testing, each offering distinct advantages in identifying security flaws. We investigate how collaborative security approaches can help improve the whole process. The chapter also discusses future directions related to secure software development, focusing on trending and emerging technologies to enhance software resiliency against threats.

Finally, we review related work on source code vulnerability detection research that is available to date.

In Chapter 3, we elaborate on the related work of this research and analysis. We provide a detailed examination of the related work that forms the foundation of this research. By analyzing existing studies on source code vulnerability detection and secure software development, we aim to highlight the field's progress and recommend crucial areas for further research. The chapter identifies the research gaps that emerge from the review of existing work. This directly relates to the aim to identify gaps in current research through studies and methodologies, specifically in use of both LLMs and NN. These gaps highlight areas where additional research is needed, particularly in leveraging LLMs and neural networks for more accurate and efficient vulnerability detection.

In Chapter 4, we outline the methodology used for the research, from the research design to the justifications for selecting the specific models used. This chapter starts with the goals of the comparative analysis, which is to compare the performance of diverse models for secure software development. We give a detailed explanation of the model selection process and the experiment setup. The architecture of each model is thoroughly investigated, exploring how their architecture affects both vulnerability detection and overall performance. The chapter concludes with a performance analysis of the characteristic parameters and metrics used to compare the models against each other and with an experimental assessment of effectiveness. The methods used in the study are correctly understood as this is a structured approach.

In Chapter 5, we detailed the implementation process for source code vulnerability detection using various approaches. This chapter addresses the objective of applying and evaluating different approaches and first, we outlined the dataset preparation process, including data collection, preprocessing, and labelling for vulnerability types. This dataset was then used to fine-tune open-source LLMs, such as CodeGen2 and LLaMA2, where we described the training configurations, hyperparameters, and optimization techniques applied. For proprietary models, we demonstrated the process of making API calls to GPT-based models (GPT-3.5, GPT-4), detailing how these models were used for vulnerability detection without fine-tuning. In addition, we implemented a traditional neural network approach using Word2Vec embeddings and LSTM to provide a baseline for comparison. The chapter concludes by presenting and analyzing the results, displaying performance metrics through graphs to compare the effectiveness of open-source LLMs, proprietary models, and the LSTM-based neural network in detecting vulnerabilities.

In Chapter 6, we extensively compare the models we employed to discover vulnerabilities in source code. The analysis includes two open-source LLMs, a proprietary LLM, and a neural network with two input formats: plain and AST. This chapter directly addresses the objective of analyzing the strengths and weaknesses of these models across different vulnerability detection use cases. It supports the goal of identifying optimal strategies for source code vulnerability detection.

In Chapter 7, we conclude the comparative analysis by focusing on the different model's ability to address code vulnerabilities. The discussion on future work aligns with the objective of proposing enhancements to improve scalability and accuracy in real-world applications. This work discusses refining the training datasets, introducing a broader range of programming languages, and developing techniques to limit the computational cost of the program.

CHAPTER – 2

Background Theory

2.1. Overview

The NIST [114] defines vulnerabilities as defects in the structures of information systems, procedures under which software is developed, or implementations that allow attackers to take advantage of them. Attackers can exploit such defects to perform damaging actions such as modifying or reading restricted data and interfering with operations or control systems, even though they are not considered significant concerning software maintainability. This implies that vulnerabilities are an uncommon kind of software defect, it is an analysis of what proportion of all software defects can be classified as identified vulnerabilities of general varieties of defect. Finding vulnerable code is still tricky despite feature recognition problems such as correctly identifying flawed and safe code. Some of these are software metrics or using an anomaly detection or pattern recognition system. For instance, one example of this is that vulnerability means we can identify it by patterns such as missing input validation or code complexity issues. With higher levels of software development for added qualities, such as in systems integrated with third-party components or particularly with poorly updated code, the probability increases that such defects will be proven. To reduce these risks, the software industry pays particular attention to the principles of secure code generation. It uses tools and techniques for static analysis, dynamic analysis and collaborative approach. Such approaches are necessary even with the modern world relying more on software systems.

2.2. Evolution of Secure Software Development

From the latter part of the twentieth century onwards, the concept and the method of secure software development occurred to address the rising incidence of cyber risks. Where there is no direct regulation in place to ensure code quality, one of the first significant steps towards better coding practices is writing guidelines and standards that would regulate the code being written to minimize risks [116]. Some organizations, such as the OWASP [115] and the SANS Institute [117], created lists of vulnerable applications and ways of writing secure code, such as input validation, error handling, and proper authentication. These guidelines helped the developers to write more secure codes, but they were optional and needed a well-defined framework. The coming of the Secure Software Development Lifecycle (SSDLC) marked a significant turning point. Traditional SDLC is more focused on design, implementation, and testing, while SSDLC involves security in all its phases, including requirement gathering, design, implementation, testing, deployment, and maintenance [12]. In this way, security takes place at the first stage of a project and in all project stages to minimize security vulnerability. Technology giants like Microsoft first developed the model with the SDL, which laid the foundation for the industry.

The SDL introduced a concept of threat modelling, security code review, and strict security testing practices, all of which other companies began to follow.
Technological advancement made the threats more complicated, and better solutions were needed. In this era, they developed automated tools for code analysis and static and dynamic analysis that allow the developers to capture the flaws that likely impact the software system in the early software development process without much manual work. Static analysis tools analyze source code without running code to identify security defects. However, dynamic analysis tools are based on code execution and help discover vulnerabilities that static analysis cannot notice. Combined, they would provide a more proactive approach to vulnerability detection and mitigation with the help of automated testing environments.

Adopting the Continuous Integration and Continuous Delivery (CI/CD) approach also amplified the trends towards agile and DevOps towards further evolutionary changes towards secure software development. Security has integrated DevSecOps into the CI/CD pipeline to adapt to this shorter development cycle. DevSecOps is an attempt to ensure everyone participates in the security process at all stages of development and enforces security automation at each stage. By doing this, the security issues are vulnerabilities fixed and resolved as fast as they are in the current dynamic software development [119].

With the new AI and ML trends, new opportunities have been opened to enhance the safety of software development. Therefore, AI-driven tools can recognize complex code patterns, guess problems ahead, and identify immediate security flaws. The key is that these have allowed management of the rapid growth and increasing complexity of these software systems, which characterize today's software systems, while simultaneously fulfilling these systems with very strong security measures.

Generally, the evolution of secure software development facilities goes from response to threats to prevention, considering security in the SDLC. As the digital world becomes more prominent, the train of secure software development will continue to evolve, hunting for new threats and new technologies to protect the software systems' integrity, confidentiality and availability.

2.3. Security Integration in SDLC

The current trends in cyber incidents and attacks on software systems have revealed a significant issue in the SDLC, adopting security from the beginning of development. Traditionally, the SDLC has been concerned with providing helpful, effective software but only giving enough attention to security matters once vulnerabilities are noticed after implementation. In the present threat landscape, reacting to exploits is ineffective as attackers utilize software vulnerabilities to compromise the data and harm data, services and organizations [118]. On the other hand, security practices in the SDLC have been realized to play a significant part in software development today. Given that vulnerabilities can now cause disastrous effects such as leaks of data and severe disruption of their core systems, software security has become a critical

problem. One of the significant causes of software weakness causing a system to fail in ways beyond its prediction is not integrating security at each step through the development cycle. Many traditional development approaches do not consider the possibility of the system being attacked intentionally or maliciously. Hence, they are vulnerable. Therefore, security must be integrated into the SDLC to develop secure applications that handle different threats.

Figure 2.1: Secure Software Development Lifecycle.

The objective of the security integration is to ensure that security is incorporated at every stage of the SDLC, which includes the requirement gathering, design, implementation, testing and maintenance. This approach is called the Secure Software Development Life Cycle (SSDLC), which prevents the incorporation of possible security threats into the final product. Threat modelling is an example of something we can apply in the design process to locate security risks that might be present and how they might be mitigated. At the time of coding, Secure coding practices ensure the most widespread vulnerabilities, such as cross-site scripting, SQL Injection, etc. Security testing can be achieved using code analysis, such as static and dynamic approaches.

To tackle this problem, several frameworks and methodologies were developed to aid in managing the inclusion of security in the SDLC. Microsoft's Security Development Lifecycle is a pioneering one, which was brought forward as part of the Trustworthy Computing Initiative. SDL is a process that integrates security measures into the development of software life cycle across threat analysis, secure coding practices and techniques and security testing. Because of

its consistent approach to software development integrated with security, the SDL has been implemented widely and has become the reference model for many other organizations.

The Comprehensive Lightweight Application Security Process (CLASP), by Secure Software and later merged with OWASP, is another well-known approach [120]. The twenty-four generic, composable security tasks presented by CLASP can be quickly tailored to fit into SDLC, and the CLASP framework is general and extensible enough to incorporate security tasks [121]. SDL is more formalized and better suited for larger organizations, while CLASP is more lightweight and more suitable for smaller organizations with less strict security needs.

Another framework that can be highlighted is Cigital's touchpoints, which also focus on integrating security in the specific set of best practices and activities that can be applied throughout the SDLC [120]. This approach may be defined as ongoing risk analysis and enhancement by incremental steps to ensure that security measures evolve alongside the software.

However, specific challenges arise when implementing an SSDLC. Among the most critical challenges, cultural change is needed within the development teams to adopt security as a primary concern along with the functional and performance requirements [122]. Security is usually associated with more resources, time and expertise, which may be seen as obstacles to rapid development. However, measures like code review, vulnerability testing, and risk assessment are critical in enhancing the security of the application, and these practices are likely to require specific tools and skills that may lead to an increase in the cost of development.

However, the advantages that can be gained from SSDLC in the long run are much more than the costs of initial investments. Organizations can reduce maintenance costs, avoid costly breaches, and protect their reputations by identifying and addressing security vulnerabilities early in development. Some studies reveal that the cost of remediation is much lower during the design or coding phase than after the deployment of an application [123].

Integrating security into the SDLC will become increasingly essential as cyber threats evolve. However, the SDLC gradually incorporates new technological tools such as AI and ML to enhance security. Another trend is DevSecOps, where security is applied to DevOps methodology and applications for continuous security testing and monitoring are developed.

Therefore, security should be a necessity rather than an option in today's world of software development. Tools such as SDL, CLASP, and Touchpoints are helpful for organizations that would help implement security in the development process to develop robust and secure software resistant to ever-increasing cyber-attacks [120]. Thus, when it comes to this, the SDLC continues to change with technological advancement and new threats for security to be a vital part of the SDLC.

2.4. Vulnerability Detection Techniques

Source code review for vulnerabilities is one of the most important methods of software security. It is used to identify potential threats which an attacker may use to gain unauthorized access to, and modify or degrade, the system. Since the software systems are increasingly complex and the connections are growing, risks for vulnerabilities must be addressed and eradicated before the software systems are rolled out. In today's world where it is flooded with different forms of technology, vulnerability detection techniques are essential and must be as effective and efficient as possible so that no attacks can happen and our system can remain secure. Static, dynamic and hybrid techniques are the standard source code vulnerability methods and each of them contains their own way of doing things, their strengths and weaknesses.

2.5. Manual Code Review

Experienced developers, security personnel, or software engineers do manual code reviews to identify possible security risks which other automated techniques cannot detect. Manual code review is unlike the different kinds of analysis as it’s a more thorough view of the code structure, coding logic, and design so the reviewer can leverage his knowledge and experience to discover the security dangers, coding blunders, and design problems that might exist in the code [124]. It is a valuable approach, especially when discussing large-scale software projects where patterns of integrated code as overly complex interdependencies between the codes make it almost impossible for automatic tools to detect such potential threats [125].

Manual code review is one of the main tools that assist in finding complicated security vulnerabilities in the software. This supports a customizable method, which allows reviewers to choose their analysis techniques according to the complexity of the code. It’s beneficial for code with specific business logic or needs frameworks. Manual reviews enable continuous learning, getting the team security knowledge to improve and enforcing security practices. Moreover, they consider business context and design choices to embed security concerns into the design phase. This also keeps the communication open, of course, between reviewers and developers with immediate feedback, allowing them to discuss improvements and security enhancements in the code constructively.

Code review is laborious and time-consuming work due to a slow pace and the need for human resources. Furthermore, it is also subject to subjective and inconsistent results based on the reviewer’s expertise. As projects grow, scalability issues mean keeping security assessments consistent across large and dynamically changing codebases is too tricky. In addition, human error is an issue because it is easy to overlook subtle vulnerabilities when fatigued or cognitively biased, as reviewers may be. The manual review can be of great quality depending on the reviewer's skills and knowledge.

One of the most efficient and effective software security measures is manual code review and while more than one automated tool can substitute the human factor, there is still much manual code review can do [124]. Identifying complex vulnerabilities and improving team learning are just two of the benefits it has. However, on the reverse side, the problems of time, human error, and scalability are the challenges that came with the method. An attempt to combine the strengths and weaknesses of manual code review with automated analysis to develop a better and more efficient method of ensuring software security.

2.6. Static code analysis

SCA is performed on the program's source code without executing the program. This also checks if the source code has any security issues before deploying it into production, this is because the source can cause many security risks so it is better to check if the source code is clean and secure or not, and for this reason, source code is checked using the SCA result, this approach utilizes a range of techniques, including control flow analysis, data flow analysis, taint analysis, and rule matching which makes the process best suited to uncover problems related to access control, APIs misuse and information flow. Static analysis tools can discover most of the code's vulnerabilities at the development stage before the code is executed, improving software products' quality and security [126].

The major strength of static analysis is that it can deliver immediate feedback on code quality issues such as performance issues, error-prone APIs and deadlocks. It is beneficial in detecting generic vulnerabilities and, thus, helpful in the SDLC process. Static analysis helps automatically detect vulnerability and, therefore, saves time that otherwise would have been used in manual code reviews to identify such vulnerabilities. Additionally, the static analysis tools are flexible in that developers can use them at different levels of the SDLC to continuously evaluate the quality of the codes [127].

Nevertheless, static analysis has several apparent drawbacks [128]. A significant limitation of the approach is that it cannot understand the context of a program at runtime and, hence, can give rise to false positives and false negatives. False positives happen when the tool labels the standard code as a vulnerable code, and developers spend time analyzing the reported issues, which are not the issues. On the other hand, there are false negatives, which happen when the tool misses a vulnerability that is indeed there, meaning that essential security holes will not be closed. This problem is worsened in complex or, especially, not well-documented code bases when static analysis tools cannot accurately evaluate the security threats. Moreover, the performance of static analysis tools is directly linked with the quality of the rules and patterns used in them and as the software develops. There is an increase in potential vulnerability, and modifying these rules can be time-consuming and challenging. There may be tools that cannot detect a vulnerability if there is no known pattern for it.

Another major area for improvement of static analysis is its inability to manage large and incredibly complex codebases. While it is highly effective in identifying common issues and

improving code quality, its limitations in accurately assessing runtime behavior, high false rates, and dependency on predefined patterns underscore the need for complementary techniques.

2.7. Dynamic Code Analysis

Dynamic Code Analysis (DCA) is the kind of analysis performed on the software to identify its vulnerabilities while the application is running, unlike most of the SCAs performed on the code without the need to execute it. Unlike SCA, dynamic analysis identifies vulnerabilities in the software when executed and is most efficient at uncovering issues that may not be seen in the static code review phase. This involves installing the application and simulating the activities, which include inputs, touches and clicks, to determine the vulnerabilities the attackers could exploit [129].

Fuzz and Search-based testing techniques are the most frequently used DCA processes. These techniques differ in their approach to test generation, test efficacy, and the breadth of vulnerability coverage. Dynamic analysis has far more benefits than static analysis, for example, it can detect vulnerabilities overlooked by providing an overall wider security net [130]. DCA is more helpful in diagnosing runtime environment risks, and it can be fitted to search for specific threats, even where the source code is hardly accessible. Further, dynamic analysis is good at identifying the emerging vulnerabilities that stem from the interaction of different components in a software system that might be difficult to detect during static analysis.

However, some things could be improved in DCA. It requires more computational capacity and can be rather time-consuming, mainly in the case of large-scale applications. Usually, dynamic analysis requires expensive tools, and the result obtained from such automated tools contains a significant percentage of false positives and false negatives. Hence, it must be checked manually, which adds to the time and cost. As mentioned earlier, dynamic analysis is based on predefined rules, so the quality of the specific rules used determines the efficiency of the analysis. Also, it may be challenging to identify where precisely the vulnerability is within the source code.

DCA is usually done in contexts where an application is operational, like on a production site. The debugging process checks the operation of code at the time of execution before or after the program has run. This helps developers know the weaknesses that can only be noticed after the application software has been released to production. DCA is effective in runtime vulnerabilities such as memory usage errors, null pointer dereferences, pointers, and out-of-the-bound read/write that mainly arise from business context or environment changes [131].

DCA's first and foremost aim is to identify problems when the program is being executed, when functions are being invoked and when variables are being operated upon, not the standard approach of analyzing each statement individually. Some investigation techniques used in DCA are Regression testing, Growth analysis, State capture analysis and stability detection. These

methods are helpful to keep the software safe and proper at the development and integration stage and in combination with other facilities.

Overall, dynamic analysis can be considered a robust method for runtime vulnerability identification. Still, it is sensitive to the computational capabilities, the strength of the rules, and the analysts' experience in correctly identifying and considering security issues in modern software systems [131].

2.8. Strengths and Limitations

Manual code review, SCA, and DCA are three distinct approaches to identifying and mitigating software system vulnerabilities, each with unique strengths and limitations. While manual code review can detect complex logic errors and context-specific issues, it is slow, prone to human error, and cannot scale to a large codebase [132]. SCA, which runs code in a dry run without executing it and has scalability and automation to quickly detect syntax errors, security vulnerabilities, and compliance violations. At the same time, it can produce false positives and does not detect run-time issues or vulnerabilities depending on dynamic states. However, DCA runs the application while it is executing. For example, it can reveal vulnerabilities that only follow the application when it is run, such as a memory leak or race condition. Dynamic analysis is highly effective at locating runtime vulnerabilities but requires a complete application to work with this approach. It is of limited use before certain stages of development, and it cannot identify some structural code problems.

2.9. Collaborative Approach

When dealing with other security methods, it is possible to cover vast areas and make security testing processes more efficient. It will help address most software application vulnerabilities if combined with other methods such as DCA and manual code review. SCA can also help filter and target security testing by integrating different techniques and ensuring that the most security-critical items are first fixed. Furthermore, it can help define which areas of the code require more work so that resources for the manual review and testing should be directed most appropriately. On balance, integrating static analysis with other security testing types enhances the completeness and effectiveness of security testing, thus enhancing the development of a secure application [133].

2.10. Future Directions

In today's fast-paced world in which digital development is skyrocketing, the requirement for secure software development practice is a necessity. Software vulnerabilities have become the favorite attack vectors of cyber criminals, and cyber threats are on the rise. Consequently,

software communities involved in development must be vigilant in improving security procedures, including SCA, the most used secure coding practice. With the increasing complexity of software applications, SCA will continue to develop, and several trends will define the future of secure software development.

AI and ML are two promising areas that can be considered the most important future trends of SCA. Traditional static analysis tools rely on predefined rules and patterns to detect vulnerabilities, which can result in false positives or fail to detect complex security threats. AI and ML models can learn from large datasets of known vulnerabilities and apply this knowledge to new threat patterns that may not be defined in static rule sets. This adaptive learning approach helps detect with high accuracy, decreases the possibility of false alarms, and enhances the overall analysis. Some of the more sophisticated automated tools can also categorize the vulnerabilities according to their level of severity, thus allowing developers to tackle the most severe ones first. In the future, these models will improve based on the feedback and real-world applications of SCA and thus help improve the proactive approach to security. SCA can be integrated into CI/CD processes, allowing developers to detect and address vulnerabilities in code as it is developed and tested.

Including static analysis as a part of CI/CD pipelines helps the development process work efficiently as it gives immediate feedback to the developers about the security vulnerabilities that can be fixed at that moment. This approach not only increases security but also decreases the cost and time required to fix vulnerabilities at a later stage. The next generation of static analysis tools must be fully integrated with the primary CI/CD tools such as Jenkins, GitLab, and GitHub [74] Actions.

While traditional static analysis tools are great at identifying syntax errors, hard-coded login credentials, simple SQL injection, or XSS Attack vectors, they lack the dynamic behavior of the application. However, they fail to address the more complicated security problems such as business logic flaws that result from code that performs actions that are not expected by the developers yet can be manipulated by hackers. This is why future static analysis solutions must focus more on discovering these vulnerabilities. These tools, using data flow analysis, symbolic execution and understanding the context of application logic, will be able to detect the flaws in the business logic which traditional analysis techniques will not recognize thus, it will enhance the ability of developers to protect applications against different attacks.

As new programming languages and frameworks, there is a need for static analysis tools to support them. The migration towards Rust, Go, and Kotlin, which are secure by default, allows the static analysis to grow parallel with these languages. Upcoming tools must address these languages and frameworks and provide adequate security regardless of the stack used. Also, a growing number of applications will be developed using microservices, serverless, and cloud-native architectures and will require that the static analysis tools are also designed to offer security checks relevant to these distributed systems.

SCA is not only a technique to find issues, but also a way to teach developers. The future of secure software development is to feed static analysis results into code review activities and encourage developers to become more knowledgeable and aware of the risks. Future static analysis platforms will guide developers of specific contexts, code snippets, and best practices that will assist them in identifying why a specific vulnerability is undesirable and how it can be avoided. Thus, integrating security learning into the SDLC can decrease the number of issues leading to vulnerabilities.

The usage of open-source libraries and components is increasing, bringing new risks because the vulnerability within the dependencies will affect the whole application. SCA tools of the future will rely more and more on open-source security, whereby third-party libraries will be automatically scanned for vulnerability and license checking. This focus will be very important in managing the security of today's applications built on hundreds of open-source dependencies.

Due to increasing concerns about data privacy globally, and other industry regulations, compliance has become mandatory in software development. Future static analysis tools will integrate compliance checks into the tool to check whether the code complies with specific security and privacy regulations. These tools will assist organizations in preventing expensive fines and preserving the confidence of the users.

In the future, the advancement of SCA tools will help developers protect the software against new and emerging threats and ensure the development of secure, robust and compliant applications. Thus, by adhering to these innovations, the software industry can enhance its ability to safeguard its consumers and preserve the quality of its software products in the rapidly changing threat landscape.

CHAPTER – 3

Related Work

3.1. Literature Review

This contains works related to this thesis's main topic. It discusses the research that is available in identifying sources of vulnerabilities in source code using AI, ML, DL and LLMs along with other works that are related to identifying vulnerabilities or exploits that can be used as reference to give a broader understanding of the topic. This literature review is critical in providing a background of the first stage, and further implementations will be discussed in the following chapters.

3.2. Traditional AI Approaches

This research focuses on the static analysis methodology using some ML approaches, so the following studies engaged different static analysis deep learning and ML methods to detect vulnerability in source code.

i). Traditional ML techniques

There were some ordinary ML techniques adopted in those research [23]-[47], and traditional techniques like Naive Bayes, Random Forest, Decision Tree, and Support Vector, and Linear Regression were used.

Exclusively, those [37][24] a Support Vector Machine (SVM) is more suitable as it is not affected by overfitting when the issue is associated with the high dimensional spaces of variables. In the same way, [33] a working SVM and a linear kernel are used to train the model. Other research [36] used the decision Tree and Lasso and regression to train the models, and the study [23] found that Random Forest outperformed the others in estimating cross-project vulnerabilities. In contrast to prior works, the author [40] trained their models using both semi-supervised (Co-trained Random Forest) and supervised (Random Forest and Linear Regression) techniques because a significant portion of the data was not categorized. Trained Naive Bayes, Random Forest algorithms, and SVM using text-based features [46]. More research [48]-[60] developed models using various ML methods and evaluated their effectiveness. Automatic Detection and Correction of Web Application Vulnerabilities research [56] evaluated numerous variations of the SVM, which proved to be the most successful classification method in their investigation compared to Random Forest, Decision Tree Naive Bayes, K Nearest Neighbors, and Linear Regression models. Similarly, [57] [59] research used numerous ML approaches when training vulnerability detectors and realized that the SVM offers the best efficiency rate.

Unlike the previous research, [52] findings indicate that K Nearest Neighbors yields the most promising outcomes following training on their dataset using Deep Neural Network (DNN), K Nearest Neighbors, Decision Tree, SVM, Linear Regression, Naive Bayes and Random Forest. This research [58] tested the Decision Tree, SVM, Bayes, Naive Bayes, Stump, and Ripper models to determine the best model for vulnerability detection, their findings indicate that the SVM is the most reliable model for detecting potential security breaches. Similarly, [51][55][53] associated several ML methodologies and discovered that Decision Tree and Random Forest performed better than other techniques.

ii). Deep Learning techniques

According to the deep learning approach, plenty of research [61]-[70] used Recurrent Neural Network (RNN), Artificial Neural Network (ANN) and Convolutional Neural Network (CNN) to use their training models. In more detail, the BP-ANN technique was used to perform vulnerability detectors [70]. Trained an array of LSTM models on data for their Achilles project [68]. It contains Java code snippets for a particular collection of vulnerabilities. An additional investigation proposed a DL framework for training vulnerability detectors using RNN models [63]. Using a sequential approach, the authors feed the code embeddings into a BI-LSTM model to capture the feature semantics, retrieve the vector weights via an attention layer, and then evaluate if a code is safe or susceptible by feeding it into a dense layer.

iii). Hybrid and Emerging Techniques

This research [71] investigated classic ML and DL algorithms, contrasting previous investigations focused on one. They utilized classic ML techniques like Random Forest, K Nearest Neighbors, SVM, and Linear Regression with BI-LSTM, GRU, and CNN, and the best-performing model, according to the results, is BI-LSTM. Created a benchmarking framework that supports BI-LSTM, LSTM, BI-GRU, GRU, DNN, and Text-CNN but may be expanded to deep learning models [65]. Authors [64] created graphical semantics based on code semantic properties and trained them using Graph CNNs.

iv). Specialized Frameworks

Further detailed analysis was conducted on selected research papers, focusing on frameworks designed for real-world usage. The analysis involved examination of the methodologies, experimental setups, and results presented in each paper. This in-depth review aimed to uncover key findings, trends, and gaps in existing literature and provide a comprehensive overview of the current state of research in this field.

The Leopard framework created by Du et al. [26] does not rely on known bugs and uses an SVM, Naive Bayes, and a Decision Tree to detect such vulnerabilities. The Leopard framework for detecting vulnerable functions in C/C++ applications without prior expertise. They tested eleven real-world projects that covered seventy-four percent of the susceptible methods and

offered three applications for manual code review. This framework discovered twenty-two new issues, including eight new vulnerabilities in applications such as Radare2, FFmpeg, and PHP. To enhance the contextual appearance of source code tokens for ML techniques, The authors proposed a guided approach that utilizes trained embeddings from context-aware language models, like ELMo. They tested RNN, NN, CNN, and Ensemble models.

This study introduced VulBERTa [75], a model based on DNN for finding vulnerabilities in source code. The suggested DNN trains source code representations of knowledge at both the syntax and semantic levels. The datasets Vuldeepecker [76], Draper [77], Reveal [78], and muVuldeepecker [79] were used to evaluate their suggested methodology for discovering vulnerabilities in binary and multi-class settings. They experimented using static analysis methods to identify known vulnerabilities.

This research led to the creation of the Developer vulnerability framework [80]. It is a unique learning approach for creating source code weakness recognition models. To solve the issue of data shortages, the Developer automatically acquires training sets from open-source projects. The collected data comprises a variety of real-world susceptible source code training models that supplement those found in traditional vulnerable databases. To create a robust code vulnerability assessment model, the Developer uses an architecture of CNN and approaches to gather code representations from the abstract syntax tree (AST). They utilise the developer to uncover vulnerabilities in program source code, and evaluation shows that the Developer outperforms cutting-edge methodologies by detecting more vulnerabilities with fewer false positives. This study investigates the capability of Highway BiLSTM Networks for model identification. They compared six neural networks, including CNN [81], LSTM [82], BiLSTM [83], GRU [84], BiGRU [85], and Tree-LSTM [86], along with the developer.

AI and Blockchain-based source code vulnerability detection [87], this solution targets multi-party software development environments, particularly those with work-from-home setups where developers lack constant supervision. A hybrid AI-blockchain architecture has been developed to help identify and address vulnerabilities in multi-party development of software projects. During the testing phase, an AI technique is developed to find vulnerabilities in source code. A blockchain-based secure solution is presented to boost confidence and clarity in standard SDLC testing methodologies. This study used LSTM and Bi-LSTM models to detect code vulnerabilities. This uses four datasets, including vulnerable and non-vulnerable C and C++ codes.

This research [88] examines how Graph Neural Networks might help identify source code peculiarities from a security standpoint. They rely on code written in C. Source code freely lends itself to graph models in various forms, including AST, control flow graphs, data dependency graphs, etc. The study aims to determine if a NN can recognize and learn the links between nodes and edges in a source code graph and automatically extract templates of these interactions. The study also compares the performance of a Gated Graph Neural Network (GGNN) with other approaches, such as RNN and CNN, showing that GGNN performs better on the Juliet dataset

[72]. However, on the Draper dataset, CNN edges past GGNN, indicating the importance of the dataset's features in model performance.

According to their study, VulChecker [89] is the first deep learning framework capable of identifying vulnerabilities in source code at the line and instruction level and categorizing the type of CWE. VulChecker is the first SAST tool based on deep learning mythology for developers, allowing for speedy identification and remediation of code vulnerabilities, like standard SAST tools. They propose using message-passing neural networks for vulnerability identification to learn and discover the relationship between a vulnerability's root cause and manifestation point in a PDG. This approach enables the direct attributing of features to this PDG's edges, such as the data type of the DFG component. They describe the ePDG, a graph-based machine-learning optimal program representation that overcomes the inefficiencies and downsides of past work. Furthermore, researchers provide a novel data augmentation technique that solves the shortage of fine-grained datasets for GNN-based source code vulnerability identification while staying applicable to real-world software projects. They collected a diverse set of C/C++ projects to train and test VulChecker and its baselines and trained the baselines using the Juliet C/C++ dataset [72].

Based on the findings of the selected research papers, notable developments, and approaches were found to be linked to source code vulnerability detection. The work by Du et al. [26], known as the Leopard framework, is quite an exceptional analysis as it does not require known bugs as input. Instead, it employs SVM, Naive Bayes, and Decision Trees for vulnerability identification in C/C++ applications, and the authors managed to make it work on real-world applications. Likewise, a paper [39] suggested using DL tools and CNN to identify vulnerabilities in the source code based on large datasets about open-source projects, including the NIST's SATE IV Juliet Test Suite [72]. However, also relying on a DNN this time, VulBERTa [75] aims at training source code representations both with syntax and semantically, utilizing differentiated datasets such as, for instance, Vuldeepecker [76] and Draper [77]. A different learning technique, based on CNN and bi-directional LSTM, is incorporated in the Developer Framework [80] to overcome the problem of data scarcity and enhance vulnerability identification. AI and Blockchain detection [87] present a combined system of mitigating multi-party development risks using LSTM and Bi-LSTM models. Graph Neural Networks, as discussed by [88], also show the ability to distinguish the characteristics of the source codes, and among them, GGNN is effective in handling the Juliet dataset [72]. Lastly, VulChecker [89] introduces a state-of-the-art deep learning-based SAST tool to precisely identify and classify vulnerabilities thoroughly, utilizing message-passing neural networks, and developing a new data augmentation approach to improving the detector's performance, trained on the Juliet C/C++ benchmark [72]. The paper shows the specific approaches of each framework, different techniques, and datasets, and focuses on the model's performance and features of datasets for vulnerability detection.

3.3. Large Language Models

In recent years, LLMs have demonstrated their ability in software development activities. such as understanding source code, code generation and bug fixing, significantly improving these processes. However, there is still much space for improvement around vulnerability detection in source code, as there are fewer technical reports available in the field regarding the source code vulnerability detection [90].

In this paper [90], they research whether and how LLMs may assist with code review and investigate a range of proprietary and open-source LLM models involving those from the Dolly, GPT, LLaMA, and Falcon families. This experiment focuses on the following tasks. The first one is identifying security vulnerabilities in code, and the second one is ensuring that code matches its intended functionality. Reviews contribute to knowledge sharing and maintainability, among other things. This paper uses three data sets with Python code at the functional level. First Humane Val dataset Contains 148 observations after discarding 16 observations that do not meet the criteria of one function and one docstring per observation. MBPP dataset Consists of 476 observations selected from a primary dataset of 974 annotations.

Detecting software vulnerabilities with LLMs [163]. This article outlines a set of tests that used four renowned LLMs on two well-known public datasets to assess LLM performance in discovering code vulnerabilities. Results show a substantial performance difference between these LLMs and standard static analysis techniques, primarily because of high false positive rates. Moreover, LLMs have more potential to detect small patterns related to software vulnerabilities. This study suggests that the next step should be integrating LLMs with other analysis methods to identify source code vulnerability for better results. They selected two classed datasets for this study, the first one is code gadgets, and the other one is CVEfixes [97] both have been published in peer-reviewed journals. According to the findings, GPT-3.5-Turbo and GPT-3.5-Turbo-0613 are ineffective at detecting vulnerabilities in source code when working with the direct prompt option.

ChatGPT for Vulnerability Detection, Classification, and Repair Engaging [91] ChatGPT with other fine-tuned language models designed specifically for coding purposes. They conducted research to compare two editions of ChatGPT (GPT 3.5 turbo and GPT 4) with four established methods (AIBugHunter [92], CodeBERT [93], GraphCodeBERT [94] and VulExplainer [95]) specifically created for identifying software vulnerabilities across four vulnerability scenarios. ChatGPT was thoroughly tested on two vulnerability datasets (Big-Vul [96] and CVEFixes [97]).

The author used this approach [98] to assess ChatGPT's capability to repair smart contract flaws. This evaluation used the SmartBug curated dataset. The SmartBug curated dataset consists of a hundred and forty-three susceptible smart contracts, each of which has been manually annotated based on a recognized classification (the DASP classification3) [99]. The goal of this study was to examine ChatGPT's ability to remedy code vulnerabilities, not to determine its ability to identify them. Manual annotation was not considered. The system uses Slither (a static analysis

tool for vulnerability detection) [100] in two phases. The first one is before sending a request to ChatGPT to attempt to offer the AI some suggestions regarding the embedded vulnerability, the other is done after obtaining the resolved code from the AI to ensure the accuracy of the correction. The researchers examined the original and patched ChatGPT contracts using a cutting-edge smart contract analysis tool.

In a Preliminary Study on Using LLMs in Software pen-testing [101], researchers developed AI agents using OpenAIs GPT models [102][103] and Google's Gemini model [104]. They used Gemini Pro as the LLMs GPT 3.5 Turbo and GPT 4 Turbo. They constructed a couple of agents for the GPT models, one utilizing the chat completions API 4 and one with the assistants API 5. They constructed interfaces for these LLMs and fed them the application's code. Ask LLMs about source code vulnerabilities and their location (line number). To evaluate the correctness of these AI agents, they rely on the test scenarios from the OWASP Benchmark Project 6. The OWASP Benchmark is a Java test suite that evaluates the effectiveness of ML-based vulnerability detection tools like SAST and DAST. It utilizes test cases from version 1.2 representing a web application. The benchmark includes 2740 test cases, each replicating web pages within web applications. SonarQube is widely used as a SAST tool in industries. A research study compared the testing results produced by SonarQube Community Edition 7 and those generated by LLMs. The results suggest that with engineering improvements, an LLM can improve its accuracy gradually over time, reaching a level of precision that's on par with or even surpasses that of SonarQube, which is a popular tool for static software security testing. Overall GPT-4-Turbo Assistant performs better.

The author proposes SecRepair [105], a cutting-edge AI-assisted tool for detecting and repairing insecure scripts using a personalized description generation method appropriate for authors with limited security experience. Additionally, this system uses Reinforcement Learning with semantic reward value to improve its efficacy and capability in generating comments for code contributions. To enhance system training and create a reliable model for vulnerability assessments, the InstructVul vulnerability dataset, specialized in the C/C++ programming language, is the first of its kind and focuses on security flaws. This research includes a quantitative examination of zero-day and N-day findings and elimination and case studies to demonstrate the model's quality. For all these tests, datasets were randomly divided into 80:10:10 for training, validation, and testing, and 32 decoder layers from the trained CodeGen2 model were used. This method assesses the model's performance using BLEU and Rouge-L ratings. They evaluated this model's ability to generate developer-friendly descriptions and code comments and suggested that it outperforms discovering, correcting, and explaining vulnerabilities using code comments.

This paper is for Fine-tuning LLMs for vulnerability detection [106]. This work used the primary pre-trained model, a significant library of GitHub source code, to pre-train WizardCoder [107] using large language modelling. The authors compare WizardCoder [107] to ContraBERT [108], the most advanced model in the CodeBERT family. ContraBERT improves the encoder design by combining two pre-trained CodeBERT encoders. ContraBERT is trained using masked language modelling (MLM) and contrastive pretraining tasks. It may be

fine-tuned for defect detection [108]. An improved version of the CodeBERT model, ContraBERT produces more reliable and accurate fault discovery results. ContraBERT was therefore selected for the tests.

This paper [164] explores the capabilities of LLMs for vulnerability detection and repair across eight popular programming languages, including C, Ruby, PHP, Java, JavaScript, C#, Go, and Python. The study queries models such as GPT-4 using APIs, with tasks designed to identify programming languages, locate vulnerabilities, and propose fixes. The research demonstrates the potential and limitations of LLMs in automated vulnerability detection and code repair, highlighting their ability to handle diverse coding languages and contexts. However, it also identifies challenges such as false positives and limited performance in complex scenarios, highlighting the necessity for further advancements in this area.

Table 3.1: Summary of Studies on Source Code Vulnerability Detection Using LLMs

Paper/Study	**Models/Tools Used**	**Datasets**	**Key Focus/Findings**
Study on LLMs For Code Review [90]	Dolly, GPT, LLaMA, Falcon	Humane Val, MBPP	Assessed LLMs for identifying vulnerabilities and verifying code functionality. Demonstrates gaps in current LLM capabilities.
Detecting Software Vulnerabilities Using LLMs [163]	GPT-3.5-Turbo, CodeGen2,davinci, GPT-3.5-Turbo-0613	Code Gadgets, CVEfixes[97]	Found LLMs less effective than static analysis tools due to high false positive rates but promising for small-pattern detection.
ChatGPT for Vulnerability Tasks [91]	GPT-3.5-Turbo, GPT-4, AIBugHunter, Code-BERT, GraphCode-BERT, VulExplainer	Big-Vul [96], CVEFixes[97]	Compared ChatGPT with specialized models for vulnerability detection and repair, highlighting its limitations and strengths.
Fixing Smart Contracts Vulnerability with ChatGPT[98]	GPT, Slither (static analyzer)	SmartBug curated dataset	Evaluated ChatGPT's capability to repair smart contract vulnerabilities are validated by static analysis.
LLMs in Software Pen-Testing [101]	GPT-3.5-Turbo, GPT-4-Turbo, Gemini Pro, SonarQube	OWASP Benchmark Project	Showed that GPT-4-Turbo can match or surpass traditional tools like SonarQube for static security testing.
SecRepair [105]	Reinforcement Learning, CodeGen2	InstructVul	Introduced advanced detection and repair methods for insecure scripts using personalized descriptions and semantic rewards.
Fine-Tuning LLMs for Vulnerability Detection [106]	WizardCoder, ContraBERT	GitHub library of source code	WizardCoder and ContraBERT demonstrated significant improvement over traditional pre-trained models for defect detection.
Can Large Language Models Find And Fix Vulnerable Software [164]	GPT-4	Sample code in 8 languages: C, Ruby, PHP, Java, JavaScript, C#, Go, Python	Explored LLMs for vulnerability detection and repair across diverse languages. Found potential for automated tasks but highlighted challenges like false positives and performance limitations in complex scenarios.

The comparison of those papers demonstrates the various methods and methodologies involving the utilizing of LLMs in identifying and fixing code vulnerabilities. This [90] study examines the implementation of LLMs models such as Dolly, GPT, LLaMA, and Falcon, which are proprietary and open source, to detect security vulnerabilities. This contradicts another study that focuses on the competency of LLMs such as GPT-3. 5-Turbo on code gadgets and CVEfixes [97], where high false positive rates and the differences between GPT-3.5-Turbo and GPT-3.5-Turbo-0613 to traditional static analysis have been discussed [163]. One more work investigates how ChatGPT performs in terms of software vulnerability detection and remediation in comparison with specialized models such as AIBugHunter [92] and CodeBERT [93], using the datasets such as Big-Vul [96] and CVEFixes [91]. The SmartBug dataset is used in a distinct work to assess ChatGPT's capability of fixing smart contract defects and incorporating static analyzers such as Slither for validation [98]. Additionally, a relatively recent study regarding the applicability of LLMs such as GPT-4 Turbo for spearheading software pen-testing came out positive when benchmarked with tools like SonarQube based on the OWASP Benchmark Project [101]. Finally, the SecRepair [105] uses reinforcement learning and personalized description generation and can detect and repairing unsafe scripts, it is an example of the collaboration of the most advanced AI approaches and specific datasets such as InstructVul. Each of the studies shows its specific features and weaknesses, which indicates the development of possibilities that LLM capability of detecting source code vulnerability detection.

3.4. Research Gap

LLMs have shown, over the last few years, to have a massive general understanding of programming languages, mostly when generating source code and analyzing patterns. So, the question is whether LLMS are fit for source code vulnerability detection. If so, what is their limitation. More Research is needed to determine the capacity of fine-tuning and proprietary LLM in source code of C# that classify vulnerability.

Some researchers have studied the ability to classify weaknesses using a proprietary LLM and an open-source LLM, but they have not compared them. Most of them are built on the C/C++ and Python languages [91][98][101][105]. The lack of this analysis made it challenging to choose the best approach for detecting security vulnerabilities in source code for developers. Moreover, more research is required on the practicality of utilizing these models and their ease of use and cost efficiency. The main aim of this study is to detect and introduce user-friendly and cost-effective C# code vulnerability detection solution for developers. The objective of this is to hence evaluate the strengths and weaknesses of different AI based vulnerability detection methods to serve as effective tools for developers.

Another research gap is the comparison on LLMs versus traditional neural network-based methods. The focus will be on the evaluation of customization techniques for open-source and proprietary LLMs to know the best approach [106]. The purpose of this research is to implement different LLM models and compare them with neural network classifiers. This classifier is based

on a code property graph, which includes sending input as AST and plain codes to a feature extractor called Word2Vec, which produces features for an LSTM model to detect vulnerabilities in C# code.
This will meet the goal of identifying the way these methods perform in the detection of vulnerabilities in C# to fill the gap in the efficiency comparison of the mentioned methods under controlled conditions. The research aims to fill the existing gap by evaluating the models in terms of their effectiveness and practical aspects, with the results helpful to software developers in enhancing their security tools.

This research addresses these gaps by implementing and evaluating

- Fine-tuned open-source LLMs (CodeGen2 and LLaMA2).
- Proprietary LLMs (GPT-3.5 and GPT-4).
- Traditional Neural Network classifier (Word2Vec + LSTM with plain code and AST input).

The accuracy, efficiency and usability of the results will be analyzed and directly applicable to the research objectives. The results will offer practical guidance for software developers as well as contribute to security toolkits, in line with the aim of improving the effectiveness of vulnerability assessment and supporting the development of secure software systems.

3.5. Selected Programming Language

Microsoft programmer Anders Hejlsberg created the C# programming language in 2000 [109]. It was then recognized as a global standard by the ECMA (ECMA-334) in 2002 and the ISO/IEC (ISO/IEC 23270 and 20619) in 2003. Microsoft released .NET Framework and Visual Studio to develop the applications. C# ranks as the fifth most popular program language in 2024.

As with other prominent programming languages, C# can be used for various development, including web development, mobile development, desktop apps, cloud-based services, and virtual reality (VR). C-sharp is also widely recognized as one of the best game programming languages. C# has the advantage of being cross-compatible with most game development platforms [109].

Most research uses C/C++ dataset for their research vulnerability detection. So, because of the lack of C# language vulnerability detection research we decided to do the C# language for our research. C/C++ continues as a language, with lower-level capabilities providing control over system resources. This feature makes it open to security risks like buffer overflows and memory leaks. Identifying these vulnerabilities demands methods due to C/C++ direct memory

access and absence of built-in memory protection mechanisms [110]. Typical strategies involve analysis, dynamic analysis and fuzz testing each offering advantages and constraints. C# is a programming language that uses the Common Language Runtime (CLR) to provide several levels of abstraction and automate memory management. The inclusion of reflection, serialization, and dynamic code generation in C# can introduce vulnerabilities that're not easily recognizable.

C# presents unique security challenges which need customized solutions particularly when AI is used to identify vulnerabilities. Tools that function for several programming languages exist, but C# requires dedicated attention because of its unique characteristics. Take memory management, for example and developers handle memory management manually in C and C++ while C# provides automated memory management through garbage collection. The system isn't without flaws since neglecting to release unmanaged resources can result in memory leaks and decreased performance.

Even though C# enforces strict type safety, null references slip through all the time, crashing apps at runtime. AI models must get good at telling the difference between safe and risky null checks, especially in cases where poor code could open security holes. Platform dependencies add another layer of complexity. C# powers everything from desktop apps to cloud services, and each environment has its own risks. A web app built with ASP.NET might face SQL injections or cross-site scripting attacks, while a desktop program could trip up over thread management or error handling. AI models need to adapt to these wildly different contexts to stay effective.

Furthermore, it is essential to explore how AI models trained on vulnerabilities in other languages, such as C/C++ or Java, generalize to C#. Differences in syntax, memory management, and API usage may lead to gaps in detection accuracy. A comparative analysis of how well existing models transfer knowledge across languages would provide valuable insights into optimizing AI-driven vulnerability detection specifically for C#. Including practical examples, such as code snippets illustrating SQL injection attacks, unsafe memory handling, and exception management failures, would further strengthen the contextual relevance of this research. By addressing these aspects, the study enhances its applicability, ensuring that the findings contribute effectively to the advancement of secure C# software development.

3.6. Selected Dataset

The Juliet Test Suite 1.3 for C# [72] is a collection of test scenarios designed to assess the security of C# applications. Developed by the NSA Center for Assured Software, this compilation consists of 28,942 test scenarios classified under 105 Common Weakness Enumerations (CWEs). These test scenarios address vulnerabilities often encountered in software, offering a testing platform for C# developers to detect and address possible security concerns in their code.

Version 1.3 marks the launch of the C# test suite following the naming convention of the test suites. The dataset can be freely downloaded as it is and is not protected by copyright. It is shared under the CC0 1.0 Public Domain License, permitting usage and distribution.

In this research, we selected SQL injection [134], OS Command Injection, and Cross-Site Scripting (XSS) to evaluate the models' vulnerability detection. SQL Injection is a type of security vulnerability commonly found in database-driven web and desktop applications. When a hacker can manipulate an SQL query by inputting SQL code into input fields, it becomes a major threat to the database or data. The outcomes of an SQL injection attack can be severe, such as stealing data, losing information, or even taking control of the system. OS Command Injection vulnerability occurs when an application construct all or part of an operating system command using user input. This can allow an attacker to execute arbitrary commands on the host system, potentially leading to severe consequences such as data loss, data corruption, or even full system compromise. Cross-site Scripting (XSS) vulnerability allows attackers to inject malicious scripts into web pages viewed by other users. These scripts can hijack user sessions, deface websites, or redirect users to malicious sites, compromising user data and security.

CHAPTER – 4

Methodology

4.1. Research Design

This research uses a comparative analysis of research design to compare the performance of the different models in detecting a vulnerability in the source code. This approach includes the comparison of benchmark open-source and proprietary LLMs and an NN-based approach to measuring their efficacy, preciseness and overall potential in detecting source code vulnerabilities. The primary objective is to compare in a systematic manner the advantages and disadvantages of different models to determine which of them is the most suitable for detecting source code vulnerabilities.

4.2. Rationale Behind Choosing the Models

AS open-source LLMs, we selected LLaMA2[136] and CodeGen2[135] models. LLaMA2 and CodeGen2 were chosen as the models because they are easier to obtain and are now being used more frequently in research. The models are open source so researchers can finetune and optimize them according to their needs. The LLaMA2 is chosen as one of the general-purpose LLMs, and the CodeGen2 is selected for its particular importance in code generation. This comparison is intended to identify the differences between general language comprehension models and models developed to work within coding and software development. Also, these open-source models make evaluating and replicating results easier, which is very important in research and comparing the results with proprietary LLM models.

The use of proprietary models such as GPT3.5-Turbo and GPT4-Turbo via Assistant API and Chat Completion since they are the latest models with large training data sets and perform well in natural language understanding and generation [103]. These models are trained with large-scale code datasets and other general data sets, which will help them identify more advanced vulnerabilities. Proprietary software solutions may include enhanced and highly specialized training algorithms and other improvements that perform better than open-source solutions. Comparing these models with open-source alternatives helps determine if the additional cost and the lack of transparency about the model's inner workings are justified by increased accuracy.

The Word2Vec LSTM approach is included to provide a traditional NN baseline. This is an embedded code token into the Word2Vec vectors and then trained to learn sequential algorithms

using LSTM networks. This is another base solution for neural networks but going in a different direction than the LLMs.

4.3. Comparative Analysis Goals

In evaluating model performance, it is crucial to assess each model's capability to accurately identify vulnerabilities in source code by examining key performance metrics, including Accuracy, Precision, F1 Score and Recall. These metrics provide understanding of each model's potential to identify vulnerabilities, providing a concrete basis for comparing model accuracy and reliability. In addition to performance, assessing resource efficiency is vital, as it enables computational complexity and scalability across models, which has practical implications for their deployment. Evaluating resource usage helps determine how computationally intensive each model is, an important factor in practical applications where computational cost and time efficiency matter. Lastly, it is essential to examine the applicability and flexibility of each model by analyzing the ease of fine-tuning and configuration for specific vulnerability detection tasks. This includes understanding the degree of fine-tuning required, the model's usability in varied contexts, and the ease of adapting each model for specific needs, influencing its overall practicality and flexibility in situations that occur in the real world.

This comparative study aims to provide a broad picture of the state of vulnerability detection mythology to be useful to the academic research and software security world.

4.4. Models Selection and Setup

4.4.1. LLaMA2 Model

LLaMA2 is an open-source LLMs best suited for natural language processing (NLP) and understanding tasks such as coding, summarization, and generative AI. Created by Meta AI, LLaMA2 is based on its previous LLaMA models, LLaMA2, with a better design, training methods, and performance metrics [141].

LLaMA2 has different parameter sizes to support various computational powers, allowing deployment flexibility. An open-source model allows fine-tuning and flexibility to be adjusted according to the research requirements while lacking options compared to proprietary models. Compared to other open-source models, LLaMA2 has enhancements in training data, model and fine-tuning that enhance its performance in code-related tasks [139].

v). Model Architecture

With the use of ordered tokens in the input sentence, LLaMA2 is a decoder-only language model that predicts the next token. illustrates how the LLaMA2 model is autoregressive. The model's first input is the first prefill sentence, "The sky clear," it predicts the first output, "today," as the following word based on how it is executed. In addition, the anticipated word is utilized as input, and the process continues until the end of the sentence (EOS) is produced. The model uses three inputs: tokens, attention masks, and positional embedding. A tokenizer that internally employs a lookup table-style technique converts the input text into a series of tokens [141][143].

To point out a word's significance in a sentence with other words, utilize an attention mask. Table 4.1 illustrates how the terms "the," "sky," and "clear" focus solely on themselves and their predecessors rather than being used for conditioning, this mask aids the model in doing generic computations.

Table 4.1: The input sentence "The sky clear" is an example of an attention mask.

The	sky	clear
0	-inf	-inf
0	0	-inf
0	0	0

Positional embedding is used to supply the model with information on the placements of the words in the phrase, allowing it to assess the importance of a word in relation to other words based on their distance from one another. LLaMA2 employs a novel positional embedding method known as RoPE, which works in the rotational domain rather than direct position weightage [142].

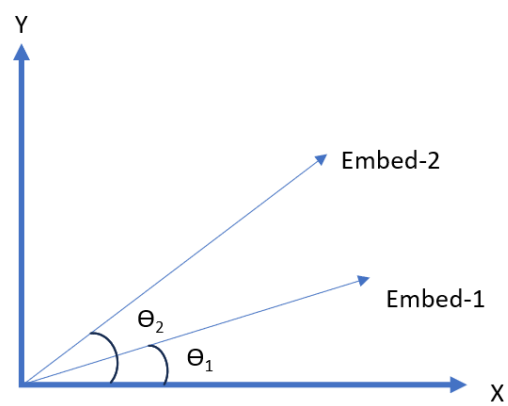


Figure4.1: Rotary Position Embedding[143]

The hidden state embedding is transformed or rotated at a particular angle depending upon the distance along the phrase, and the angle is proportional to the word's location. The correlation between two embedding vectors is always proportional to COS(Θ2-Θ1), where Θ1 and Θ2 denote the angular rotation for a specific word embedding based on its position in the phrase. According to the explanation, the correlation between words in a sentence increases as the difference between angles one and two decreases [143].

The model generates a token, which is detokenized to form a word using a de-tokenizer. If the new word is not EOS, it is passed back into the model as new input.

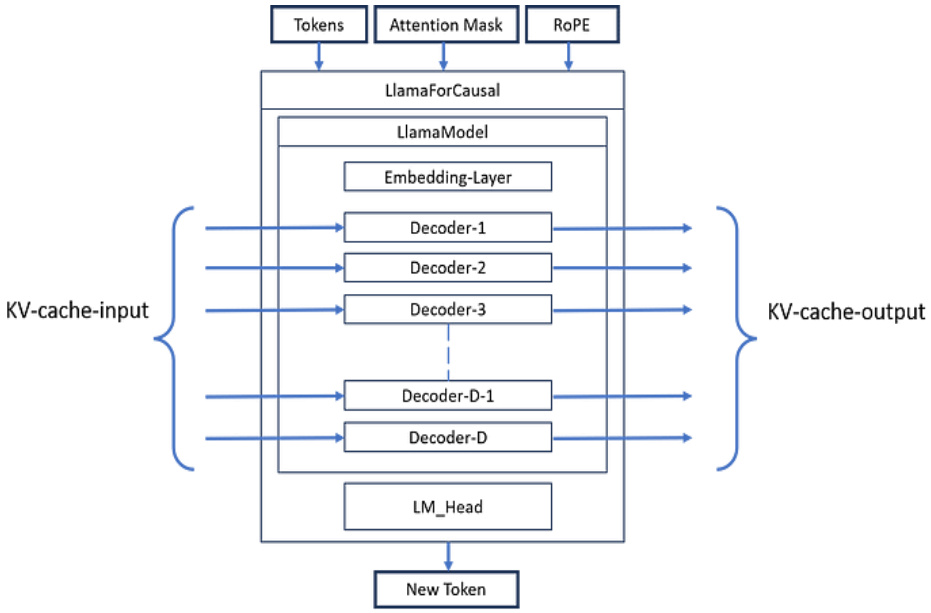


Figure 4.2: Model architecture of LLaMA2[143]

Figure 4.2 displays the model architecture of LLaMA2. The model starts with an embedding layer, then D decoder blocks, and finally LM_Head, which predicts the next token. We will go through each component in detail. The embedding layer functions as a lookup table, storing each token in an appropriate embedding vector. For example, each word token in the LLaMA2-7B model is encoded in a 4096-dimensional embedding vector, also known as hidden state size.
The decoder, made up of RMSNorm, Multi-Head Attention, and MLP layers, is the most important component of the LLaMA2 model. Because the LLaMA2-7B model contains 32 decoder blocks, the output of one decoder block is passed on to the next. The final layer in the LLaMA2 model, LM_Head, is a linear layer that converts hidden states into tokens. The output dimensions of LM_Head are always the same as the size of the tokenizer dictionary. This layer predicts the possibility of tokens becoming the next token. Figure 4.2 indicates that for an Autocomplete use-case, we pass the input sentence "The sky clear" and it guesses the following

word 'today'. Later, in subsequent phases, we pass the previously predicted word to obtain the next predicted word as output. Who is directing the model in terms of previously used language or context? This is where KV-caching comes into play. KV-caching simply remembers the context of some intermediate outputs. As previously stated, the LLaMA2 model is made up of multiple decoder blocks, including attention and norm layers. To learn more, we'll refresh our transformer knowledge.

vi). Selected Model

LLaMA2 offers three versions, each designed for different use cases. The LLaMA2-7B model, trained over 184,000 GPU hours, is the smallest and most efficient, excelling in text comprehension despite having less world knowledge. It's suitable for tasks like enterprise chatbots and custom data training. The LLaMA2 -13B model, trained over 368,000 GPU hours, balances size and knowledge, making it ideal for web applications, content writing, and SEO. Lastly, the LLaMA2-70B model, with over 1,720,000 GPU training hours, is the most powerful, supporting complex tasks like education and e-commerce, and offering deep comprehension and scalability. As a selected model for this research analysis, we choose The LLaMA2-7B model [140][141].

4.4.2. CodeGen2 Model

The CodeGen2 model is an LLM model for code generation and code summarization tasks. It’s built by Salesforce, lies on top of the transformer architecture and thinks of itself as a transformer that can handle code across numerous programming languages. Take a huge dataset of programming languages, such as Python, JavaScript, and train the model. It’s capable of generating code that is both logically and contextually correct [135].

CodeGen2, an open-source language model, is hypothesized to excel in spotting vulnerabilities in C# programming source code due to several compelling reasons. Firstly, CodeGen2 has a great understanding of both syntax and semantics at a level essential to identifying vulnerabilities such as SQL injection or buffer overflows due to tiny code inconsistencies. However, to detect such flaws, it is essential to have such parts of programming languages understood. Previous research [161] underscores the relevance of semantic understanding to code vulnerability detection.

Secondly, CodeGen2’s open-source nature can be fine-tuned with domain specific datasets. As a result, it stands out as a good target to fill the gaps for C# vulnerability detection. Indeed, studies have shown the capability of customizable models to improve vulnerability classification performance [106]. However, these types of models work only in environments that are reasonably close to the ones they are trained in. The research objective of improving the C# vulnerability detection aligns closely with this adaptability.

CodeGen2 is additionally pre-trained on a large-scale quantity of diverse programming languages and is effective at spotting anomaly in the code pattern. Previous research has shown

that training the model on large datasets helps the model better detect security vulnerabilities [135]. By training on this large scale, CodeGen2 is provided with this large knowledge base about many different coding patterns and what deviations can occur in those patterns.

Furthermore, CodeGen2 performs well on various tasks such as code summarization and debugging. They derive from the fact that this will be able to understand the logical and structural code flow, therefore that is potentially a fit for identifying for vulnerabilities. Related work validates the importance of these capabilities for secure code analysis.

However, the deployment flexibility of CodeGen2 is also apparent with multiple parameter sizes as a tradeoff for different use cases and resource constraints. This versatility enables comparisons of different configurations of the same device, which is an important part of this study.

CodeGen2 deployment flexibility may be tuned based on the user's needs by allowing different parameter sizes. Automatic code completion, debugging, vulnerability detection, and structural summarization of complex code are its primary use cases. It is an open-source model, providing a tremendous amount of flexibility for researchers and developers to fine-tune the model using custom datasets and optimize on for specific tasks or programming languages.

This capability is especially useful to developers who want to automate trivial, repetitive coding tasks as well as automation of small and large enterprises, and large-scale code review. With these strengths, CodeGen2 is shown as a potential candidate for vulnerability detection in C# source code. Its open-source nature, adaptability and robust capabilities are therefore aligned with its research objectives and closing existing gaps in the field.

i). Model Architecture

CodeGen2 is an architecture of a decoder only transformer model for tasks about code. Following the principles of transformer models, it adds specific augmentations to better understand and generate programming languages [145][135].

The Embedding Layer, Code snippets are fed to the system as discrete tokens, and then vectorize code using dense embedding vectors that represent the syntactic and semantic meaning of the code. The embedding vectors of these represent basic components of code including functions, variables, and syntax related factors. This layer is adjusted especially for CodeGen2 to deal with programming language specific structure and patterns, so the model will produce code that is syntactically accurate and contextual in the given use case [145][135].

Decoder Blocks are the core processing units of the model, decoder blocks are part of CodeGen2. The multi-head self-attention layers and FFNs are contained in each decoder block. With self-attention layers, the model does not rely on positional embeddings but learns relations between tokens in the input sequence to understand code logic in a sentence. To this effect, the multi-head attention mechanism ensures that the model can take for granted multiple parts of

the code simultaneously, helping it generate contextually interesting and logically wise code snippets.

Like LLaMA2, CodeGen2 uses self-attention to encode token relationships, but also adds RoPE to keep track of source token position information in the input. In the angular domain, RoPE operates and retains positional token information from the model, as it is important to generate code that will possess correct syntax and logical flow. The RoPE calculates the angular distance between tokens and ensures the model generates code that respects the distances between variables or control structures and functions.

A feed-forward network is processed on each decoder block that takes in the attention-weighted tokens. But this network further refines the relationships between tokens such that the generated code remains syntactical and logical. During inference, the FFNs allow the model to represent more complex dependencies between code tokens, and as a result the model generates coherent, functionally correct code.

In CodeGen2, layer normalization and residual connections are introduced for stable training and efficient information propagation in the network. The mechanisms support the model learning from shallow and deep layers, thus increasing code comprehension in the model and providing it with the capability to generate accurate code for the most complex tasks [146].

The CodeGen2 model has the final output layer, called the LM_Head, that takes processed tokens and transforms them into a distribution over the model token vocabulary. This makes it possible for the model to predict the next token in a sequence, and produce coherent, suited to context code. Tokenizer dictionary size is the output dimension and corresponds to a large dictionary size encompassing a producer of various programming language tokens so that the model can perform programming tasks in various languages.

Codogen2 builds on Key Value (KV) caching to take advantage of long code sequences. By caching intermediate outputs from the attention layers the model can keep context over several layers and generate long code snippets without recomputing all the way through. In practical coding environment, KV caching further improves the model's ability to do tasks such as auto completion and real time code generation.

A significant step forward for AI driven code generation, Codecgen2 provides a collection of models that can be used for small-scale real-time assistance to code generation and vulnerability detection in a large-scale enterprise. Equipped with powerful self-attention mechanisms and positional embeddings, its decoder-only architecture allows it to generate accurate contextually relevant code across multiple programming languages. This model is capable of handling syntax and semantics of the code for it to be a useful tool for the developer, researcher and enterprise that wants to automate and enhance their coding tasks.

ii). *Selected Model*

The CodeGen2 series consists of several model's purpose designed for different coding tasks. The CodeGen2 1B is the smallest, suitable for small scale work such as auto completion and debugging inside the IDE but provides a good trade-off between performance and efficiency for small applications. The CodeGen2 3.7B is a mid-sized model, better at parsing complex tasks that require multi language code audit or especially code vulnerability detection with higher accuracy and generality. For this reason, CodeGen2 16B, the largest available model, is designed for enterprise level applications, being ideal for large scale code generation, code reviews and advanced vulnerability scanning, providing deep understanding and scalability for complex tasks [144] [135].

A version of the CodeGen2 family is being pre-trained on natural language data and is initialized with the CodeGen-NL-2B model, from which the CodeGen2-2B-Multi model is generated. In addition to being further pre-trained on a large-scale dataset drawn from GitHub repositories based on several programming languages, the 2B-Multi variant is pre-trained on a language dataset for Python, object-oriented programming languages, and object-oriented enterprise languages. The dataset includes 119.2 billion tokens from C, C++, Java, Go, JavaScript, and Python. The pre-training on this multi-lingual dataset enables the model to perform various code generation tasks across different programming languages [144].

The "2B" in the name refers to the number of trainable parameters (approximately 2 billion), while "Multi" indicates the inclusion of multiple programming languages in the training dataset. The model is optimized to handle multi-language code generation tasks and can generalize better across different programming languages.

Table 4.2: CodeGen2 Model Details [144].

Feature	**Description**
Model Name	CodeGen2-Multi 2B
Pre-training Initialization	CodeGen-NL 2B
Number of Parameters	2 Billion
Pre-training Dataset	GitHub repositories (BigQuery)
Total Tokens	119.2 Billion
Languages Covered	C, C++, Go, Java, JavaScript, Python
Training Objective	Code generation across multiple programming languages

4.4.3. Finetune LLMs

LLMs, such as GPT-4, have become the current norm in the NLP industry for developing products and applications. These models can perform various tasks and may quickly adapt to new ones using prompt engineering techniques. However, training with these models offers a significant barrier. Massive models such as GPT-4 take millions of dollars to train.

However, smaller models cannot generalize to different tasks, resulting in several models for multiple tasks performed by multiple users. This is where PEFT techniques like LoRA come in they enable us to train huge models far more efficiently than fully finetuning them. We used to fine-tune LLMs using LoRA and Quantized Low-Rank Adaptation (QLoRA) techniques [147][138].

iii). PEFT Finetuning

PEFT Finetuning is a parameter-efficient fine-tuning technique, a set of fine-tuning algorithms that enable us to fine-tune and train models considerably more efficiently than traditional training. PEFT approaches typically operate by lowering the number of trainable parameters in a neural network. Prefix Tuning, P-tuning, LoRA, and other PEFT approaches are well-known and commonly used methods. LoRA is likely the most widely utilised for LLMs. LoRA has various versions, including QLoRA and LongLoRA, each with its own set of uses [147].

As the number of trainable parameters reduces, we must spend less time on training. But that is only one aspect of it. With fewer trainable parameters, we can train models and test them considerably faster. With less time, we may experiment with different models, datasets, methodologies, and so forth.

Furthermore, with more significant time, we may train our models for much longer periods of time, resulting in a much smaller loss and a larger batch size, as PEFT approaches are well-tuned for memory utilization.

PEFT offers substantial cost savings in computing by optimizing memory usage. These optimizations reduce the need for instances with high VRAM capacity, enabling the training of larger batches on smaller VRAM. Consequently, this decreases computational costs and facilitates training on larger datasets by accommodating bigger batch sizes, thereby improving overall efficiency.

iv). LoRA Finetuning

LoRA is the most popular, and probably is the most used, PEFT technique, and it was released back in 2021 [149]. In LoRA terms, it is more of an adapter type, where it adds new parameters into the model to train the model through these new parameters. The essential component is how new parameters are added and merged back into the model in a way that does not increase the total number of parameters in the model.

As previously mentioned, LoRA is an adapter-based approach, where new parameters only arrive as part of the training step and not as part of the model. It still allows parameter-efficient finetuning while keeping the model size unchanged, which is possible because this is already a compact model.

LoRA works by decomposing the weight update matrix (ΔW) into smaller matrices, A and B, with a rank parameter R controlling their size. Instead of updating the full-weight matrix during finetuning, LoRA updates these smaller matrices using normal backpropagation. After training is completed, A and B can be multiplied to recover the original weight matrix. This method minimizes the number of trainable parameters, cutting computational costs and storage requirements, because only the smaller matrices must be saved, resulting in smaller checkpoints.

v). QLoRA Finetuning

QLoRA is a finetuning approach that combines high-precision computing with low-precision storage methods. This also keeps the model tiny while remaining very performant and precise.

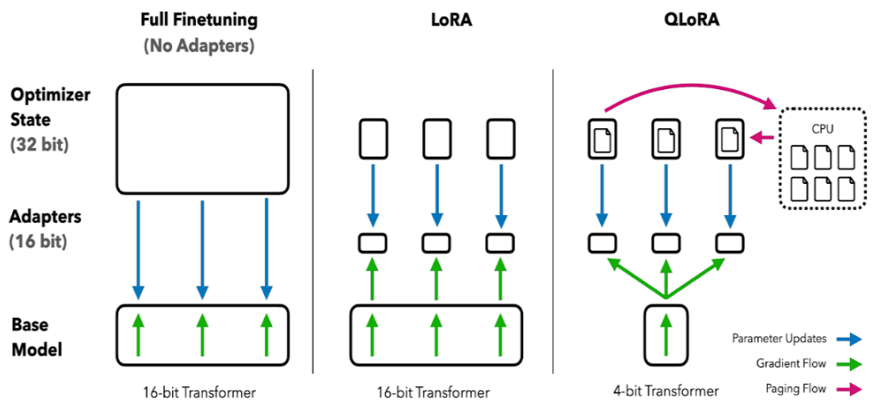


Figure 4.3: Full Fine-tuning, LoRA and QLoRa Fine-tuning Methods [148]

QLoRA operates by introducing techniques to reduce memory consumption while maintaining the same performance quality. In this thesis, we utilized 4-bit normal float as part of the quantization process. Based on Quantile Quantization approaches, 4-bit normal float data is information-theoretically ideal. It works by calculating the 2k+1 quantiles (where k is the number of bits) in a distribution between 0 and 1 and then normalizing these values into the [-1, 1] range. After normalization, neural network weights are scaled into the [−1,1] range and quantized using previously estimated quantiles [148].

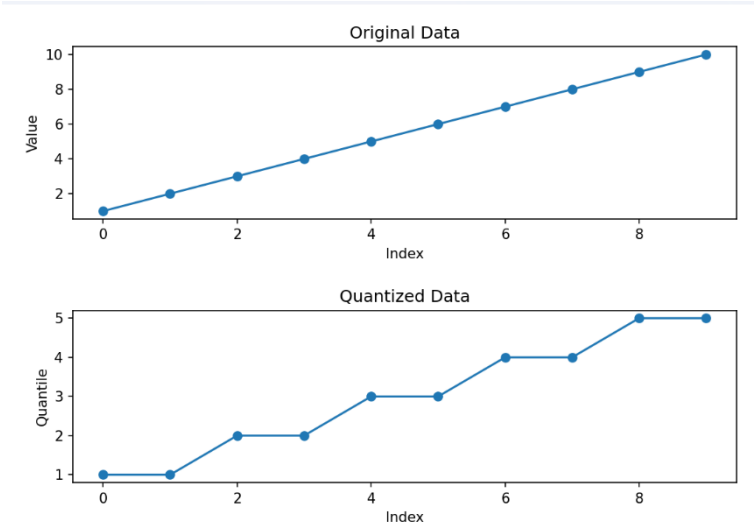


Figure 4.4: Original Data and Quantized Data [150]

Quantization occurs when data is grouped into "buckets" or 'bins". Take for instance, values 2 and 3 fall into the same quantile 2. The quantization process is a reduction in the number of unique values by rounding data to the nearest quantile. This way it allows the model not to lose information and keep it with having fewer values that leads to less memory usage [150].

4.4.4. GPT Models

OpenAI created a family of language models known as GPT. According to the architecture of the transformer, which has been very successful in NLP tasks. The most important goal of a GPT model is for it to generate human-like text conditioned on a given context. They are taught using a huge amount of text corpora possible to be fine-tuned to suite particular NLP tasks [103][151].

i). Model Architecture

The GPT architecture itself consists of many central components that all serve to understand and generate text. So, let's get into these components in detail. Transformers are the backbone of GPT models. It was in the form of an encoder–decoder structure, but in GPT's case the decoder alone is used. Each of multiple layers of the transformer architecture contains self-attention mechanisms and feedforward neural networks [152].

To handle the sequential nature of language, GPT models use positional encodings. These encodings are applied to the input embeddings to offer information about each token's position in the sequence.

In the GPT models, the input text is represented with the token embedding. Each token is embedded into a continuous vector space, which allows the model to process the text. The

transformer layers are stacked on top of each other to form multiple layers, we can use up to multiple transformer layers because they help in capturing complex patterns in the text. The number of layers is a hyperparameter that can be tuned case by case per task and resource availability.

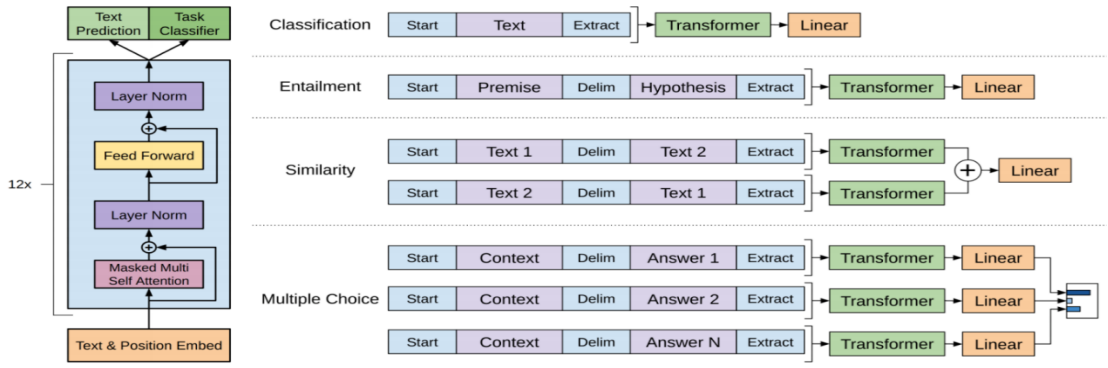


Figure 4.5: GPT Models Transformer Architecture [152]

ii). Selected Model

To evaluate performance, we used four different GPT models as shown in Table 4.3. Their strengths and limitations in the tasks of evaluation were assessed based on the key metrics for each model.

Table 4.3: GPT models [153].

Name	**Model Name**	**API Type**
GPT-3.5-Turbo	gpt3.5-turbo	Chat Completion
GPT-4-Turbo	gpt-4-1106-preview	Chat Completion
GPT-3.5-Turbo Assistant	gpt-3.5-turbo	Assistant API
GPT-4-Turbo Assistant	gpt-1106-preview	Assistant API

GPT 3.5 turbo is an upgraded iteration of the GPT 3.5 model, with enhancements in speed, effectiveness, or overall performance, and it maintains the core structure and features of GPT 3.5. It might have been Improved to cater to application scenarios.

GPT 4_Turbo shares similarities, with GPT 3.5 turbo as it is an iteration of the GPT 4 model. It is expected to offer enhancements in speed, efficiency, and overall performance when compared to the GPT 4 model while maintaining its core architecture and functionalities.

GPT 3.5 Turbo Assistant is a model that functions as an assistant specifically designed for tasks that involve AI features. It helps users with tasks like answering queries, providing information, and engaging in natural language conversations.

GPT-4-Turbo Assistant is Similar to GPT-3.5-Turbo Assistant, and the GPT 4 Turbo Assistant is another model based on the GPT 4 architecture tailored for AI tasks with improved performance and capabilities beyond the standard GPT 4 model, making it a more efficient virtual assistant.

iii). API Integration and Usage

All four models could be called through the respective APIs open to public provided by OpenAI, in a flexible and easy to integrate fashion in diverse applications. We used the Chat Completion API to interact with the models in real time, dynamic, interactive exchanges where the models simulated real time conversations and managed user input across multiple turns. As it is best suited for systems that will be live such as virtual assistants, conversational agents and customer service bots [153].

The Assistant API allowed us to use a programmatic interface to deploy the models in tasks where interaction with users is unnecessary. Using this API, batch processing, large scale text analysis and report or summary generation based on structured inputs were possible. The Assistant API enabled continuous operation of these models without real time feedback and allowed for high throughput language processing tasks by integrating them into back-end systems.

Both APIs gave us the flexibility to plug in powerful language models into different workflows and apps. The API based architecture was idol modular, so models were easily scaled and adapted to suit a variety of tasks while maintaining high efficiency and consistent output across the use cases.

4.4.5. Neural Network

For our research, we used a design in the form of a Word2Vec + LSTM neural network for detecting vulnerabilities in code. Building on the strengths of both Word2Vec embeddings and LSTM network, the approach can capture semantic and sequential relationships in source code.

Figure 4.6 shows a generic example of how Word2Vec embeddings are integrated with a LSTM network to perform sequence-based tasks e.g. text classification, sentiment analysis, or other NLP tasks. The workflow can be broken down into the following key components:

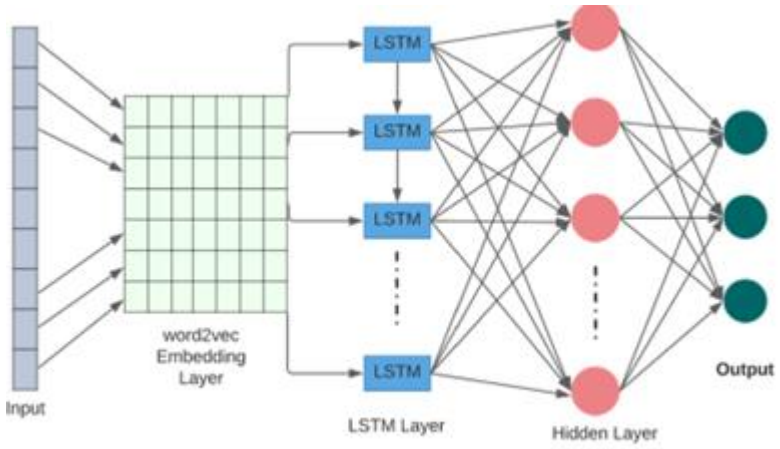


Figure 4.6: Word2Vec + LSTM Neural Network [154]

In this research, two approaches were used to feed the input into the Word2Vec + LSTM NN. One is plain code input, and the other with AST representations of the code.

The first approach tokenizes the raw code itself into a code tokens (e.g. keyword, operators, variables) sequence. This token converted into numerical indices is passed to embedding layer as these tokens. In this case, input size for code is determined by the number of tokens in code, i.e. number of words or symbols in the source code. By using this method, the model forces us to learn the exact syntax of the code, which may be very important to detect vulnerabilities caused by little but important changes to the code.

The second approach breaks the code into an AST which is a structured representation of code, the AST represents the syntactic components of the code (functions, variables from loops). The AST node represents a code construct, and each node is tokenized into a sequence. Once a series of AST tokens is produced, we turn it into numerical indices of the corresponding embeddings learned by the Word2Vec model. In this approach, input size depends on how many AST nodes there are, it depends on how complex the code is. When we feed the compiler's AST as the input to the model, we can restrict the model to focus on structure relationships within the code since they tend to be better at catching vulnerabilities that arise from bad logic or control flow.

In both approaches, the numerical indices are fed into the Word2Vec embedding layer, which transforms the tokens into dense vector representations, capturing the semantic relationships between the tokens. These vectors are then passed through the LSTM layers, which process the sequence while maintaining the context, allowing the model to identify potential vulnerabilities based on the structure and flow of the code [155].

The first major component is the Word2Vec Embedding Layer. Word2Vec is a type of word embedding that transforms each word in the input sequence into a dense vector of fixed dimensions [155]. These dense vectors capture semantic information about words based on their

contextual similarity. Every word in the terminology is represented as a continuous-valued vector, allowing the model to understand relationships between words, such as synonyms or words that commonly appear together [156].

In the diagram, the Pretrained Vectors serve as the input to the next layer. In other words, the embedding matrix is like a lookup table that relates to words, having as many rows as words, and the columns that correspond to the learned features. Before passing on to the LSTM layers, we map each word in the input sequence to its respective vector representation. This allows the model to use semantic structure of language without having to learn embeddings from scratch by pre training the embeddings on the large corpora [154].

The LSTM Layer is the core of the architecture. LSTM networks are a type of RNN designed to handle sequences and maintain information over long time periods, making them well-suited for text data, where understanding word order and context is crucial [156].

In the architecture shown, multiple stacked LSTM layers are used. Each LSTM cell takes the Word2Vec embedding vectors as input and processes them sequentially, retaining information from previous steps in the sequence. LSTMs excel at handling long-term data dependencies by controlling information flow with memory cells and gating mechanisms to control the flow of information.

The First LSTM Layer, the word2vec embeddings are received and processed word by word from the input sequence to learn the sequential pattern in the text. Subsequent LSTM Layers, these layers further refine the representation learned from the previous LSTM outputs, capturing deeper sequential relationships in the input data. The model can learn with more complex temporal dependencies, which is necessary for detecting further deep patterns on the sequence, by stacking multiple LSTM layers [156].

The final LSTM layer outputs are fed into hidden layer which are fully connected dens neurons. This second layer transforms the sequence-level information from LSTM outputs to a higher-level representation using these higher-level features. The hidden layer neuron connects to every neuron in the LSTM layer learning nonlinear combination of sequential features supplied by the LSTM. These hidden layers are normally utilized to extract patterns and features that are most relevant to the final prediction task, helping the model focus on the most informative aspects of the input.

The Output Layer is the architecture's final layer, responsible for making the final prediction or classification based on the learnt representations. In the output layer, we can utilize the SoftMax activation function to create probabilities for each class and in our model, we have one class predicted on top with maximum probability.

This architecture combines the semantic feature learning ability of Word2Vec which preserves syntactic relationships with the sequential learning ability of LSTM networks. The input is reduced in dimensionality by using the pretrained Word2Vec embeddings which keep important

word representations that refer to semantic meaning. LSTM layers are especially adept at learning dependencies in an order or sequential manner, something that makes them appropriate for learning sequentially over textual data. The hidden layers of this structure extend the features discovered by the LSTM further, and the output layer recombines these to create the required predictions for the target task.

4.5. Performance Analysis

There are many assessment metrics which can be used to evaluate an ML model. The most important thing with these indicators is not only how to tell if the algorithm is indeed working but it is also necessary to alter its parameters to improve the ML model. This thesis will solely discuss metrics about the classification problem as the trained model must automatically discover and determine whether code exhibits vulnerabilities. In this approach, we will evaluate using accuracy, precision, recall, confusion matrix, AUC curves, and F1-score.

i). Accuracy

In ML where accuracy is one of the key performance metrics that gives the accuracy of the model by measuring the number of correct predictions out of total number of predictions [10]. It is a very straightforward measure to make that lets us understand how much the model predicted the target outcome correctly. In its essence, accuracy is defined as a few accurate predictions divided by the number of all predictions, and it is more suitable being used for measuring the model's performance in the case when the cost of wrong classification is low.

ii). Precision and Recall

Precision is the metric that tells us how many of our positive predictions are correct. It is the ratio of TP over the number of TP and FP. The frequency with which the model is positively correct on the existence of positive data is an indicator of the consistency of its positive predictions[10]. Since the model is highly precise the outcome is likely to be accurate when it predicts positive as this reinforces confidence in the model's positive classifications.

Yields the recall metric, which is often referred to as sensitivity or TP rate. It measures the percentage of TP that the model correctly predicts [10].

While recall and precision should both be maximized, there are frequently trade-offs between them, so increasing one could mean lowering the other. Consequently, it's critical to strike a balance between the two based on the issue at hand and its demands.

iii). *Confusion Matrix*

A table that describes a model's performance is a confusion matrix, showing how many positives and negatives the model correctly and incorrectly classified. It evaluates the model's accuracy

by showing four possible outcomes, TP, TN, FP, and FN [10]. A data point's TP indicates that it is likely to belong to a particular category, while its TN indicates that it is likely to not belong to a particular category. FP classifies a data point as belonging to a category it doesn't, and False Negatives are the cases where the model does not classify a data point that it does belong to. These metrics together shed light on how exactly our model performs on classification tasks.

Based on correlation terms, statistical measures, including TP, TN, FP, and FN (Figure 4.7), are computed to create this matrix [157]. It is significant to remember that an ML model generates a continuous probability value, which is then, in many situations, converted, instead of producing a true or false output, into a positive or negative number using a median of 0.5. Comprehending

These terminologies is essential for computing the residual metrics, which include accuracy, recall, precision, and AUC curves.

		Actual	
		1	0
Predicted	1	True Positives (TP)	False Positives (FP)
	0	False Negatives (FN)	True Negatives (TN)

Figure 4.7: Confusion Matrix

iv). F1-Score

A harmonic indicator of recall and precision is the F1 score, 1 represents the highest score and 0 represents the lowest. Two models with low recall and high precision can be compared and vice versa [10]. By counting the number of cases that are correctly identified and not overlooked, the F1 Score evaluates the model's accuracy and robustness mathematically.

v). *AUC-ROC*

In Figure:4.8, AUC-ROC [158] is the Area Under the Curve (AUC) of the Receiver Operating Characteristic (ROC) curve.

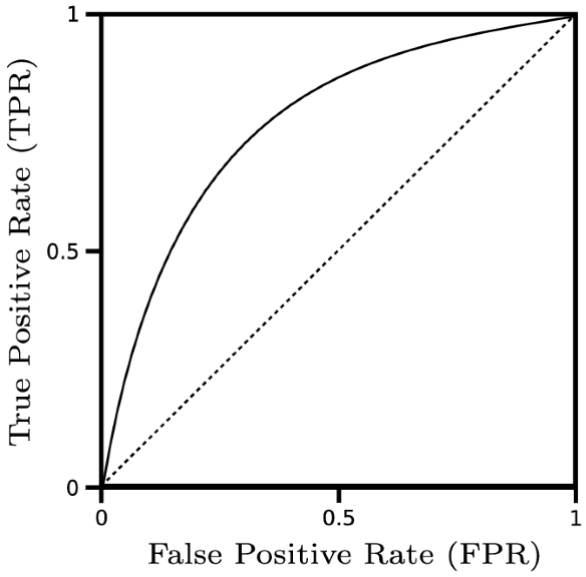


Figure 4.8: AUC-ROC [158]

In binary classification, ROC is an evaluation metric that is a probability curve where x and y axes are FP and TP rates. Alternatively, we can also say that this curve gives the performance of a classification model across all possible classification thresholds. However, AUC is a measure that assesses the model's overall ability to differentiate between classes [159].

vi). Performance metrics and methodology

In this thesis, it is important to explore how changes in the model design, especially hyperparameter tuning, can affect the expected outcomes so one can establish a clear connection between a model's performance metrics and methodology. For example, the most crucial performance metrics, including accuracy, precision, recall, and F1 score, are impacted by the hyperparameters, learning rate, batch size, and number of training epochs. Fine-tuning these parameters allows for underfitting and overfitting, helping create a better relationship between the model being able to correctly classify vulnerabilities.
Furthermore, threshold tuning is particularly important for metrics such as the AUC-ROC, which calculates the model's performance under different classification thresholds. Changing these thresholds allows for the tradeoff between true positive and false positive rates to be managed by the model, giving optimal response based on different categories of vulnerabilities.

Model evaluation depends crucially on understanding how these parameters are taken into performance metrics. For example, the confusion matrix gives valuable insight into how it differentiates between different class points where it could need more work to develop. However, these insights are invaluable to improving the model's architect and hyperparameter configuration. Using these performance indicators, iteratively, the methodology attempts to get aligned with the objective to achieve high reliability in identifying C# vulnerabilities, while accommodating to previous dataset and domain specific needs. The model's efficacy for

vulnerability detection can be continually improved via careful analysis of its performance for deriving values of parameters.

CHAPTER – 5

Implementation

5.1. Overview

This chapter provides the practical implementation and experimentation of several ML and language model techniques to detect source code vulnerabilities. It builds upon the theoretical framework established in previous chapters, translating research methodologies into functional, comparative applications. In the first section, we discuss our data preparation process, including data preprocessing to achieve good quality and meaningful inputs needed for the training of models. In this phase, the dataset is cleaned, normalized, and augmented to represent real-world software vulnerabilities and correct problem cases. Three different approaches are implemented to address this problem, using open-source LLMs through fine-tuning, using proprietary LLMs through API calls, and using a neural network approach to compare models. All these elements play a part in the goal of performance evaluation of vulnerability detection across different methods.

5.2. Dataset Preparation

In the dataset, Each C# file is organized within its folders, which helps separate types of vulnerabilities and other test cases. These C# files contain functions that are critical for research and analysis in the field of vulnerability detection, and each function may represent one typical coding pattern or flaw that may result in a security risk. The first step for us to be able to analyze or extract data from these functions is to retrieve the filenames individually. This is necessary because each file contains different function bodies that must be treated or analyzed separately.

To do that, we wrote a method that passes through the data set's folder structure and finds the corresponding C# file in its coordinating subdirectory. Since the dataset consists of files across different folders to mirror some test cases with related vulnerabilities. For example, in the Juliet Test Suite, the files are divided into categories per specified vulnerabilities, for instance, SQL Injection (CWE89) and cross-site scripting (CWE79), to name a few. Inside these categories, subfolders are used to further group different types of vulnerabilities, each with many C# source files.

In the next step, we read C# files to identify function names within files. The next critical step is to differentiate vulnerable and non-vulnerable functions, which are the key steps to the overall goal of automatic vulnerability detection. In this stage, our approach utilizes the entity sets to extract the names of the functions in code files, most of which contain a single vulnerable function together with one or two non-vulnerable functions. These functions are used for both supervised machines learning as well as rule-based security analysis systems.

First scan the code line by line and check the presence of an opening parenthesis that is the sign of a function declaration in C# code. It is the start of the parameter list. Next, we extract the function name, and this line is split at the first occurrence of the opening parenthesis to do this. This approach separates the function name, its modifiers and the parameters to appear after.

Then after splitting the line, we split the portion before parenthesis further into smaller parts on basis of space. Normally, in this segment, the function name is the last part, which follows the modifiers or return type. The final element is selected for the function name successfully.

After extracting function names, we again scan the C# files. The file is opened and read line by line. Each line is then processed by the program and identified whether the target function is contained within that line or before it. With the ability to review each line of the code, we can isolate specific sections on a per-line basis, which is helpful for the function body of the code. Once the function declaration is found, subsequent lines comprising the function's body are grabbed. All lines inside the curly braces are considered. The way it works is we count the number of openings as well as close curly braces and make sure the total function body inside it, including nested structures such as loops or conditionals enclosed in the function, is properly gathered.

5.2.1. Dataset Clean

In this process, we focus on cleaning the dataset by removing comments and irrelevant code segments from C# files. This is a cleaning phase which ensures that for analysis only the functional part of the actual code will remain.

The first thing we will do is find and remove single-line comments, usually signified by Comment delimiter (//). Everything after this marker is removed leaving what was placed before the comment. Similarly, multi-line or block comments, enclosed between (/*) and (*/), are identified and removed. The logic ensures that the content inside these comment blocks is skipped, preventing the inclusion of unnecessary text.

Additionally, any inline characters used for documentation or commenting purposes, such as asterisk (*) or hash (#), are also removed from the lines of code. These characters often appear in comments or annotations and are stripped out to retain only the executable part of the code.

By nature, comments are non-executable. However, since LLMs may consider comments to be part of the source code, they may attempt to understand it along with executable instructions. Additionally, mixing functional and non-functional text confuses the model and can introduce noise into the training process. Thus, the model may not detect or classify vulnerabilities that it would have done based purely on the functional code.

For the development of datasets intended for training LLMs, renaming function names is often a very important task when function names may already have an inherent positive meaning of ‘good’ or a negative meaning of ‘bad’ as regards their security or vulnerability. This renaming process prevents LLMs from using the names as a simple experiment to make predictions but rather to look at the actual code flow. So, we replace function names with random names to prevent unnecessary identification while maintaining the credibility of LLM-based vulnerability detection systems.

5.2.2. Dataset Splitting

To split the dataset, we started with a total of 9,863 records, consisting of function samples that are both vulnerable and non-vulnerable. To ensure an effective model training and evaluation process, we divided this dataset using an 80:20 ratio. This means 80% of the records were allocated to the training set, providing a large and diverse sample for the model to learn from, while the remaining 20% were reserved for testing purposes. This division helps assess the model’s generalization ability,

allowing us to evaluate its performance on unseen data after training. The 80:20 split is a commonly used practice, balancing the need for a substantial training dataset with sufficient test data for robust performance validation.

After creating the initial 80:20 split between training and testing datasets, we further divided the 80% training dataset into two subsets for fine-tuning and validation. Specifically, we used a 90:10 split for this purpose. This approach ensures that 90% of the training data is used for actual model fine-tuning, providing a large sample size for learning, while the remaining 10% is used as a validation set to monitor and optimize the model during training. This additional split helps assess the model's performance after each training epoch, allowing for adjustments in hyperparameters and preventing overfitting. By maintaining a separate validation dataset, we ensure that the model is not just learning patterns specific to the training set but is capable of generalizing to unseen data.

5.3. Open Source LLMs Fine Tune

We applied the same structured process to finetune LLaMA 2-7B and CodeGen2-multi models to guarantee consistency in the training procedure across models. We designed the process of fine-tuning to maximize the training effectiveness of the LLMs while at the same time maintaining the model performance. Following the process for LLaMa 2-7B and CodeGen2-multi models, we used fine-tuned LLMs.

To fine-tune LLMs, we need to optimally run them so that they do not become computationally infeasible on the resources we have available on our local PC, and training such models on a CPU is impractical. So, as a solution, we used the Google Colab T4 environment to finetune the LLMs, and it became crucial to cut a significant amount of training time. The process of building these models takes advantage of several essential libraries and tools for performing model loading, quantization, and efficient parameter tuning.

As essential Libraries for Fine-Tuning, Hugging Face Transformers, is fundamental for loading and managing LLMs. The Bitsandbytes is used for memory efficiency and allows to load models in 4-bit precision, a feature which can be used in BitsandbytesConfig class. This, in turn allows for model quantization, an important technique for compressing deep learning models by decreasing the number of bits needed to represent their weights and activations, which facilitates faster implication, less memory usage, and opens the possibility of utilizing existing hardware designed for quantized workloads. It also has tools for training and inference scaling, as well as parameter-efficient fine-tuning. The Transformer Reinforcement Learning library enables training models with reinforcement learning, extending the support of fine-tuning in a flexible manner.

5.3.1. Model Access and Quantization

When working with a model like Meta's LLaMA 2, one must acquire access through platforms like Hugging Face Hub. Loading such a model involves creating a BitsAndBytesConfig to

define quantization settings. Quantization techniques like FP4 (the default) or NF4 (a newer 4-bit precision type) help minimize the model's memory footprint. The bnb_4bit_use_double_quant and bnb_4bit_compute_dtype parameters further fine-tune this compression, making inference more efficient.

There are several steps to load the LLMs. These include setting GPU memory limits, selecting the appropriate device (GPU:1) and initializing the model tokenizer, also set up the model. Preprocessing inputs is crucial, and tokenizers convert text into tokens the model can understand. The padding tokens can be leveraged as special tokens like the End of Sentence token to make all sequence lengths consistent.

Input token sizes must be managed to guarantee the model can successfully process data. So first, we find our model maximum sequence length is usually set to 1024 tokens. This limit is enforced so that the preprocessing is carried out only with sequences that do not exceed the capacity of the model.

To eliminate the need for tokenized data, we introduce functions like preprocess_batch that truncates and pads sequences to a maximum length that our model would accept. Prompt formatting, filtering large prompts, and randomization for training are the final dataset preprocessing.

The growing size of such models is one big problem in fine-tuning LLMs. With models such as LLaMA 2–7B, full fine-tuning becomes too costly for consumer hardware to run. However, PEFT techniques such as QLoRA do the job efficiently by freezing nearly all the model's parameters and only fine-tuning a small subset. In addition, this method utilizes 4-bit quantization to decrease memory consumption while continuing to have performance.

Applying LoRA and Calculating Trainable Parameters, Finding the appropriate modules to apply LoRA involves the find_all_linear_names function, which identifies linear layers where LoRA can be inserted. The number of trainable parameters is then calculated using print_trainable_parameters, ensuring only a tiny fraction of the model is updated during finetuning.

The fine-tuning process itself is encapsulated in the fine_tune function. This function initializes gradient checkpointing to minimize memory use, configures the model for training with PEFT, and applies LoRA to the designated modules. The model is then wrapped for fine-tuning, and training proceeds using a trainer object. This object manages training hyperparameters like batch size, learning rate, and optimizer settings while controlling gradient accumulation and mixed-precision (FP16) training. It logs training and saves checkpoints for possible future interpretation.

At the end of the training, when there is, the fine-tuned model is merged with its original weights and the saved model and tokenizer are saved. After optimization, this model can be pushed to a Hugging Face Hub platform for future use. The focus of the entire process is to keep the balance

between maintaining model performance, computational efficiency and storage overheads using PEFT techniques, such as QLoRA.

5.3.2. Hyperparameters

During the fine-tuning of the LLaMa 2 and CodeGen2 model, we carefully selected several key hyperparameters to optimize performance. The model was trained with num_train_epochs set to one, allowing a single pass through the entire dataset. We increased the number of epochs, the model started overfitting and nearly performed poorly on unseen data. To resolve this issue, we trained the models using a single epoch. To help the model adjust in the early stages of training, we included warmup_steps in two, which gradually increased the learning rate at the beginning. The optimizer used was paged_adamw_32bit, a memory-efficient variant of the Adam optimizer that works well with large models. We set the learning_rate to 2e-4 to ensure a stable learning process, and the per_device_train_batch_size was set to 8, which effectively balanced memory constraints and training speed. When we increased the value of this parameter, the model required more GPU power, often resulting in out-of-memory errors in our training environment. However, a reduction of the batch size didn't significantly affect the performance of the models in this case. Therefore, to ensure that our hardware specifications and to improve the effectiveness, we decided to keep the batch size value as eight. Additionally, gradient_accumulation_steps was set to four, meaning gradients were accumulated over four batches before updating the model, effectively increasing the batch size without requiring more memory. The task type for fine-tuning was defined as "CAUSAL_LM," suitable for causal language modelling tasks.

Table 5.1: Hyperparameters for LLaMA2 and CodeGen2 models.

Hyperparameter	**LLaMA2**	**CodeGen2**	**Description**
per_device_train_batch_size	8	8	Number of samples processed per GPU during training.
gradient_accumulation_steps	4	4	Steps to accumulate gradients before weight updates.
learning_rate	2e-4	2e-4	Initial learning rate for AdamW optimizer.
optim	paged_adamw_32bit	paged_adamw_32bit	Memory-efficient optimizer used during training.
warmup_steps	2	2	Steps for warming up the learning rate linearly.
fp16	True	True	Enables 16-bit floating point precision for faster training.
logging_steps	1	1	Interval of steps for logging training progress.
per_device_eval_batch_size	0	2	Number of samples per GPU during evaluation.
num_train_epochs	1	1	Number of complete passes through the dataset during training.

Interestingly, we observed the Lama2 model's consistent accuracy and results even when we varied the dataset size. When initially used a split of 90% for training and 10% for testing, the model delivered a stable performance, achieving reasonable accuracy during testing. But when the CodeGen2 dataset training set is increasingly like 80% to 90%, it seems like it is getting overfitting. However, to ensure fairness and consistency across all other models and to avoid potential bias from the training data, we ultimately chose a split of data when we discussed the data splitting section ratio. This final setup provided balanced results, with similar testing outcomes across different models and configurations. Overall, the fine-tuning process effectively achieved reliable performance, regardless of the dataset size.

5.4. Open Source LLMS Validation

LLaMa 2: We performed the validation separately using the validation dataset because, during the training process, running validation caused a sudden spike in memory usage, exceeding 40GB of GPU memory. Due to these resource limitations, it was not feasible to run validation alongside training. Therefore, we started by fine-tuning the models and downloading them from Hugging Face. We then performed the validation in isolation to avoid overcoming the GPU and ensure smooth performance given the hardware limits.

We have written the code to evaluate a model on a validation dataset in parts which can be assessed at a time when working with a large dataset or with limited computational resources is concerned. Unlike the rest of the dataset, the dataset is divided into chunks so that the memory is used at a minimum capacity and to make sure that the processing is smoother.

We evaluate the process in a loop whereby the dataset is split into smaller chunks of a defined size. For each iteration, a subset of the validation dataset is evaluated separately. Such an iterative method brings a reduced memory footprint and is practical in cases when the hardware has limited memory capacity.

In each iteration of the loop, we create a new instance of the Trainer class. The Trainer is the one in command of evaluating by establishing the default model training arguments and data collator and the current part of the validation dataset. This modular approach allows the model to be assessed on different dataset sections without overwhelming system resources.

In each iteration, the model is validated on the current subset, and its resulting evaluation loss is recorded. This allows the model to store these evaluation losses for later analysis, so that we can see the model’s performance in different parts of the dataset. Additionally, for each step performed, we track the values of steps, which can be helpful for plotting and seeing the overall model behaviors throughout each step of evaluation.

The code divides the validation process into small pieces, and iterate over them in this way because this guarantees that if we have a large dataset or have a limited computational resource,

our validation process runs well. This approach is resource efficient, with a good balance between accurate usage evaluation.

CodeGen2: During the CodeGen2 model fine-tuning process, we implemented a validation process to monitor and evaluate model performance. We configured several parameters to ensure the model's effectiveness and efficiency during this phase.

The validation was performed using a per-device batch size of two, enabling smaller chunks of data to be evaluated per device and facilitating efficient GPU memory management. Additionally, we set the training epochs to one the model passed through the entire dataset once during fine-tuning.

Validation was carried out on a separate evaluation dataset, allowing us to check the model's performance at each step of the fine-tuning process. This setup provided a structured approach to measure the model's effectiveness and stability and helped track and analyze improvements across each validation step.

5.5. Open Source LLMs Testing

To test the use of LLMs for vulnerability detection in C# code, we developed a prompt-based testing approach. The end goal was to understand the accuracy with which model categorizes code snippets into 'vulnerable' and 'not vulnerable' by using an understanding of provided inputs. The test dataset was used, and each code snippet was processed individually.

For every instance, the prompt was constructed to instruct the model exactly its specific task very deliberately. The model was informed that it would function as a sophisticated C# vulnerability detection system and should return a single-word response whether it is either "vulnerable" or "not vulnerable." This became a very clear instruction that led the model to a pure binary classification task without adding any details or explanations that could distract it.

Once the prompt was ready with the code snippet, then, we tokenized it so that the model could consume it. The model was then tested with the tokenized input. Given that the model acted in an inference mode, we turned off the gradient calculations to save resources and memory. Additionally, with the use of beam search, we were able to make the model output the best results possible and as stable as possible. A temperature setting was applied to balance randomness and determinism for consistent yet flexible predictions.

Then, after producing the model's output, the response was decoded, and the relevant classification was extracted. We added the result to a list of predictions and measured the performance of the model by comparing true labels in the dataset with the predictions. This method allowed us to test more reliably and efficiently whether the model can classify C# code vulnerabilities.

5.5.1. Lama2 Performance Analysis

This experiment used meta-llama/Llama-2-7b-hf, a model fine-tuned to create the Finetune-llama-2-7b model. It has a total of 3.54 billion parameters and 40 million trainable parameters representing 1.13% out of the total. A set of 7,101 prompts was used during training time and trained on one epoch. It all took around 2h 40 minutes. We measured the performance of the model on training, with a training loss of 0.1737, throughput of 0.73 samples per second, and 0.023 steps per second. These metrics give an excellent clear glance at the efficiency and effectiveness of the fine-tuning process.

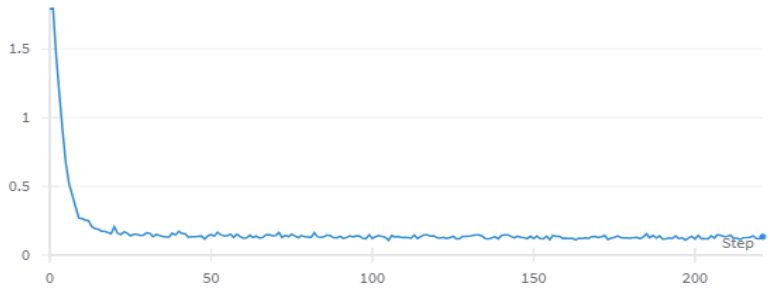


Figure 5.1. Result of the training loss of the fine-tuned LLaMA-2 model.

The above graph illustrates the training loss of the fine-tuned LLaMA-2 model over training steps. The number of training steps is indicated by the x-axis, while the loss values are represented by the y-axis. At the start of the training, the loss value is relatively high, around 1.7, which is common when the model is being adjusted to the task at hand. Within the first few steps up to approximately 50 steps, the loss decreases rapidly, suggesting that the model quickly learned the basics of the training data. After around 100 steps, the loss begins to stabilize, oscillating around a much lower value near 0.1, indicating the model reached a state where further training brought diminishing returns.

This smooth curve demonstrates that the model is improving and converging effectively during the training process, with no signs of overfitting (no sudden spikes in loss). The stabilization at lower loss values reflects that the model has learned the task well after the initial sharp descent.

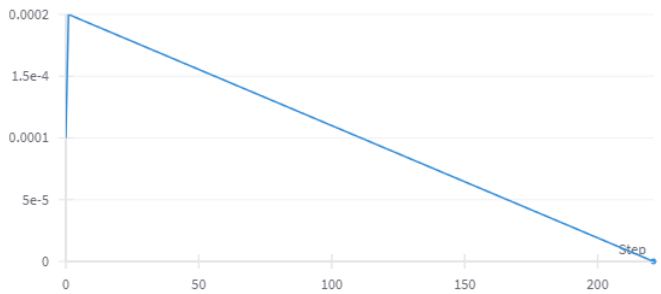


Figure 5.2. Result of the learning rate of the fine-tuned LLaMA 2 model.

The above graph displays the learning rate schedule of the fine-tuned LLaMA 2 model during training. The number of training steps is indicated by the x-axis, while the learning rate is represented by the y-axis. The learning rate starts at around 0.0002, indicating a relatively high learning rate at the beginning of the training. A higher initial learning rate helps the model quickly adjust its weights during the early stages.

The learning rate decreases steadily and linearly throughout the training process, reaching near zero by the end of 200 steps. This learning rate decay strategy is designed to help the model converge smoothly by making smaller updates as it gets closer to an optimal solution.

Gradually reducing the learning rate during training allows the model to fine-tune its parameters with greater precision in later stages, which helps avoid overfitting. This approach leads to a more stable model, as shown by the levelling of the loss values in the final part of the training process

In summary, the learning rate fall-off allows the model to start learning aggressively and then slows down once nearer to a better solution, both aiding rapid loss end and stabilizing it.

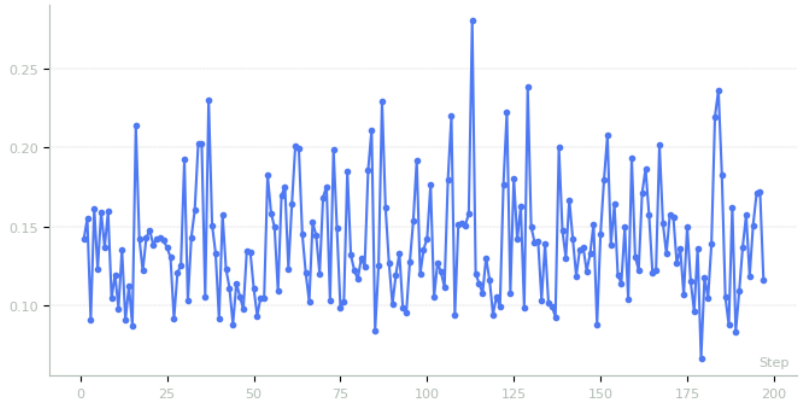


Figure 5.3. Result of the validation loss of the fine-tuned LLaMA 2 model.

Figure 5.3 shows the validation process result. This validation was done after fine-tuning the model, the loss values fell in a narrow range between 0.10 and 0.20, establishing that the model's performance did not vary significantly. This shows that the model is not experiencing extreme overfitting or dramatic degradation in performance but rather moderate oscillations in validation loss.

The overall accuracy of the model indicates that 91% of all predictions made were correct. This high accuracy reflects the model's overall reliability.

Precision, or the positive predictive value, is 97%. This metric evaluates how many instances predicted as "vulnerable" were vulnerable. A precision of 97% suggests that when the model predicts a code as vulnerable, it is correct 97% of the time.

The recall, or sensitivity, of the model is 72%. This represents the ratio of actual vulnerable individuals accurately identified by the model. While the model excels at avoiding false positives, its recall indicates that 28% of the vulnerable cases were missed.

Calculating the F1 Score involves determining the harmonic mean of precision and recall, which is 83%. It balances precision and recall, offering a metric that reflects the model's performance in classifying vulnerable cases. This score offers a fair assessment that considers both false positives and negatives.

The AUC score of 85% indicates that the model has an excellent ability to distinguish between the vulnerable and not vulnerable classes. An AUC of 85% signifies that there is a chance that the model will rank a randomly chosen vulnerable instance higher than a randomly non-vulnerable instance selected.

The model performs exceptionally well in terms of precision, meaning it is highly reliable when it identifies an individual as vulnerable. However, its recall is somewhat lower, suggesting that the model misses a notable portion of the vulnerable cases. This trade-off between precision and recall is a common challenge in classification tasks, mainly when the cost of false negatives missing vulnerable individuals might be significant.

The following shows a confusion matrix that outlines the performance of a classification model.

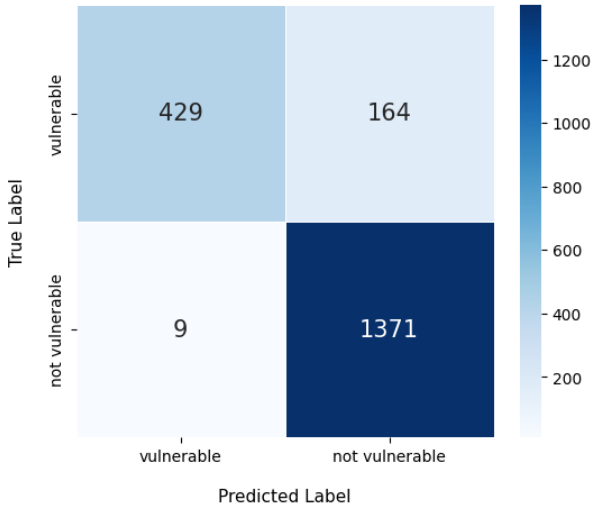


Figure 5.4. Confusion Matrix of the LLaMA 2 Model.

The LLaMA 2 model confusion matrix to perform the classification of code vulnerabilities. The reliability of this performance shows that the model can correctly identify "not vulnerable" instances with a low false negative rate minimizing overlooked vulnerabilities. Yet the increase in false positives indicates that precision could lag, and it may not appropriately classify non vulnerable cases from vulnerable ones. Overall, the matrix shows a model with high classification ability and opportunity for additional optimization of precision and recall tradeoff.

5.5.2. CodeGen2 Performance Analysis

The model used for this study is Salesforce/codegen-2B-multi, a large-scale code generation model. The Fined-tuned model was pushed to the FineTune_codegen-2B-multi_V1_4 model. This model has a total of 1.54 billion parameters, after finetuning 20.97 million trainable parameters, representing 1.35% of the total parameters. Fine-tuning was carried out using 7,101 prompts over one epoch, and the process took one hour and forty-nine minutes to complete. During training, the model processed 1.07 samples per second and achieved 0.03 steps per second, which indicates the task's computational complexity. The final training loss was recorded at 0.19, suggesting a solid convergence toward the target output.

Figure 5.5. Result of the training loss of the fine-tuned CodeGen-2 model.

Figure 5.6 graph illustrates the validation loss during the fine-tuning process of the CodeGen2 model. From the graph, we observe a sharp decline in validation loss at the initial evaluation steps, which indicates that the model quickly learned and adjusted during the early stages. Following this steep initial drop, the decrease in validation loss then becomes much less gradual before eventually stabilizing at an evaluation step of around 10. At the end of the process, the final loss value touches near 0.13, demonstrating a relatively consistent level of performance reached by the model.

Such a trend in loss reduction is a great indicator of the model's learning process, that the model can adapt well and tends to prevent any drastic overfitting. This validates the robustness of CodeGen2's training, which itself proves a perfect fine-tuning process.

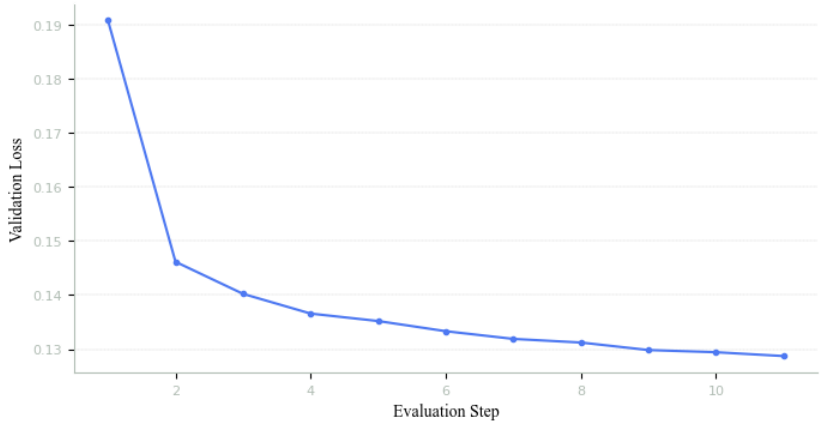


Figure 5.6. Result of the validation loss of the fine-tuned CodeGen-2 model.

Several key metrics show that the CodeGen2 model performs extremely well at achieving its task. The accuracy value of 97% shows that our model successfully predicts the output of them, 97% of the data of the dataset was correctly guessed by the model. Additionally, the F1 score of 95%, which balances precision and recall, further supports the model's strong performance, highlighting that it maintains a good trade-off between identifying true positives and avoiding false positives.

Especially, the model's precision is strong, achieving a perfect score of 100%, that means every prediction made by the model positive that found is true. The model recalls a score of 91%, which implies that the model is exact but missed a few of the actual positive cases, so there is space for improvement in tracing all the important cases.

Finally, the value of AUC of 95% is very good for separating positive and negative classes. This, therefore, indicates that the model does not just succeed well, it can also distinguish different categories within the dataset with discriminatory solid power. Overall, these metrics together suggest that the CodeGen2 model is significantly performing well in this task.

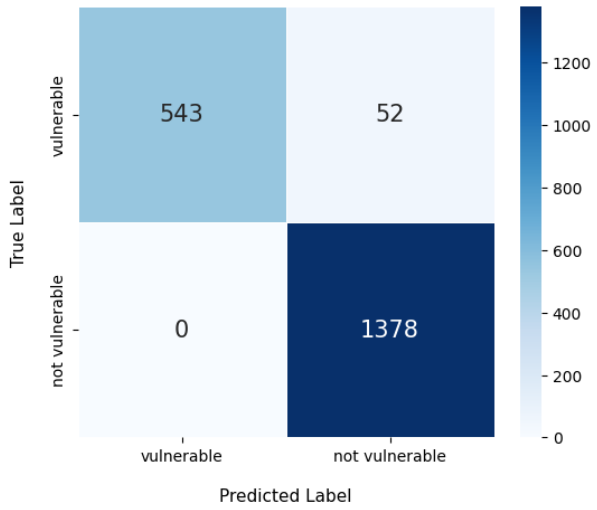


Figure 5.7. confusion matrix of the fine-tuned CodeGen-2 model.

This confusion matrix demonstrates a very good behavior of the model, specifically in identifying evidence of perfect classification in "not vulnerable" applies with no false negatives. Additionally, the model effectively identifies vulnerable individuals with relatively few errors.

The near-perfect classification results indicate that the model has learned the task well and is highly accurate in its predictions. However, the small number of false positives suggests that the model still occasionally misclassifies non-vulnerable cases as vulnerable, which could be an area for further improvement, depending on the application's tolerance for such errors. Overall, this performance indicates robust and reliable classification capabilities.

Finally, we demonstrate the performance of the fine-tuned CodeGen2 model using the confusion matrix on identifying vulnerabilities and not vulnerable. The model has high accuracy and reliability, it does not have false negatives for the "not vulnerable" cases and a relatively high true positive rate in the "vulnerable" predictions. Nevertheless, 52 false positives indicate that the precision may still be improved. Overall, the model shows robust classification ability and can be used in vulnerability detection tasks.

5.6. GPT Models Testing

To evaluate proprietary GPT models for C# code vulnerability detection, we tested several versions of the GPT models using an OpenAI-based prompt. The goal was to compare the performance of four different models: GPT-3.5-Turbo and GPT-4, both in their C# vulnerability identification and security assistant roles. Each model was tested with the same input data and prompt structure to ensure consistency in testing.

For each model, the system message was tailored to set the context. Two variations of GPT-3.5-Turbo were tested. The first defined the model as a C# vulnerability identification expert, the purpose of the model being to classify the code snippets based only on their vulnerability status. The second version positioned the model as a "security expert assistant," providing similar task instructions but focusing on broader security expertise. These distinctions aimed to assess whether framing the model differently impacted the output.

Similarly, GPT-4 was tested with two configurations. One defined it as a chat completion vulnerability expert, while the other treated it as a security assistant. All models were evaluated using the same prompt structure to determine consistency across the models.

The prompts were designed to be simple, clear, and focused as they asked the model to decide whether a C# code snippet contains vulnerabilities in 'Yes' or 'No'. This binary response format was chosen to streamline the testing process and ensure that the models provided consistent, unambiguous outputs across all code snippets.

Prompt One:
Indicate whether the security vulnerability is present in the provided C# source code using the Yes/No format: 'Yes' means that there are vulnerabilities. 'No' means that there are no vulnerabilities. Return only the label.
Code: {}
Label:

Prompt Two:
Following C# method include a security vulnerability? Answer in Yes/No format: 'Yes' means that there are vulnerabilities. 'No' means that there are no vulnerabilities. Return only the label.
Code: {}
Label:

Prompts Three:
Respond if this function is vulnerable? Yes/No Format:
'Yes' means that there are vulnerabilities. 'No' means that there are no vulnerabilities.
Return only the label.
Code: {}
Label:

Each model processed many test cases, and predictions were generated and compared against actual labels for every snippet. Throughout the testing process, rate limits were observed to manage the API usage, with timed delays introduced when the request limits were reached. This ensured a smooth and continuous testing flow without violating API rate restrictions.

Overall, the evaluation compared two key dimensions, the version of the model (GPT-4 vs GPT-3.5 Turbo) and the framing of the task. The goal of the results was to determine which combination of features would best classify C# code vulnerability. This test's findings could

help to shed light on how prompt framing and model versions affect output results in specialized tasks such as code vulnerability detection.

5.7. GPT Models Performance

For the GPT model performance evaluation we have chosen the metrics Accuracy, Recall, Precision, F1 Score, and AUC to determine the effectiveness of each model.

5.7.1. GPT-3.5 – Chat Completion Mode

In GPT-3.5 chat completion mode, here are three prompts' evaluation metrics, presented below. The model is evaluated against these metrics and considers prediction on each prompt's capability.

Table 5.2: GPT-3.5 – Chat Completion Evaluation Metrics for Three Different Prompts

Prompt	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**AUC**
Prompt 1	0.46	0.34	0.87	0.49	0.58
Prompt 2	0.50	0.37	0.91	0.52	0.62
Prompt 3	0.44	0.34	0.94	0.50	0.58

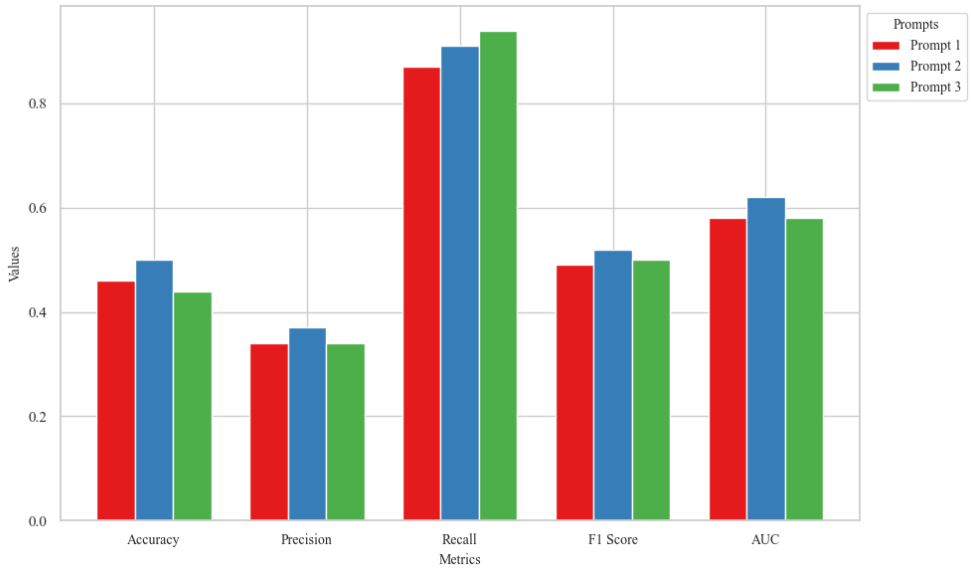


Figure 5.8: GPT-3.5 – Chat Completion Evaluation Metrics for Three Different Prompts.

5.7.2. GPT-3.5 – Assistant Mode

Below are three prompts' evaluation metrics in GPT 3.5 assistant mode. We evaluate the model with these metrics, considering the capability of each prompt to provide prediction.

Table 5.3: GPT-3.5 – Assistant Evaluation Metrics for Three Different Prompts

Prompt	Accuracy	Precision	Recall	F1 Score	AUC
Prompt 1	0.47	0.35	0.92	0.51	0.59
Prompt 2	0.58	0.44	0.93	0.60	0.66
Prompt 3	0.50	0.40	0.94	0.56	0.61

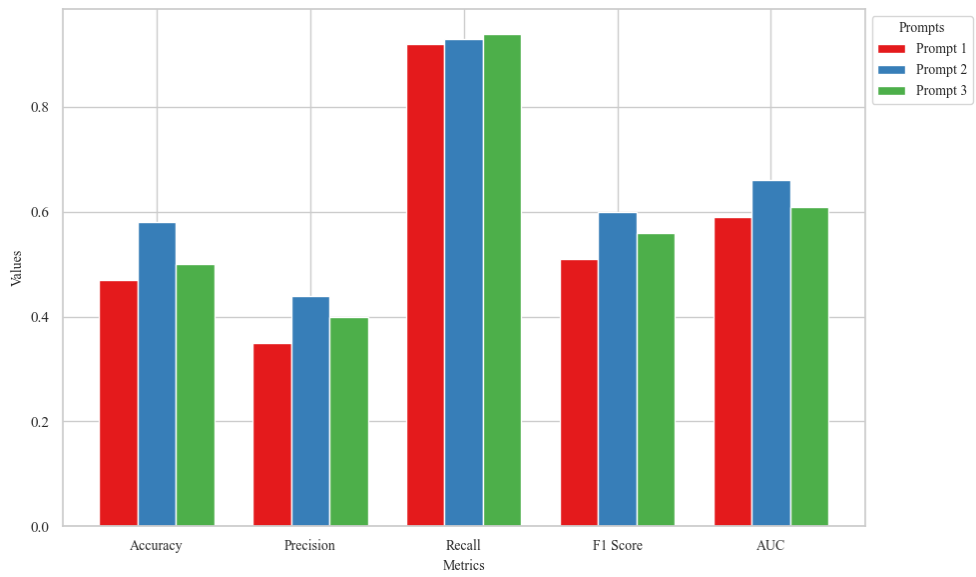


Figure 5.9: GPT-3.5 – Assistant Evaluation Metrics for Three Different Prompts.

5.7.3. GPT-4 – Chat Completion Mode

Below Table 5.3 includes three prompts' evaluation metrics in GPT-4 chat completion mode. Based on these metrics, we evaluate the model based on the capability of each prompt to provide prediction.

Table 5.4. GPT-4 Chat Completion Evaluation Metrics for Three Different Prompts

Prompt	Accuracy	Precision	Recall	F1 Score	AUC
Prompt 1	0.53	0.48	0.99	0.64	0.59
Prompt 2	0.57	0.50	0.99	0.66	0.62
Prompt 3	0.55	0.49	1.00	0.65	0.60

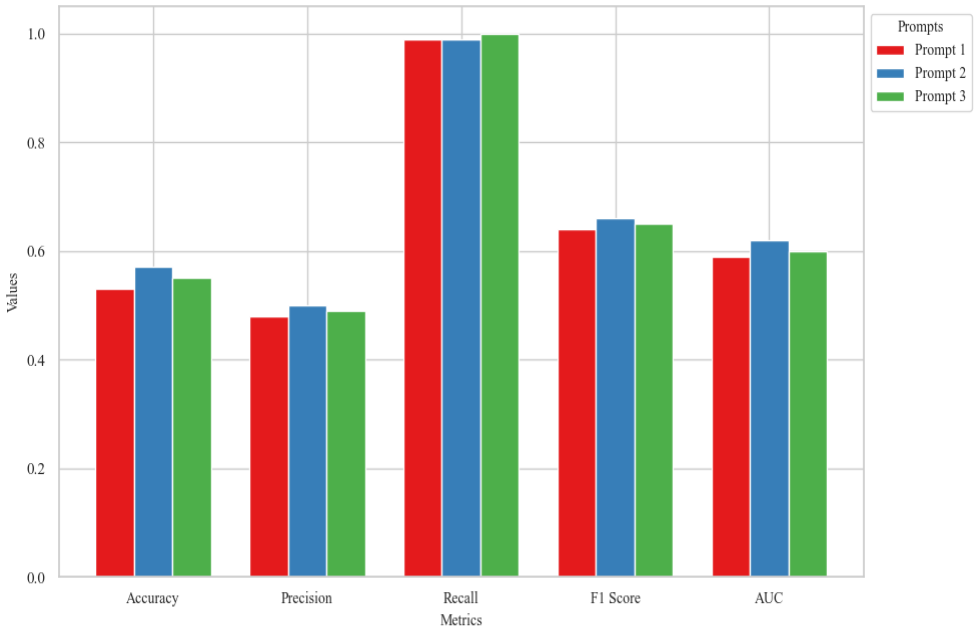


Figure 5.10: GPT-4 Chat Completion Evaluation Metrics for Three Different Prompts.

5.7.4. GPT-4 – Assistant Mode

Below are three evaluation metrics of prompts in the GPT-4 Assistant Mode. With these metrics, we evaluate the model using each prompt's capability to predict.

Table 5.5: GPT-4 – Assistant Evaluation Metrics for Different Three Prompts

Prompt	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**AUC**
Prompt 1	0.63	0.51	0.99	0.67	0.70
Prompt 2	0.55	0.46	1.00	0.63	0.63
Prompt 3	0.54	0.45	1.00	0.62	0.62

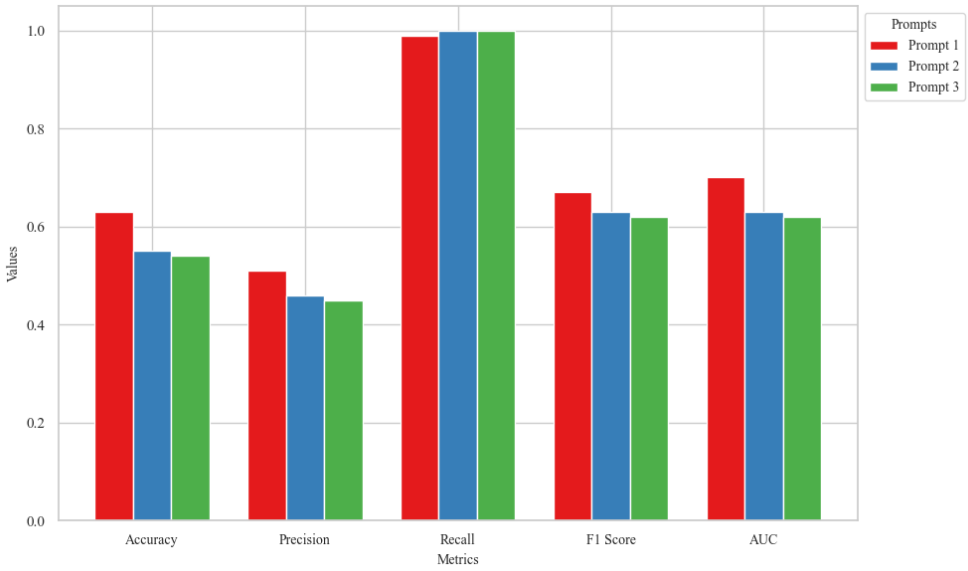


Figure 5.11. GPT-4 Assistant Evaluation Metrics for Three Different Prompts.

5.8. Neural Network Classifier

For the source code classification, we applied Word2Vec + LSTM neural network in this approach. Input was provided in two ways. The first one is plain code, and the second one is AST representations. In the first approach the code was directly fed into as the plain code and the model learnt from the raw stream of tokens. For the second approach, we utilized the code AST representation, together with structural code information to better understand and classify the code vulnerability.

Several studies show that Word2Vec embeddings with LSTM baselines are supported for source code vulnerability detection, and their effectiveness is well known. The problem of vulnerability detection in source code is well suited for LSTMs, because they naturally capture long term dependencies that are so crucial for detecting vulnerabilities. When combined with Word2Vec which maps code token into dense vector representations such that the model can learn syntactic and semantic relationships more efficiently [68][87], this capability is further enhanced. As described by Dessauw et al. (2022), the modular and expandable nature of testbeds highlights the necessity of adaptive approaches in machine learning based vulnerability detection model evaluation. By adding LSTM on to Word2Vec, there is a need for flexible and robust base methods which align with this, so that we can compare against advanced methods [162].

Additionally, it shows the effectiveness of recurrent neural networks in static vulnerability detection [68], which proves the robustness of using recurrent neural networks for the context filtering at the method level. They also pointed out that AI driven systems can be used for vulnerability detection in collaborative development environments, which will have good future

implications for multiparty software systems. Similarly, Xu et al. (2022) showed that large scale repositories can be used to good effect when training models, such as Word2Vec and LSTM [87] [80]

5.8.1. Hyperparameter Tuning

For our LSTM model, we perform hyperparameter tuning using the Keras Tuner. It is a powerful but extremely straightforward library, which allows us to automatically try out various hyperparameter combinations to tune our model for better performance. We did the hyperparameter tuning process by defining a function to build the LSTM model so that we can indicate which parameters get the best results.

We focus on several critical hyperparameters, such as number of LSTM units in each layer, dropout rate, and the learning rate of the optimizer. We achieve this by changing the numbers of LSTM units, to see how many of them are needed to learn complex patterns from data. We adjust dropout rates by randomly removing a fraction of neurons during training in order to prevent overfitting, which helps enhance the model's generalization capability.

To optimize the learning process, we also experiment with different learning rates. This is crucial, as the learning rate controls the step size at each iteration while moving toward a minimum of the loss function. To do this, we conduct a random search over a set of these hyperparameters to explore various configurations to discover the best combination of hyperparameters that cause the validation accuracy to maximize. The tuning process requires many trials for each configuration to perform robust evaluation and selection of the most performing model setup. Finally, this systematic approach improves the overall performance and reliability of our LSTM model in source code classification tasks.

First Approach: For a code classifier with plain code inside, we find that 32 units in both the first and second LSTM layers are the best hyperparameters for the LSTM model, finding the balance between capturing patterns on the code and drowning the model. The third LSTM layer of 48 units provides more capacity for deeper feature extractions. The first layer has a dropout rate of 0.4, which means that 40% of neurons are dropped randomly, and then a slightly lower dropout rate, 0.3, applies to all subsequent layers. The learning rate of 0.001 ensures a gradual, stable training process, allowing the model to learn and generalize from the sequence of plain code tokens.

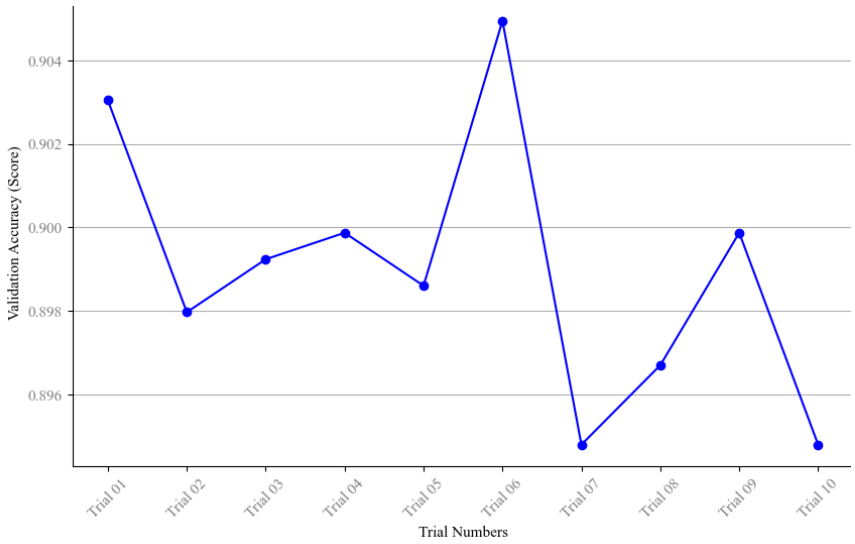


Figure 5.12. Word2Vec+LSTM Plane Code Input Hyper Parameter Tuning Result.

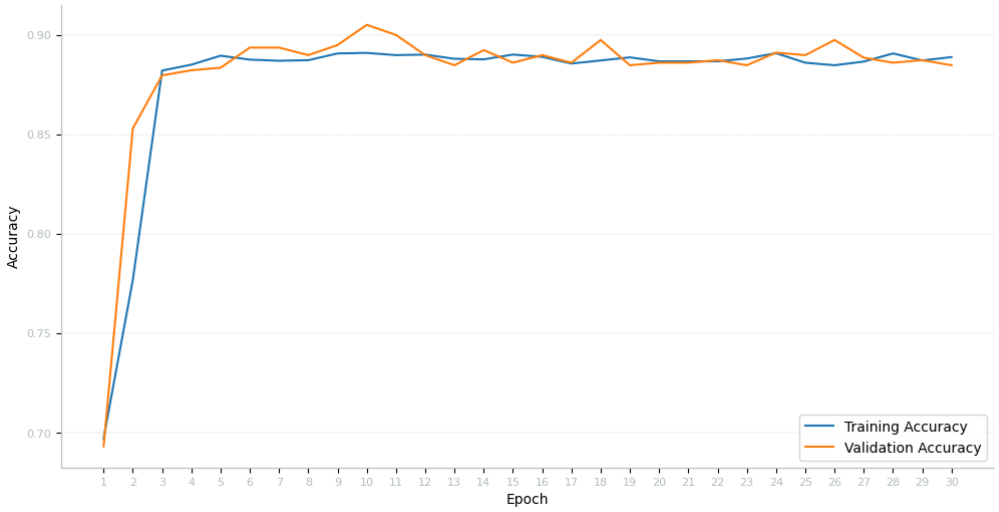


Figure 5.13. Word2Vec+LSTM Plane Code Input of Training and Validation Accuracy *Result.*

The graph illustrates the progression of training and validation accuracy over 30 epochs for a classification model, likely an LSTM neural network. Both curves follow a similar pattern, where accuracy improves sharply within the first few epochs before stabilizing. By the 5th epoch, training and validation accuracy surpassed 87%, indicating that the model quickly learns to classify the data effectively.

Throughout the remaining epochs, the accuracy fluctuates slightly but remains around 90%, with the validation accuracy generally aligning closely with the training accuracy. The model appears to generalize effectively to new data without overfitting, based on the strong correlation between training and validation accuracy, indicating the effectiveness of the chosen architecture and hyperparameters.

Second Approach: For the AST input model, the hyperparameter tuning process resulted in the following optimal configuration the first LSTM layer consists of 160 units with a dropout rate of 0.4, which helps prevent overfitting by randomly dropping 40% of the units during training. The second LSTM layer includes 64 units with a dropout rate of 0.2, followed by a third layer with 32 units and a similar dropout rate of 0.2. These layers were optimized to balance model complexity and generalization capability. Additionally, a learning rate of 0.0001 was chosen such that the model converges steadily and does not lead to overfitting the model. We found this set of hyperparameters to achieve the best accuracy and stability on the tasks of code vulnerability detection.

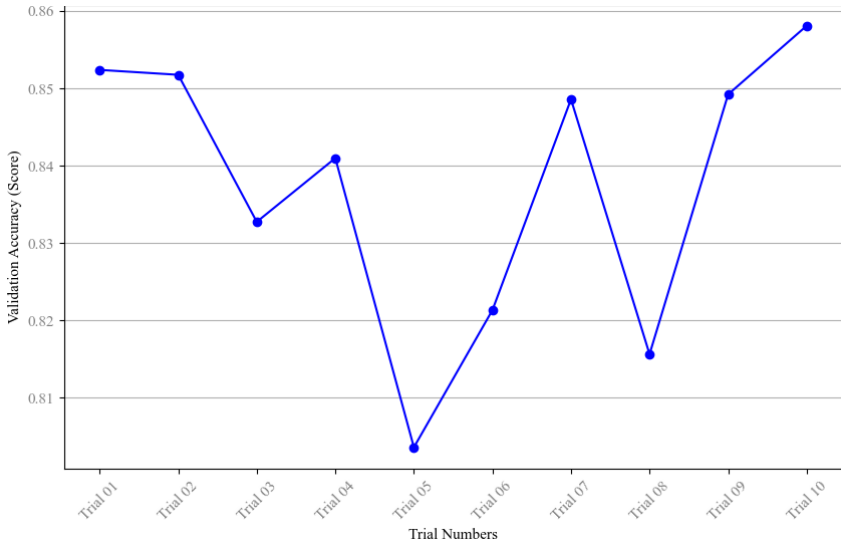


Figure 5.14. Word2Vec+LSTM AST Code Input Hyper Parameter Tuning Result.

For the Word2Vec + LSTM model with input AST, we show the validation accuracy for 10 different trials of the hyperparameter tuning graph. The accuracy range varies from 81% to 85% based on different hyperparameter configurations. The highest validation accuracy is observed in Trials 1, 7, and 10, while the lowest occurs in Trials 5 and 8. This underscores the importance of exploring various hyperparameter settings to optimize model performance and avoid configurations that may lead to lower accuracy or overfitting.

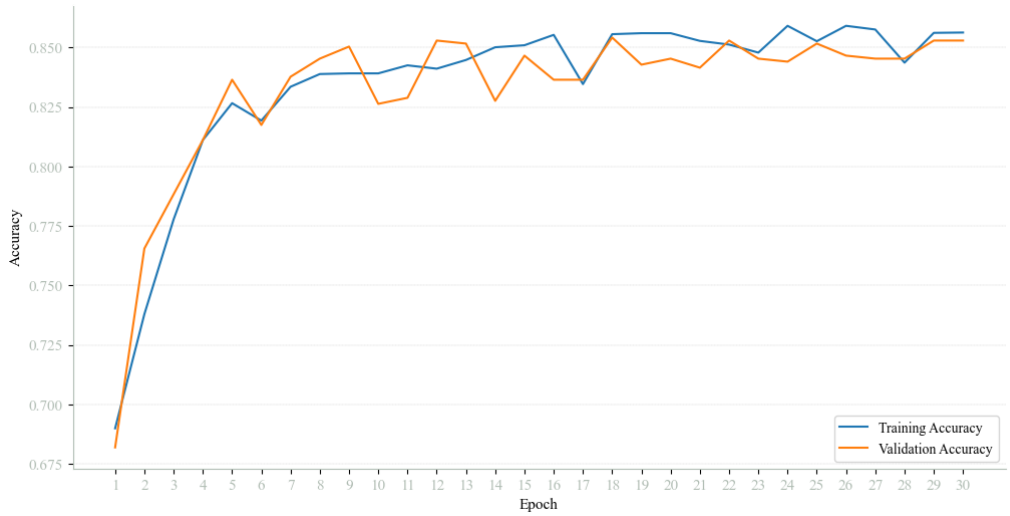


Figure 5.15. Word2Vec+LSTM AST Code Input of Training and Validation Accuracy *Result.*

The training and validation accuracy curves are plotted for the Word2Vec + LSTM model trained using AST inputs across 30 epochs, as shown in the figure. We see that both curves have consistency in that accuracy keeps rising every epoch, which means the model made sense of the training data. Then, training accuracy increases sharply at first, breaking over 80% accuracy within just a few epochs. A similar curve is observed for validation accuracy, except for minor fluctuations possibly attributed to variations in the validation dataset. Both curves oscillate as the number of epochs increases. In neural networks, this is typical, where a model attempts to generalize from the training data by overfitting or underfitting on some batch. Despite these oscillations, the overall trend of validation and training accuracy for both training and validation is upward. With the later epochs, the training accuracy stabilizes around 85%. We find that the validation accuracy closely maintains the training accuracy indicating that the model does generalize well and does not appear to be overfit. We can see that the two curves are close to each other, indicating that the model's prediction on data that hasn't been seen is almost identical to the model's prediction on the training set.

Overall, the model reaches a satisfactory accuracy level, maintaining approximately 85% accuracy for training and validation, with minor fluctuations, which is typical for LSTM models when handling complex data like AST representations of code.

CHAPTER – 6

Comparative Analysis

6.1. Overview

In this chapter, we comprehensively compare the models we used in vulnerability detection in source code. Two open-source LLMs with the proprietary LLM and a neural network implemented with two input formats, plain code and AST code, are included in the analysis.

Specifically, the literature review of previous papers, such as [91] [101], shows that proprietary models like GPT-4 Turbo have scalability, ease of use and integration capabilities. Nevertheless, they lack significant flexibility regarding fine-tuning, they do not exhibit transparency in their architecture or the decision-making process, and face issues like false positives and performance problems in more complex scenarios [164].

On the other side, open-source models including CodeGen2 and LLaMA2 have more adaptability and transparency to allow domain-specific fine-tuning and optimization [105] [106]. However, these models need massive computational resources and expertise, but they have the potential for customization tasks or underrepresented domains as mentioned in [105].

This chapter attempts to clarify the strengths and weaknesses of every single method by comparing the applicability of the models and input formats to various use cases of vulnerability detection.

6.2. Opensource LLMS

This bar chart compares the performance of two models, LLaMA2 and CodeGen2, across five key evaluation metrics Precision, Recall, F1 Score, Accuracy and AUC. Each metric gives us a reliable sense of how the models perform on a particular task.

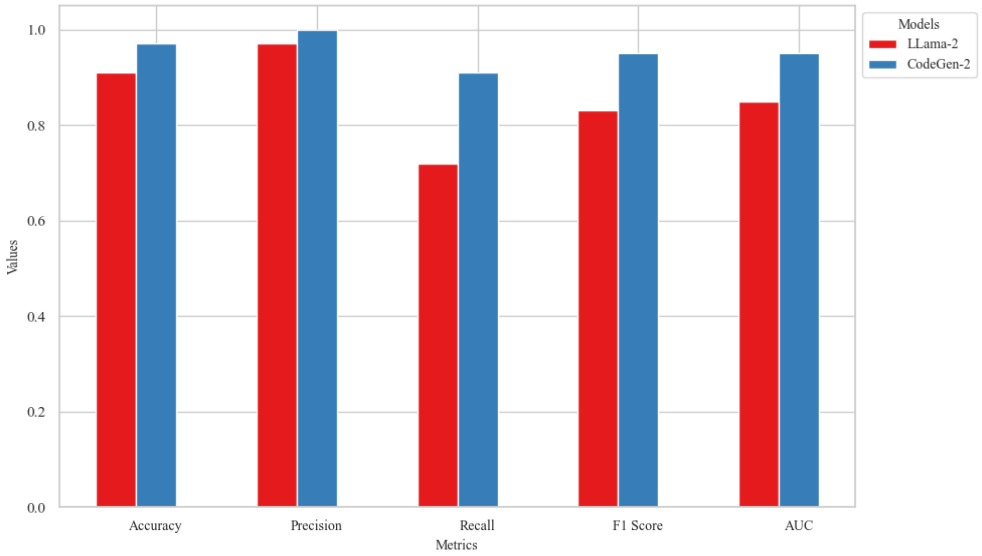


Figure 6.1. LLaMA2 and CodeGen2 Models Performance.

Accuracy: The accuracy of CodeGen-2 slightly exceeds that of LLaMA2 in accuracy, so CodeGen2 has a higher percentage of correct predictions overall.

Precision: The precision achieved by CodeGen-2 is perfect, made by not arising any false positives. LLaMA2 performs well, but it is still less than CodeGen2.

Recall: CodeGen2 leads again in terms of recall, suggesting that it captured more true positive so-called relevant examples. We find that LLaMA2's recall is significantly lower than CodeGen2's.

F1 Score: The F1 score balances precision and recall, and CodeGen2 has a slight edge over LLaMA2, though both models demonstrate strong performance.

AUC: Both models show strong discriminatory power between classes, but CodeGen2 slightly outperforms LLaMA2 in this metric as well.

Overall, the comparison highlights CodeGen2 as the stronger model, consistently achieving higher values across most metrics, particularly in Precision, Recall, and AUC, while LLaMA2 still performs competently. This suggests CodeGen-2 may be better suited for this task, especially in reducing false positives and identifying relevant cases.

6.3. Proprietary LLMS

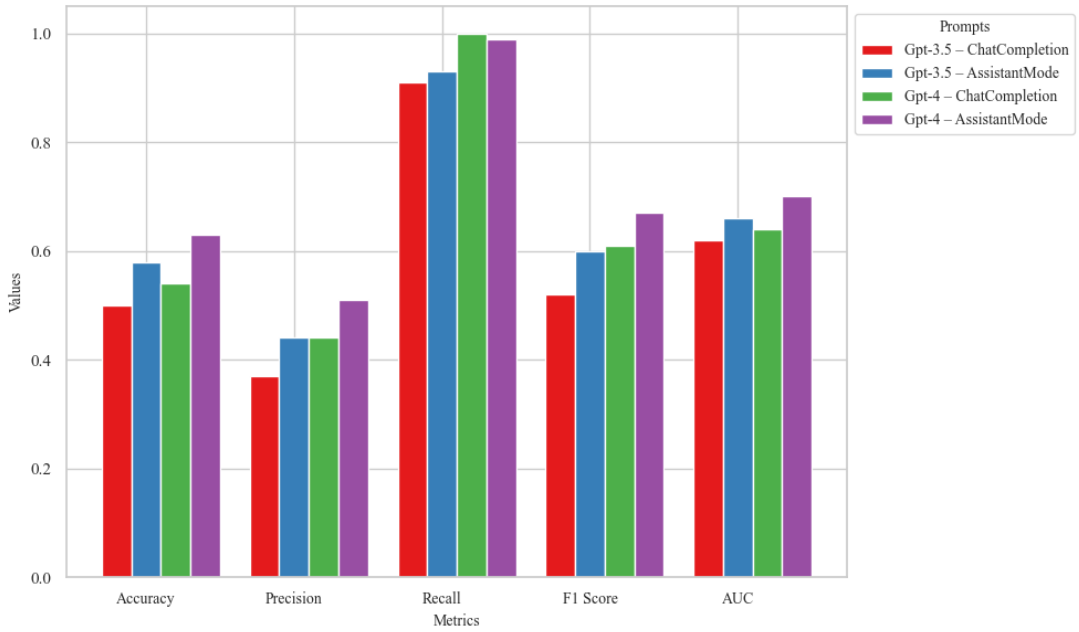


Figure 6.2. GPT Models Best Performance Prompts Result.

From the above analysis, the GPT-4 heavily dominates all key performance metrics. In most cases, the GPT-4 versions, particularly in the assistant mode exceed all GPT-3.5 models. This suggests that GPT-4 benefits from advancements in architecture and training, making it a more capable model for general and task-specific challenges, such as code vulnerability detection or other real-world applications. GPT-4's design and functionality have improved to reach higher precision, recall and overall balanced performance than GPT-3.5.

One of the observations in the presented results is a correlation between precision and recall values. GPT-3.5 is better at recall, identifying truer positives, but at the same time, it is worse at precision, which generates false positives. In contrast, GPT-4 models have a better trade-off between the two metrics, high precision, and low recall without being too low on recall. This balance thus makes GPT-4 a more credible model when false indicators and failure to detect cases are undesirable, especially in real-world applications such as code vulnerability analysis.

In addition, the comparison of the Assistant mode and the Chat Completion models shows that the Assistant mode is better in both GPT-4 and GPT-3.5 models. The Assistant mode appears to be designed for tasks requiring precision and decision-making and, hence, better suited for practical, supportive applications.

In conclusion, GPT-4 models perform across all measured metrics, particularly in the Assistant mode among other GPT models. Therefore, they are better at jobs that need to be precise, such as code vulnerability detection, where we want to minimize the number of false positives but

still have enough recall to find as many vulnerabilities as possible. GPT-4 and its training method have achieved architectural and training method improvements that elevate its capabilities far above GPT-3.5. These findings' practical implications underscore those systems relying on high precision and AUC, such as security applications, would benefit most from deploying GPT-4 models. Their ability to minimize false positives while maintaining high detection rates makes them ideal for environments where the correctness of results and comprehensive detection are critical factors. Thus, this analysis supports the conclusion that GPT-4 Assistant mode is superior to other GPT models, particularly in contexts where reliability and precision cannot be compromised.

6.4. Neural Network

The results of this performance comparison indicate that while Plain Code input tends to deliver better results on almost all metrics, including precision, accuracy, and F1 score, this is a more reliable method of avoiding false positives and achieving overall balanced performance. However, the Plain Code input excels in recall, which implies that it is more effective at identifying actual vulnerabilities, although at the cost of precision. This trade-off may be acceptable in scenarios where identifying all potential vulnerabilities is critical, even if it increases false positives.

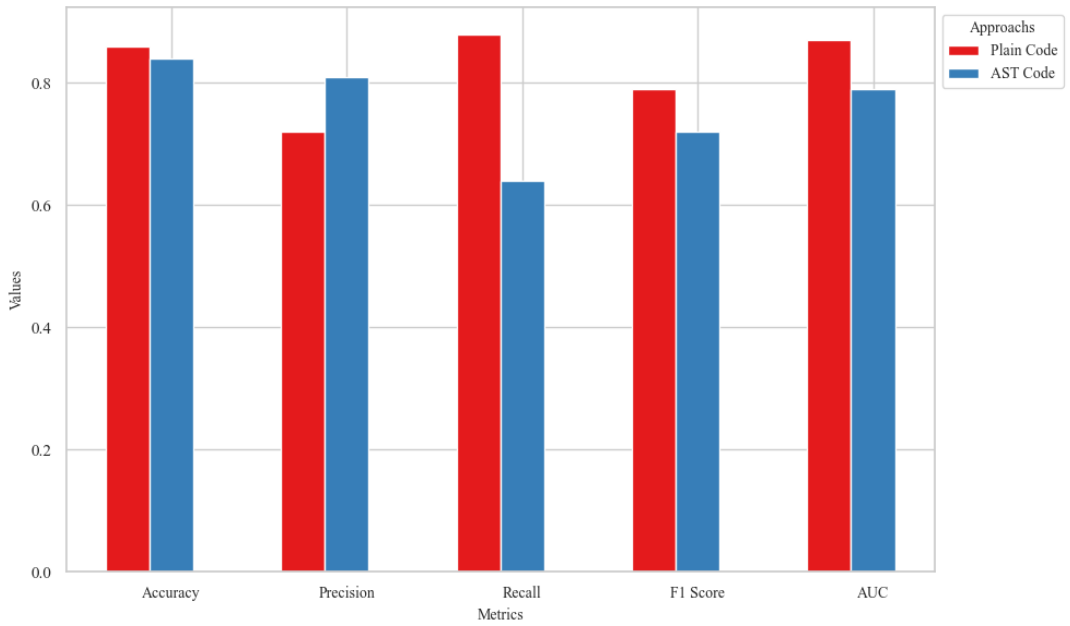


Figure 6.3. Neural Network Two Types of Input Performance Result.

In general, the results imply that the Plain Code versus AST Code choice depends on the requirements. If we want to minimize false positives, Plain Code would be preferred. The more exhaustive recall of the AST Code approach may be better suited than approaches that achieve high precision for a more thorough vulnerability detection process. From their AUC we can see that both the methods are suitable, but Plain Code might be more reliable across different thresholds.

6.5. Overall Comparison

In this section we will perform an extensive model comparison, ranging from qualitative to quantitative analysis of our experimental results as well as relevant research. Using this global comparison, we determine the advantages and drawbacks of the different models and propose how they can be used to detect source code vulnerabilities. Then, we evaluate the models' criteria from different key attributes like architectural differences, accuracy, training time, computational efficiency and interpretability to have a balanced understanding of their capability.

6.5.1. Qualitative Comparison

Architectural Differences: Each model in our comparison has a diverse architecture that influences how it processes data and performs specific tasks, particularly in source code vulnerability detection. LLMs such as LLaMA2, CodeGen2, and proprietary models like GPT-4 employ transformer architectures designed to handle large amounts of sequential data, capturing long-range dependencies within text and code. This capability enables these models to recognize complex patterns and relationships across different functions or modules in the code, making them highly effective for tasks requiring a deep understanding of code structure.

For instance, LLaMA2 model and CodeGen2 are the open-source models that can work effectively well in vulnerability detection tasks. In our study, we demonstrate that fine-tuned open-source models can uncover vulnerabilities that the simple models might have missed. We demonstrated that these models capture the complex pattern in the code more effectively. Using fine-tuned models of transformer-based architectures like LLaMA2 and CodeGen2 was shown to be better at identifying small or complex code vulnerabilities over Word2Vec + LSTM based models. This performance gap is demonstrated as a proof of its effectiveness in leveraging LLMs models in code vulnerability detection tasks. However, these advanced models come with huge computational costs and the running of these models needs more resource and hardware consumption.

Scalability and Adaptability: In evaluating the different codes vulnerability detection models, scalability and adaptability become important factors in determining how effective and practical they will be in practical, real-life applications. Because of the transformer-based architectures they use on LLMs, LLaMA2 and CodeGen2 scales are extremely good. They use large datasets

to train and can be fine-tuned to various other tasks, e.g., code vulnerability detection or code analysis. Not only do they have the scalability to handle more significant numbers of data, but they also can generalize across different types of code and coding standards.

In recent years, the usability and adaptability of various models for detecting vulnerabilities have been rigorously discussed, especially in programming languages. Wang et al. [160] proved that transformer-based models are adaptable to multiple programming languages with minimal retraining by introducing CodeT5, a pre-trained model that supports various coding tasks. They showed that CodeT5 can handle code-to-text generation, vulnerability detection, and adapt to language-specific standards, making it highly effective across diverse codebases and coding conventions.

According to Salesforce Research CodeGen2 Lessons for Training LLMs on Programming and Natural Languages [135] the paper demonstrates the capability of transformer models of the likes of CodeGen2 are scalable, they have been fine-tuned over large datasets and obtained high enough accuracy. Evaluation of CodeGen2 to detect vulnerabilities showed that CodeGen2 performs well providing high results for detecting large amounts of vulnerabilities with sequential data. These models were not only able to generalize on many different vulnerabilities, but they also proved practical for enterprise-scale use by being fine-tuned.

The comparative analysis highlights that LLMs like LLaMA2 and CodeGen2 offer superior scalability and adaptability, making them suitable for dynamic environments where security requirements and codebases constantly evolve. However, in the face of opposition, in situations where interpretability and computational efficiency are placed in a higher priority, traditional methods are helpful especially in smaller or less complex datasets. The trade-off between scalability, adaptability and interpretability means that the choice of model needs to be aligned with specific project requirements and resource constraints.

In contrast, while more interpretable and resource-efficient, traditional models like Word2Vec combined with LSTM exhibit limitations in scaling to handle complex tasks. Word2Vec captures word-level embeddings efficiently, and LSTM models are adept at sequential processing. However, their scalability is an obstacle when dealing with long-range dependencies or large datasets, making them less adaptable to increasingly complex codebases and evolving security threats.

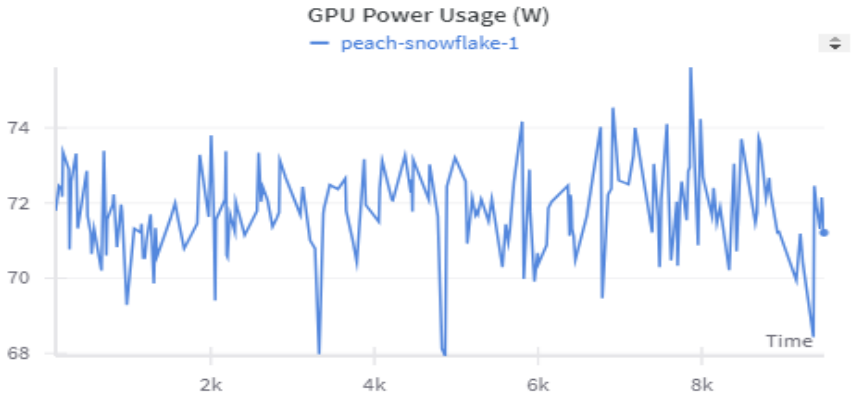
Complexity Handling: When examining how LLMs handle complexity, our research showed differences between open-source models like CodeGen2 and LLaMA2 compared to simpler models such as Word2Vec + LSTM. Opensource LLMs proved their ability to understand and control complex dependencies in code. For example, while evaluating tasks that have nested logic or contextual workflows for dealing with context-sensitive code structures, CodeGen2 scored higher in correct vulnerability identification. This can be attributed to transformer-based LLMs' capacity to model long-range dependencies and understand code patterns through self-attention mechanisms.

Moreover, LLaMA2's pre-training on extensive code datasets enabled it to learn a broad range of patterns, helping it distinguish between similar looking yet functionally different code blocks. It excelled in recognizing context differences that lead to potential security vulnerabilities.

On the other hand, simpler models like Word2Vec + LSTM struggled with these tasks due to their linear architecture and limited context comprehension. In cases involving lengthy code sequences or where code elements needed context from separate lines or files, the Word2Vec + LSTM approach showed lower recall rates. This difference was particularly evident in our tests involving multi-file codebases or projects with complex object hierarchies.

Our results support studies such as the paper by Chen et al. [137], which demonstrates that LLMs trained on extensive code datasets perform well in generating code and solving coding problems. Specifically, these models excel in handling tasks like code completion and generation based on various programming contexts. Thus, the complex handling capabilities of LLMs make them required for projects involving sophisticated and interconnected code logic. However, they also introduce challenges like increased computational cost and longer fine-tuning times compared to simpler models.

Resource Efficiency: We monitored hardware utilisation during the open-source LLMs fine-tuning and the Word2Vec+LSTM model training to evaluate resource efficiency. In this process, GPU and CPU usage, power consumption, and memory utilisation were tracked in terms of different stages of training and validation.



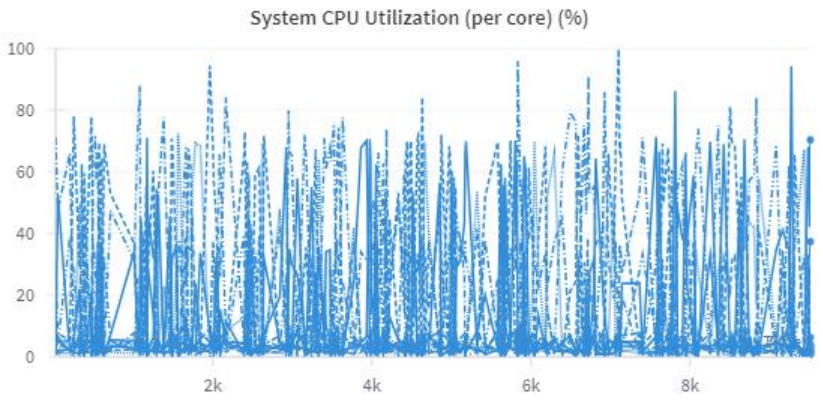
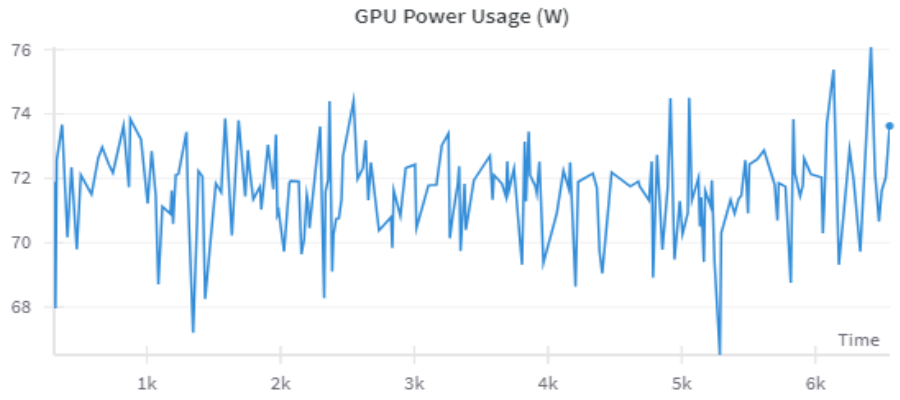


Figure 6.4. LLam2 GPU Power Usage & CPU Usage.



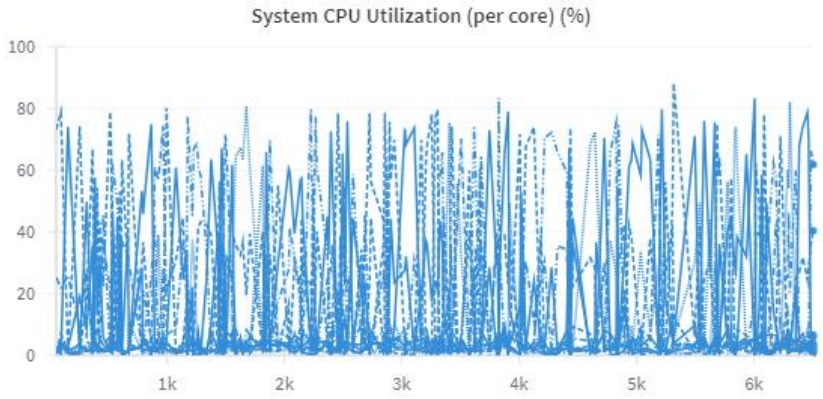


Figure 6.5. CodeGen2 GPU Power Usage & CPU Usage.

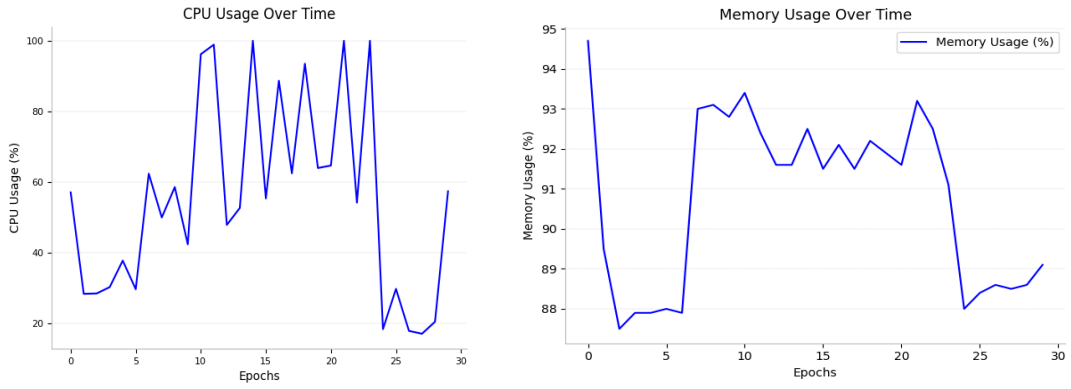


Figure 6.6. Neural Network CPU Usage & Memory Usage.

Comparing the hardware utilization of different models during the fine-tuning and testing phases indicates several essential resources and environmental efficiency points.

Fine-tuning for code vulnerability tasks with CodeGen2-2b took about 1.5 hours for relatively optimized training time. However, testing took significantly longer, around 5 hours, indicating that the code-based dataset or architecture may involve intensive operations during the evaluation phases.

The LLaMA2-7b model needed 2.5 hours for training and 3 hours for testing. LLaMA2's architecture design was targeted to make efficient testing a feature of its design and may reflect its design in optimizing multi-token handling and better generalization.

Word2Vec+LSTM fine-tuning on a local PC (Intel i5-4200U with 16GB RAM) completed 30 epochs, leveraging the CPU-only setup. While not as scalable as the other large models, the simplicity and resource efficiency of this approach make it feasible for moderate-scale tasks, especially in resource-constrained environments.

From a resource efficiency perspective, CodeGen2's consistent GPU workload as seen in the power consumption graphs indicates that it effectively utilizes GPU resources during training. But longer testing times indicate potential bottlenecks or inefficiencies in inference of large code datasets, like parsing or logical consistency checking. On the other hand, LLaMA2 is balanced in GPU and CPU tasks loading both at training and testing time, indicating that its design has a good spread of the computational work. Although CPU-dependent, Word2Vec+LSTM is a practical option for a smaller-scale model that can run locally without specialized hardware. It also demonstrates energy efficiency in executing common CPU architectures when deploying large-scale models may not be possible.

A cost-efficiency comparison is made between ChatGPT-3.5 and GPT-4. Calls to the proprietary LLMs' APIs, which provide extensive general-purpose language capability, come at a high

economic and environmental cost of cloud-based resource consumption. Running frequent API calls burns through many data centers, consuming more power and causing higher carbon footprints. Further, due to being trained on vast amounts of data, such as GPT-4, proprietary models will require large amounts of energy-intensive GPU clusters to train, which further harms the environment.

Comparing model architectures requires consideration of their carbon footprint. Fine-tuning the LLaMA2 and CodeGen2 models on cloud GPU-based instances contributes a larger amount of carbon emission compared to local CPU-based approaches such as Word2Vec+LSTM. Proprietary LLMs hosted in data centers contain a higher carbon footprint since they require much-sustained cooling and power supply.

Less environmental impact can be achieved by choosing to run on energy-efficient models or by scaling model sizes down to smaller parameter sizes according to application. Carbon emission can be significantly reduced if the models are locally executable and designed for low-power GPU when the training and test occur frequently.

Overall, CodeGen2 efficiently performs code-specific tasks and runs on an optimized GPU workload, though testing time is extended. The larger-sized LLaMA2 not only distributes the computational resources better but also speeds up testing efficiency. On the other hand, Word2Vec+LSTM is a lightweight CPU-centric yet modest hardware solution. However, costs and environmental concerns have increased, and ChatGPT-3.5 and GPT-4 offer good performance. Finally, this comparative analysis demonstrates that resource efficiency is related to task specialization, the model architecture and the hardware used, and the tradeoff between model size and the carbon footprint.

6.5.2. Quantitative Comparison

Below, Table 6.1 shows all model's performance metrics.

Table 6.1. Performance Metrics of All Models

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**AUC**
Lama2 -7B	91%	97%	72%	83%	85%
CodeGen2	97%	100%	91%	95%	95%
GPT 4-Assistent	63%	51%	99%	67%	70%
Word2Vec+LSTM Plane code	86%	72%	88%	79%	87%
Word2Vec+LSTM AST	85%	84%	64%	72%	79%

Performance Analysis: In comparing the performance of LLaMA2 -7b and CodeGen2-2b, each model shows strengths that cater to different aspects of vulnerability detection. Pulling out the analyses of all the two models, LLaMA2 shows a more balanced approach. With an accuracy of 91%, it maintains a high level of correctness in its predictions. Its precision score of 97% suggests it is effective at making correct positive predictions, minimizing false positives.

However, its recall is notably lower at 72%, which indicates that it misses several vulnerabilities. This lower recall drags down its overall F1 score to 83% and AUC to 85%, reflecting a limitation in the model's sensitivity.

CodeGen2, on the other hand, exhibits exceptional performance across all metrics. With an accuracy of 97% and a perfect precision of 100%, it achieved highly accurate positive predictions with zero false positives. The recall of 91% further reinforces its reliability in identifying vulnerabilities comprehensively, leading to a high F1 score of 95%. The AUC of 95% shows that CodeGen2 effectively balances both precision and recall, representing an overall well-rounded performance in identifying vulnerabilities accurately and consistently. This comparison suggests that while LLaMA2 provides balanced results, CodeGen2's superior recall and precision make it more suitable for applications where high accuracy and thorough vulnerability detection are essential.

In contrast, GPT-4 Assistant model was highly variable. Yet its recall of 99% and thus its ability to identify almost all the existing vulnerabilities was the highest among all other models. However, the model had a low precision 51%, and F1 Score 67%. This implies that although the model located most vulnerabilities, it created many false positives. The AUC of 70% suggests poor performance in being able to distinguish between negative and positive cases.

Two variations of the Word2Vec+LSTM models were analyzed, one trained over plain code and another trained over ASTs. The Word2Vec+LSTM plain code performance is reasonable accuracy 86%. Its lower precision of 72%, though, means that it had more false positives than some of the other models. The recall 88% was strong, the F1 Score 79% and AUC 87% reflect its capability to provide moderate stability between precision and recall.

The accuracy achieved by the Word2Vec+LSTM AST input model was lower relative to the plain code version, 85%. Interestingly, it had lower precision 84% but recalled 64%, a significantly lower percentage of vulnerabilities indicating that, although it can achieve good precision, it did not detect some of the vulnerabilities. Consequently, its AUC 79% and F1 Score 72% were drastically reduced in balanced detection.

The results of CodeGen2 are the most efficient in overall terms and its precision is the highest (100 %) with a recall of 91 %, showing that the tool correctly points out the vulnerabilities with the least probability of false alarms. This is especially important for real life problems where we need a high level of detail and accuracy. LLaMA2 is relatively adequate in the balance between precision (97%) and recall (72%) and, thus, can be considered to have some level of imperfection in vulnerability identification. The recall score of GPT-4 Assistant is 99%, meaning that it identifies most of the vulnerabilities, but its precision is low at 51% resulting in many false alarms. The Word2Vec+LSTM models perform reasonably well with the plain code model having a better recall of 88% as compared to the AST model with 64% recall; both models fail to deliver the discriminating power of the advanced LLMs. These metrics also reveal that

CodeGen2 is the most efficient in terms of achieving high levels of accuracy and completeness of the detection discussed in the thesis.

Comparative Insights: From this quantitative comparison, it is clear that CodeGen2 emerges as the most reliable model, with near-perfect precision and strong recall. These metrics show a good balance which leads to superior F1 and AUC scores, making this a good choice for vulnerability detection tasks. LLaMA2-7b also provides a reasonable performance with higher precision at the less performance of the recall. The GPT-4 Assistant is an impressive model when it comes to recall, but it fails due to the high false positive rate making it ineffective in most codes. Finally, it shows that classical neural networks such as Word2Vec+LSTM detect vulnerabilities reasonably well, but suffer in precision and recall, indicating that classical neural networks still have limitations in comparison to more modern opensource LLMs. The modern LLMs such as CodeGen2 and LLaMA2 have better generalization and pattern detection compared to classical neural networks as Word2Vec + LSTM, because modern LLMs rely on the large scale datasets consisting of billions of tokens and no matter the approach used, the transformer based architecture in LLMs uses attention mechanisms to learn both local and global relationships in the code at much better efficiency compared with LSTMs which struggled with long term dependencies as well as scalability. Also, LLMs process this raw input without manual preprocessing, drop the information loss, and are fine tunable on domain specific tasks, and therefore we have higher precision and recall.

In conclusion, the quantitative analysis shows the advantages and disadvantages of each model. The results show that although all models can be used in a weight dependent manner, CodeGen2 has the best precision recall balance which makes it the most resource efficient and most accurate model for vulnerability detection in this comparison.

CHAPTER – 7

Conclusion and Future Work

7.1.Conclusion

The threat of cyber-attacks has risen in an era where banks, e-commerce, logistics, transportation, energy, and healthcare industries heavily depend on computer systems and networks. The online ecosystem's rapid growth and the rise in the availability of sensitive data allow organized cyber criminals to exploit software system vulnerabilities. Despite the availability of the technology, it has proved a tricky and urgent challenge to safeguard software.

This thesis has shown that security measures can be integrated into the SDLC to develop secure applications. SCA is a core practice focused on looking at source code for vulnerability during the development process. It supports this proactive approach, by helping to identify and remove security flaws well before deployment, reducing the potential for exploitation in the deployment environment.

In this work, the research supports the need for improved performance of SCA tools and increases detection accuracy and efficiency. These findings can help to enhance developers' ability to create more comprehensive defects and more compelling and reliable software systems. The aim of this thesis is to utilize secure software development to obtain some insights from the efficient methods of exposing source code vulnerability to contribute to the software development field.

In particular, the effectiveness of the various approaches to source code vulnerability identification is evaluated in this research, emphasizing comparing the performances of fine-tuned open-source LLMs, proprietary LLMs, and neural network architectures. They were analyzed in detail with respect to their strengths, weaknesses and applicability to different scenarios of vulnerability detection.

The fine-tuned open source LLMs provide a good opportunity to detect vulnerability as they can be adapted and capable of aligning with a specific dataset and requirement. These models represent a cost-effective, customizable option for organizations who are looking for advanced solutions, tailored specifically to their local development environment. Like many GPT family models, proprietary LLMs achieved outstanding generalization capabilities and high performance from extensive pretraining on large-scale datasets. However, the generalization of its performance for vulnerability detection is low compared to other models.

When considering neural networks, the Word2Vec+LSTM neural network approach gave us an alternate perspective by applying sequence modelling to efficiently analyze source code. This

method is highly interpretable and ideal for finding sequential patterns in code. However, its vulnerability detection performance may not stand against that of open-source LLMs in scale and context. This alternative can be served when a simpler, resource-efficient solution is needed.

Through this comparative analysis, the thesis has reinforced the need to choose the right tool given the organization's needs, resource constraints and use cases. This research provides insights that help develop more secure software by integrating ML deeper into the software development lifecycle. This work systematically evaluates these approaches to inspire more progress in vulnerability detection and, in doing so, mitigate risk in a world where digitalization continues.

7.2. Real-World Integration

The findings of this thesis can be practically applied to a real-world case scenario like CI/CD pipelines. The vulnerability detection tools evaluated in this research can be embedded in the CI/CD workflow to automate the vulnerability identification and resolution process as part of the development. Fine-tuned open-source LLMs could be deployed in pre-commit hooks or in build stages of code repositories to early detect vulnerabilities before they reach production environments. Adaptability of these models ensures that they are in line with the organizational domain specific coding standards and practices. Domain specific models could be integrated as a supplementary layer to CI/CD pipelines to obtain broader patterns of vulnerabilities, which are typically missed by the domain specific models, with top performing proprietary LLMs integrated as the supplementary layer.

The trade-off between true positives and false positives in vulnerability detection has significant implications for real-world applications, particularly in DevSecOps, critical infrastructure, financial technology (fintech), and healthcare security. For instance, the CodeGen2 with its 100% precision and 91% recall, is highly effective in security-sensitive environments like financial institutions and government agencies, where minimizing false positives is crucial to avoid unnecessary investigations that waste time and resources. In contrast, GPT-4 Assistant, with its 99% recall but only 51% precision, is suitable for industries where missing vulnerability is far more damaging than dealing with false alarms such as in cloud security or cyber threat intelligence, where comprehensive detection is necessary to prevent zero-day attacks.

In software development, these modules can be incorporated in a CI/CD pipeline for automating the process of security scanning. During the pre-commit phase where a high recall model must flag as many possible vulnerabilities as possible, a lot of security issues can be ensured to pass their gates. This can then be followed by applying CodeGen2, a high-precision model, at the build stage for validating false positives and minimizing deployment verifications. This way, disruptions to developers are minimal while ensuring that productivity is at optimum.

However, for resource constrained environments like smaller scale projects or smaller scale environments where we can deploy and can use lightweight neural network model like Word2Vec + LSTM for fast vulnerability scans. Examples in industries such as fintech where a secure payment gateway is deployed using a CI/CD pipeline, while in the healthcare industry where there is a requirement to ensure the data is secure is highly sensitive patient data. These integrations not only lead to more secure software, but also to lower cost and time spent when fixing software issues post deployment what modern DevSecOps is about.

7.3. Future Work

In future work, conducting an in-depth research study to analyze the impact of the chain-of-thought prompting technique on the effectiveness of LLM models in vulnerability detection would be highly beneficial. This approach, shown in prior studies to enhance the reasoning capabilities of LLMs, remains unexplored in this paper specifically for source code vulnerability identification. Integrating chain-of-thought samples into the prompting mechanism during training could potentially improve model accuracy by guiding the model through step-by-step processes. This enhancement could uncover additional insights and improve the overall capability of these models in identifying vulnerabilities, providing more comprehensive detection outcomes.

Another promising avenue for future research is to explore the capability of fine-tuning new proprietary models. We could evaluate how proprietary systems behave when tailored for vulnerability detection by fine-tuning such models. It can tell us how adaptable, efficient and performant they are compared to open-source solutions and even potentially show how far these models can stretch to solving complex and domain-specific security challenges. The models could be better at addressing sophisticated tasks and developing software vulnerability detection by tracking these areas.

Bibliography

[1] N. Davis, “STN 8-2: Software Cost, Quality and Productivity Benchmarks 2 Data & Analysis Center for Software (DACS) Developing Secure Software,” *Data & Analysis Center for Software (*DACS), Accessed: July 9, 2024. [Online] Available: https://www.sis.pitt.edu/jjoshi/ devsec/secure software.pdf . [Accessed: Nov. 05, 2024].

[2] I. Gomes, P. Morgado, T. Gomes, and R. Moreira, “An overview on the static code analysis approach in software development”, *Faculdade de Engenharia da Universidade do Porto, Portugal*, vol. 16, 2009.

[3] Y. Pang, X. Xue, and A. S. Namin, ‘Predicting vulnerable software components through n-gram analysis and statistical feature selection’, in *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, 2015, pp. 543–548.

[4] D. Binkley, ‘Source code analysis: A road map’, *Future of Software Engineering (FOSE’07)*, pp. 104–119, 2007.

[5] L. Harrison, “A decade of global cyberattacks: Where they left us,” *Security Intelligence*, Accessed: July 9, 2024. [Online]. Available: https://securityintelligence.com/articles/decade-global-cyberattacks-where-they-left-us/.. [Accessed: Nov. 23, 2024].

[6] F5 Networks, “Memory Safety: CISA and NSA Guidance on CWE-1399,” *F5 Community*, Accessed: Jan. 05, 2024. [Online]. Available: https://community.f5.com/kb/technicalarticles/ memory-safety-cisa-and-nsa-guidance-on-cwe-1399/325590. [Accessed: Nov. 14, 2024].

[7] A. Petrosyan, “,Number of common IT security vulnerabilities and exposures (CVEs) worldwide from 2009.” Accessed: July 9, 2024. [Online]. Available:https://www.statista.com/markets/ 424/topic/ 1065/cyber-crime-security/#overview. [Accessed: Nov. 05, 2024].

[8] Qualys, “An in-depth look at the latest vulnerability threat landscape – Part 1,” *Qualys Blog*, Accessed: Jul. 11, 2024. [Online]. Available: https://blog.qualys.com/product-tech/2023/07/11/an-in-depth-look-at-the-latest-vulnerability-threat-landscape-part-1.[Accessed: Nov. 23, 2024].

[9] Rapid7, “ETR: High-risk vulnerabilities in common enterprise technologies,” *Rapid7 Blog*, Accessed:Sep. 19, 2024. [Online]. Available: https://www.rapid7.com/blog/post/2024/09/19/etr-high-risk-vulnerabilities-in-common-enterprise-technologies/.[Accessed: Nov. 05, 2024].

[10] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: an overview," *arXiv preprint arXiv:2008.05756*, Aug. 2020.[Online]. Available: https://arxiv.org/abs/2008.05756

[11] H. Assal and S. Chiasson, “‘Think secure from the beginning’: A survey with software developers,” in *Conference on Human Factors in Computing Systems - Proceedings*, 2019, doi: 10.1145/3290605.3300519.

[12] H. El-Hadary and S. El-Kassas, “Capturing security requirements for software systems,” *Journal of Advanced Research*, vol. 5, no. 4, pp. 463–472, 2014, doi: 10.1016/j.jare.2014.03.001

[13] M. Ozer, Y. Kose, M. Bastug, G. Kucukkaya, and E. R. Varlioglu, “The Shifting Landscape of Cybersecurity: The Impact of Remote Work and COVID-19 on Data Breach Trends,” Apr. 06, 2024, *arXiv*: arXiv:2402.06650. Accessed: Nov. 12, 2024. [Online]. Available: http://arxiv.org/abs/2402.06650

[14] Deloitte, “State of Gen AI: Q3 2023,” *Deloitte Insights*, 2023. [Online]. Available: https://www2.deloitte.com/content/dam/Deloitte/us/Documents/consulting/us-state-of-gen-ai-q3.pdf. [Accessed: Nov. 05, 2024].

[15] I. Ozkaya, “Application of Large Language Models to Software Engineering Tasks: Opportunities, Risks, and Implications,” *IEEE Softw.*, vol. 40, no. 3, pp. 4–8, May 2023, doi: 10.1109/MS.2023.3248401.

[16] I. H. Sarker, Y. B. Abushark, F. Alsolami, and A. I. Khan, “IntruDTree: A Machine Learning Based Cyber Security Intrusion Detection Model,” *Symmetry*, vol. 12, no. 5, p. 754, May 2020, doi: 10.3390/sym12050754.

[17] X. Zhou, T. Zhang, and D. Lo, “Large Language Model for Vulnerability Detection: Emerging Results and Future Directions,” in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, Lisbon Portugal: ACM, Apr. 2024, pp. 47–51. doi: 10.1145/3639476.3639762.

[18] E. K. Williams Sr, “Examining a Quantitative Correlational Study in Filling the Cybersecurity Professional Workforce Shortage Using Digital Transformation,” Northcentral University, 2023.

[19] L. Ruggahakotuwa, L. Rupasinghe, and P. Abeygunawardhana, “Code vulnerability identification and code improvement using advanced machine learning,” in *2019 International Conference on Advancements in Computing (ICAC)*, 2019, pp. 186-191, IEEE.

[20] Reid, D., Eng, K., Bogart, C., & Tutko, A. (2021, May). Tracing vulnerable code lineage. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)* (pp. 621-623). IEEE.

[21] “5 8 IEEE Software Publs,” [Online]. Available: http://findbugs.sourceforge .net.[Accessed: Nov. 05, 2024].

[22] D. Evans and D. Larochelle, "Splint," *Software Engineering Report*, Oct. 2001.[Online]. Available: https://www.splint.org/ [Accessed: Nov. 05, 2024].

[23] I. Abunadi and M. Alenezi, “Towards cross-project vulnerability prediction in open source web applications,” in *Proceedings of the International Conference on Engineering & MIS 2015*, Istanbul, Turkey, 2015, pp. 2832987–2833051. [Online]. Available: https://doi.org/10.1145/2832987.2833051.

[24] H. A. Alatwi, T. Oh, E. Fokoue, and B. Stackpole, "Android malware detection using category-based machine learning classifiers," in *Proceedings of the 17th Annual Conference on Information Technology Education*, Boston, Massachusetts, USA, 2016, pp. 54–59, doi: 10.1145/2978192.2978218.

[25] Y. Chen, A. E. Santosa, A. M. Yi, A. Sharma, A. Sharma, and D. Lo, "A machine learning approach for vulnerability curation," *Association for Computing Machinery*, New York, NY, USA, 2020, doi: https://doi.org/10.1145/3379597.3387461

[26] X. Du, B. Chen, Y. Li, J. Guo, Y. Zhou, Y. Liu, and Y. Jiang, "LEOPARD: Identifying vulnerable code for vulnerability assessment through program metrics," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 60–71. https://doi.org/10.1109/ICSE.2019.00024.

[27] Y. Du, X. Wang, and J. Wang, "A static Android malicious code detection method based on multi-source fusion," *Security and Communication Networks*, vol. 8, no. 17, pp. 3238–3246, 2015, doi: 10.1002/sec.1248.

[28] Y. Fang, Y. Liu, C. Huang, and L. Liu, "FastEmbed: Predicting vulnerability exploitation possibility based on ensemble machine learning algorithm," *PLoS ONE*, vol. 15, no. 2, p. 0228439, 2020. [Online]. Available: https://doi.org/10.1371/.

[29] M. Jimenez, R. Rwemalika, M. Papadakis, F. Sarro, Y. Le Traon, and M. Harman, "An empirical study on vulnerability prediction of open-source software releases," in *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.

[30] I. Medeiros, N. Neves, and M. Correia, "Detecting and removing web application vulnerabilities with static analysis and data mining," *IEEE Transactions on Reliability*, vol. 65, no. 1, pp. 54–69, 2016, doi: 10.1109/TR.2015.2477018.

[31] R. Moskovitch, N. Nissim, and Y. Elovici, "Malicious code detection using active learning", Proceedings *International Workshop Privacy, Security, Trust KDD*, 2008, pp. 74–91.

[32] A. Narayanan, M. Chandramohan, L. Chen, and Y. Liu, "A multi-view context-aware approach to Android malware detection and malicious code localization," *Empirical Software Engineering*, vol. 23, no. 3, pp. 1222–1274, 2018, doi: 10.1007/s10664-017-9539-8.

[33] S. Ndichu, S. Kim, S. Ozawa, T. Misu, and K. Makishima, "A machine learning approach to detection of JavaScript-based attacks using AST features and paragraph vectors," *Applied Soft Computing*, vol. 84, p. 105721, 2019, doi: 10.1016/j.asoc.2019.105721.

[34] B. M. Padmanabhuni and H. B. K. Tan, "Buffer overflow vulnerability prediction from x86 executables using static analysis and machine learning," in *2015 IEEE 39th Annual Computer Software and Applications Conference*, 2015, pp. 450–459, doi: 10.1109/COMPSAC.2015.78.

[35] Y. Pang, X. Xue, and A. S. Namin, “Early identification of vulnerable software components via ensemble learning,” in *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2016, pp. 476–481, doi: 10.1109/ICMLA.2016.0084.

[36] J. D. Pereira, J. R. Campos, and M. Vieira, “An exploratory study on machine learning to combine security vulnerability alerts from static analysis tools,” in *2019 9th Latin-American Symposium on Dependable Computing (LADC)*, 2019, pp. 1–10, doi: 10.1109/LADC48089.2019.8995685.

[37] H. Perl et al., “VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS ’15)*, 2015, pp. 426–437, doi: 10.1145/2810103.2813604.

[38] J. Ren, Z. Zheng, Q. Liu, Z. Wei, and H. Yan, “A buffer overflow prediction approach based on software metrics and machine learning,” *Security and Communication Networks*, p. 8391425, 2019, doi: 10.1155/2019/8391425.

[39] R. Russell et al., “Automated vulnerability detection in source code using deep representation learning,” in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 757–762, doi: 10.1109/ICMLA.2018.00120.

[40] L. K. Shar, L. C. Briand, and H. B. K. Tan, “Web application vulnerability prediction using hybrid program analysis and machine learning,” *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 6, pp. 688–707, 2015, doi: 10.1109/TDSC.2014.2373377.

[41] K. Z. Sultana, “Towards a software vulnerability prediction model using traceable code patterns and software metrics,” in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 1022–1025, doi: 10.1109/ASE.2017.8115724.

[42] K. Z. Sultana, V. Anu, and T.-Y. Chong, “Using software metrics for predicting vulnerable classes and methods in Java projects: A machine learning approach,” *Journal of Software: Evolution and Process*, vol. 33, no. 3, p. 2303, 2021, doi: 10.1002/smr.2303.

[43] B. A. Vishnu and K. P. Jevitha, “Prediction of cross-site scripting attack using machine learning algorithms,” in *Proceedings of the 2014 International Conference on Interdisciplinary Advances in Applied Computing* (Amritapuri, India) (ICONIAAC '14), 2014, pp. 1-5, doi: 10.1145/2660859.2660969.

[44] X. Wang, S. Wang, K. Sun, A. Batcheller, and S. Jajodia, “A machine learning approach to classify security patches into vulnerability types,” in *2020 IEEE Conference on Communications and Network Security* (CNS), 2020, pp. 1–9, doi: 10.1109/CNS48642.2020.9162237.

[45] L. Wei, W. Luo, J. Weng, Y. Zhong, X. Zhang, and Z. Yan, “Machine learning-based malicious application detection of Android,” *IEEE Access*, vol. 5, pp. 25591–25601, 2017, doi: 10.1109/ACCESS.2017.2771470.

[46] V. Yosifova, A. Tasheva, and R. Trifonov, “Predicting vulnerability type in Common Vulnerabilities and Exposures (CVE) database with machine learning classifiers,” in *2021 12th*

National Conference with International Participation (ELECTRONICA), 2021, pp. 1–6. [Online]. Available: https://doi.org/10.1109/

[47] A. A. Younis and Y. K. Malaiya, "Using software structure to predict vulnerability exploitation potential," in *2014 IEEE Eighth International Conference on Software Security and Reliability-Companion*, 2014, pp. 13–18. doi: 10.1109/SERE-C.2014.17.

[48] H. Alves, B. Fonseca, and N. Antunes, "Experimenting machine learning techniques to predict vulnerabilities," in *2016 Seventh Latin-American Symposium on Dependable Computing (LADC)*, 2016, pp. 151–156. doi: 10.1109/LADC.2016.32.

[49] B. Chernis and R. Verma, "Machine learning methods for software vulnerability detection," in *Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics*, Tempe, AZ, USA, 2018, pp. 31–39. doi: 10.1145/3180445.3180453.

[50] C. J. Clemente, F. Jaafar, and Y. Malik, "Is predicting software security bugs using deep learning better than the traditional machine learning algorithms?," in *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, 2018, pp. 95–102. doi: 10.1109/QRS.2018.00023.

[51] J. Cui, L. Wang, X. Zhao, and H. Zhang, "Towards predictive analysis of Android vulnerability using statistical codes and machine learning for IoT applications," *Computer Communications*, vol. 155, pp. 125–131, 2020, doi: 10.1016/j.comcom.2020.02.078.

[52] R. Ferenc, P. Hegedűs, P. Gyimesi, G. Antal, D. Bán, and T. Gyimóthy, "Challenging machine learning algorithms in predicting vulnerable JavaScript functions," in *Proceedings of the 7th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering*, Montreal, Quebec, Canada, 2019, pp. 8–14, doi: 10.1109/RAISE.2019.00010.

[53] A. Gupta, B. Suri, V. Kumar, and P. Jain, "Extracting rules for vulnerabilities detection with static metrics using machine learning," *International Journal of System Assurance Engineering and Management*, vol. 12, pp. 65–76, 2021.

[54] Y.-T. Hou, Y. Chang, T. Chen, C.-S. Laih, and C.-M. Chen, "Malicious web content detection by machine learning," *Expert Systems with Applications*, vol. 37, no. 1, pp. 55–60, 2010, doi: 10.1016/j.eswa.2009.05.023.

[55] J. Kronjee, A. Hommersom, and H. Vranken, "Discovering software vulnerabilities using data-flow analysis and machine learning," *Proceedings 13th International Conference Availability, Reliability and Security*, Hamburg, Germany, p. 10, 2018, doi: 10.1145/3230833.3230856.

[56] B. Medeiros, N. F. Neves, and M. Correia, "Automatic detection and correction of web application vulnerabilities using data mining to predict false positives," in *Proceedings 23rd International Conference World Wide Web*, Seoul, 2014, pp. 63–74, doi: 10.1145/2566486.2568024.

[57] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo, "Machine learning aided Android malware classification," *Computers & Electrical Engineering*, vol. 61, pp. 266–274, 2017.

[58] G. Piskachev, L. N. Q. Do, and E. Bodden, “Codebase-Adaptive Detection of Security-Relevant Methods,” in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, Beijing, China, 2019.

[59] A. Rahman, P. Pradhan, A. Partho, and L. Williams, “Predicting Android Application Security and Privacy Risk with Static Code Metrics,” in *Proceedings of the 4th International Conference on Mobile Software Engineering and Systems*, Buenos Aires, 2017, pp. 149–153, doi: 10.1109/MOBILESoft.2017.14.

[60] I. Santos, J. Devesa, F. Brezo, J. Nieves, and P. G. Bringas, “OPEM: A Static-Dynamic Approach for Machine-Learning-Based Malware Detection,” in *International Joint Conference CISIS’12-ICEUTÉ 12-SOCÓ 12 Special Sessions*, Á. Herrero, V. Snášel, A. Abraham, I. Zelinka, B. Baruque, H. Quintián, J. L. Calvo, J. Sedano, and E. Corchado, Eds., 2013, pp. 271–280.

[61] X. Ban, S. Liu, C. Chen, and C. Chua, “A performance evaluation of deep-learnt features for software vulnerability detection,” *Concurrency and Computation: Practice and Experience*, vol. 31, p. 19, 2019, doi: 10.1002/cpe.5103.

[62] C. B. Şahin and L. Abualigah, “A novel deep learning-based feature selection model for improving the static analysis of vulnerability detection,” *Neural Comput. Appl.*, vol. 33, no. 20, pp. 14049–14067, 2021, doi: 10.1007/s00521-021-06047-x.

[63] J. Kim, D. Hubczenko, and P. Montague, “Towards attention based vulnerability discovery using source code representation,” in *Artificial Neural Networks and Machine Learning – ICANN 2019: Text and Time Series*, I. V. Tetko, V. Kůrková, P. Karpov, and F. Theis, Eds., 2019, pp. 731–746.

[64] S. Kim, S. Hong, J. Oh, and H. Lee, “Obfuscated VBA macro detection using machine learning,” in *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2018, pp. 490–501. doi: 10.1109/DSN.2018.00057.

[65] G. Lin, W. Xiao, J. Zhang, and Y. Xiang, “Deep learning-based vulnerable function detection: A benchmark,” in *Information and Communications Security* (Lecture Notes in Computer Science), J. Zhou, X. Luo, Q. Shen, and Z. Xu, Eds., Cham: Springer International Publishing, 2020, pp. 219–232. doi: 10.1007/978-3-030-41579-2_13.

[66] G. Lin et al., “Cross-project transfer representation learning for vulnerable function discovery,” *IEEE Transactions on Industrial Informatics*, vol. 14, no. 7, pp. 3289–3297, 2018, doi: 10.1109/TII.2018.2821768.

[67] R. Mateless, D. Rejabek, O. Margalit, and R. Moskovitch, “Decompiled APK based malicious code classification,” *Future Generation Computer Systems*, vol. 110, pp. 135–147, 2020, doi: 10.1016/j.future.2020.03.052.

[68] N. Saccente, J. Dehlinger, L. Deng, S. Chakraborty, and Y. Xiong, “Project Achilles: A Prototype Tool for Static Method-Level Vulnerability Detection of Java Source Code Using a Recurrent Neural Network,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop*, 2019, pp. 114–121, doi: 10.1109/ASEW.2019.00040.

[69] L. Shiqi, T. Shengwei, Y. Long, Y. Jiong, and S. Hua, “Android malicious code classification using deep belief network,” *KSII Transactions on Internet and Information Systems*, vol. 12, pp. 454–475, 2018, doi: 10.3837/tiis.2018.01.022.

[70] M. Yang, J. Wu, S. Ji, T. Luo, and Y. Wu, “Pre-Patch: Find hidden threats in open software based on machine learning method,” in *Services – SERVICES 2018*, A. Yang and S. Siva, Eds., 2018.

[71] W. Zheng et al., “The impact factors on the performance of machine learning-based vulnerability detection: A comparative study,” *Journal of Systems and Software*, vol. 168, p. 110659, 2020, doi: 10.1016/j.jss.2020.110659.

[72] N.I.S.T., “Juliet test suite v1.3,” 2017. [Online]. Available: https://samate.nist. gov/ SRD/ testsuite. php. [Accessed: Sep. 05, 2024].

[73] Debian Project, “Debian - the universal operating system,” [Online]. Available: https://www. debian.org/. [Accessed: Nov. 05, 2024].

[74] GitHub, "GitHub: Where the world builds software," [Online]. Available: https://github.com/. [Accessed: Sep. 23, 2024].

[75] H. Hanif and S. Maffeis, “Vulberta: Simplified source code pre-training for vulnerability detection,” in *2022 International Joint Conference on Neural Networks (IJCNN)*, IEEE, Jul. 2022, pp. 1–8.

[76] L. Zhen, “Vuldeepecker: A deep learning-based system for vulnerability detection,” in *25th Annual Network and Distributed System Security Symposium (NDSS 2018)*, San Diego, CA, USA, 2018.

[77] R. Russell et al., “Automated vulnerability detection in source code using deep representation learning,” in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, Orlando, FL, USA, 2018, pp. 757–762.

[78] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet,” *IEEE Transactions on Software Engineering*, 2021.

[79] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, “muvuldeepecker: A deep learning-based system for multiclass vulnerability detection,” *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224-2236, 2021.

[80] R. Xu et al., “Detecting code vulnerabilities by learning from large-scale open source repositories,” *Journal of Information Security and Applications*, vol. 69, p. 103293, 2022.

[81] S. Lawrence, C. L. Giles, A. C. Tsoi, and A. D. Back, “Face recognition: A convolutional neural-network approach,” *IEEE Trans. Neural Netw.*, vol. 8, pp. 98–113, 1997.

[82] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, pp. 1735–80, 1997.

[83] A. Graves and J. Schmidhuber, “Framewise phoneme classification with bidirectional LSTM and other neural network architectures,” *Neural Netw.*, vol. 18, pp. 602–610, 2005.

[84] R. Dey and F. M. Salem, “Gate-variants of gated recurrent unit (GRU) neural networks,” in *Proceedings 2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*, 2017, pp. 1597–1600.

[85] J. Guo, J. Cheng, and J. Cleland-Huang, “Semantically enhanced software traceability using deep learning techniques,” in *2017 IEEE/ACM 39th International Conference on Software Engineering*, IEEE, 2017, pp. 3–14.

[86] Dam, H. K., Pham, T., Ng, S. W., Tran, T., Grundy, J., Ghose, A., et al., “A deep tree-based model for software defect prediction,” *arXiv preprint arXiv:1802.00921*, 2018.

[87] P. Nath, J. R. Mushahary, U. Roy, M. Brahma, and P. K. Singh, “AI and Blockchain-based source code vulnerability detection and prevention system for multiparty software development,” *Computers and Electrical Engineering*, vol. 106, p. 108607, 2023.

[88] S. Suneja, Y. Zheng, Y. Zhuang, J. Laredo, and A. Morari, “Learning to map source code to software vulnerability using code-as-a-graph.” 2020.

[89] Y. Mirsky *et al.*, “VulChecker: Graph-based Vulnerability Localization in Source Code,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6557–6574.

[90] R. I. T. Jensen, V. Tawosi, and S. Alamir, "Software Vulnerability and Functionality Assessment using LLMs," *arXiv, 2024.* [Online]. Available: https://arxiv.org/abs/2403.08429.

[91] M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, “ChatGPT for vulnerability detection, classification, and repair: How far are we?,” in *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, Dec. 2023, pp. 632–636.

[92] M. Fu, C. Tantithamthavorn, T. Le, Y. Kume, V. Nguyen, D. Phung, and J. Grundy, “Aibughunter: A practical tool for predicting, classifying and repairing software vulnerabilities,” *arXiv preprint arXiv:2305.16615*, 2023.

[93] Z. Feng *et al.*, “Codebert: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[94] D. Guo *et al.*, “Graphcodebert: Pre-training code representations with data flow,” in *International Conference on Learn. Representations*, 2021.

[95] M. Fu, V. Nguyen, C. K. Tantithamthavorn, T. Le, and D. Phung, “Vulexplainer: A transformer-based hierarchical distillation for explaining vulnerability types,” *IEEE Transactions on Software Engineering*, pp. 1–17, 2023.

[96] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.

[97] G. Bhandari, A. Naseer, and L. Moonen, "CVEFixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.

[98] E. A. Napoli and V. Gatteschi, "Evaluating ChatGPT for smart contracts vulnerability correction," in *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, IEEE, Jun. 2023, pp. 1828–1833.

[99] J. F. F. Thomas Durieux, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ACM, Jun. 2020.

[100] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.

[101] K. Shashwat et al., "A Preliminary Study on Using Large Language Models in Software Pentesting." *arXiv preprint arXiv:2401.17459*, 2024.

[102] T. B. Brown, B. Mann, "Language models are few-shot learners,"*arXiv preprint arXiv:2005.14165* 2023

[103] J. Achiam *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[104] Google Gemini Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, and others, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[105] N. T. Islam, J. Khoury, A. Seong, G. D. L. T. Parra, E. Bou-Harb, and P. Najafirad, "LLM-powered code vulnerability repair with reinforcement learning and semantic reward," *arXiv preprint arXiv:2401.03374*, 2024.

[106] A. Shestov, A. Cheshkov, R. Levichev, R. Mussabayev, P. Zadorozhny, E. Maslov, ... and E. Bulychev, "Finetuning large language models for vulnerability detection," *arXiv preprint arXiv:2401.17010*, 2024.

[107] Lin, X. and Jiang, D., "WizardCoder: Empowering Code Large Language Models with Evol-Instruct," 2023.

[108] S. Liu, B. Wu, X. Xie, G. Meng, and Y. Liu, "ContraBERT: Enhancing Code Pre-Trained Models via Contrastive Learning," in *Proceedings of the 45th International Conference on Software Engineering* (ICSE '23), Melbourne, Victoria, Australia, IEEE Press, 2023.

[109] A. Hejlsberg, S. Wiltamuth, and P. Golde, *C# language specification*. Addison-Wesley Longman Publishing Co., Inc, 2003.

[110] H. Chen, "Comparative study of C, C++, C# and Java programming languages," 2010.

[111] D. Wheeler, “FlawFinder,” [Online]. Available: https://dwheeler.com/flawfinder. [Accessed: Nov. 14, 2024].

[112] Pyt Documentation, “Pyt,” [Online]. Available: https://pyt.readthedocs.io/en/latest/. [Accessed: Nov. 14, 2024]

[113] IBM, “AI and Cybersecurity,” [Online]. Available: https://www.ibm.com/thought-leadership/institute-business-value/en-us/report/ai-cybersecurity. [Accessed: Nov. 14, 2024].

[114] National Institute of Standards and Technology, “About NIST,” May 23, 2024. [Online]. Available: https://www.nist.gov/about-nist. [Accessed: Sep. 23, 2024].

[115] O. W. A. S. P. Foundation, “About the OWASP Foundation,” May 23, 2024. [Online]. Available: https://owasp.org/about/. [Accessed: Sep. 05, 2024].

[116] P. T. Devanbu and S. Stubblebine, “Software engineering for security: A roadmap,” *Proceedings of the Conference on the Future of Software Engineering, ICSE 2000*, pp. 225–239, 2000, doi: 10.1145/336512.336559.

[117] SANS Institute, “SANS: Information Security Resources,” [Online]. Available: https://www.sans.org/.[Accessed: Nov. 14, 2024].

[118] M. Umair, A. Khan, M. Zulkernine, M. U. A. Khan, and M. Zulkernine, “A Survey on Requirements and Design Methods for Secure Software Development” 2009.

[119] Jani, Y. (2023). Implementing continuous integration and continuous deployment (CI/CD) in modern software development. *International Journal of Science and Research, 12*(6), 2984-2987.

[120] De Win, B., Scandariato, R., Buyens, K., Grégoire, J., & Joosen, W. (2009). On the secure software development process: CLASP, SDL, and Touchpoints compared. *Information and Software Technology, 51*(7), 1152–1171. https://doi.org/10.1016/j.infsof.2008.01.010

[121] De Win, B. (2007, September 6). *OWASP Day 2007 Belgium* [PDF]. OWASP. https://owasp.org/www-chapter-belgium/assets/2007/2007-09-06/OWASPDay2007Belgium_BartDeWin.pdf

[122] T. D. LaToza, G. Venolia, and R. DeLine, “Maintaining mental models: A study of developer work habits,” in *Proceedings 28th International Conference Software Engineering, ICSE ’06*, New York, NY, USA, 2006, pp. 492-501, doi: 10.1145/1134285.1134356.

[123] H. Assal and S. Chiasson, “Security in the software development lifecycle,” *Proceedings International Conference Software Engineering and Knowledge Engineering*, 2007, pp. 234-239.

[124] J. Svoboda, *Effectively Combining Static Code Analysis and Manual Code Reviews*, Ph.D. dissertation, Masaryk Univ., Brno, Czech Republic, 2014.

[125] M. Staron, M. Ochodek, W. Meding, and O. Söder, "Using machine learning to identify code fragments for manual review," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 513-516, IEEE.

[126] G. Díaz and J. R. Bermejo, “Static analysis of source code security: Assessment of tools against SAMATE tests,” *Information and software technology*, vol. 55, no. 8, pp. 1462–1476, 2013.

[127] H. H. AlBreiki and Q. H. Mahmoud, "Evaluation of static analysis tools for software security," in *2014 10th International Conference on Innovations in Information Technology (IIT)*, 2014, pp. 93-98.

[128] D. Stefanović, D. Nikolić, D. Dakić, I. Spasojević, and S. Ristić, "Static code analysis tools: A systematic literature review," *Ann. DAAAM Proceedings International DAAAM Symp.*, vol. 31, no. 1, pp. 565–573, Aug. 2020.

[129] Y. Li, L. Ma, L. Shen, J. Lv, and P. Zhang, "Open source software security vulnerability detection based on dynamic behavior features," *PLoS One*, vol. 14, no. 8, p. 0221530, 2019.

[130] T. Sutter, T. Kehrer, M. Rennhard, B. Tellenbach, and J. Klein, “Dynamic Security Analysis on Android: A Systematic Literature Review,” *IEEE Access*, 2024.

[131] T. Ball, "The concept of dynamic analysis," *ACM SIGSOFT Software Engineering Notes*, vol. 24, no. 6, pp. 216–234, 1999.

[132] O. Chebaro, P. Cuoq, N. Kosmatov, B. Marre, A. Pacalet, N. Williams, and B. Yakobowski, "Behind the scenes in SANTE: a combination of static and dynamic analyses," *Autom. Softw. Eng.*, vol. 21, no. 1, pp. 107–143, 2014.

[133] B. Kiss, N. Kosmatov, D. Pariente, and A. Puccetti, "Combining Static and Dynamic Analyses for Vulnerability Detection: Illustration on Heartbleed," in *Hardware and Software: Verification and Testing. HVC 2015*, Lecture Notes in Computer Science, vol. 9434, pp. 1–15, 2015.

[134] A. Sadeghian, M. Zamani, and A. A. Manaf, "A taxonomy of SQL injection detection and prevention techniques," in *2013 International Conference on Informatics and Creative Multimedia*, IEEE, Sep. 2013, pp. 53–56.

[135] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "CodeGen2: Lessons for Training LLMs on Programming and Natural Languages," *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2305.02309

[136] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," *arXiv*, 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[137] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[138] Z. Ali, H. Darwis, L. B. Ilmawan, S. R. Jabir, and A. R. Manga, “Memory efficient with parameter

efficient fine-tuning for code generation using quantization," in *2024 18th International Conference on Ubiquitous Information Management and Communication (IMCOM)*, IEEE, Jan. 2024, pp. 1–6.

[139] A. Pathak, O. Shree, M. Agarwal, S. D. Sarkar, and A. Tiwary, "Performance analysis of LoRA finetuning Llama-2," in *2023 7th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech)*, Kolkata, India, 2023, pp. 1-4, doi: 10.1109/ IEMENTech60402.2023. 10423400.

[140] Meta, "Introducing LLaMA: A foundational, 65-billion-parameter large language model," Meta AI, 2023. [Online]. Available: https://ai.meta.com/blog/large-language-model-llama-meta-ai/. [Accessed: Sep. 14, 2024].

[141] Meta, "Meta and Microsoft Introduce the Next Generation of Llama," Meta AI, 2023. [Online]. Available: https://ai.meta.com/blog/llama-2/. [Accessed: Nov. 14, 2024].

[142] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," Apr. 2021, doi: 2104.09864.

[143] U. Tiwari, "Introduction to LLaMA2: Part 1 - Architectural Analysis," Medium, Sep. 2023. [Online]. Available: https://medium.com/@utsavtiwari9936/introduction-to-llama2-part-1-architectural-analysis-3e335e7b1104. [Accessed: Nov.14, 2024].

[144] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "A Conversational Paradigm for Program Synthesis," *arXiv preprint*, 2022.

[145] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," *arXiv preprint*, 2023. [Online]. Available: https://arxiv.org/abs/2203.13474.

[146] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[147] M. Wistuba, P. T. Sivaprasad, L. Balles, and G. Zappella, "Choice of PEFT Technique in Continual Learning: Prompt Tuning is Not All You Need," *arXiv preprint arXiv:2406.03216*, 2024.

[148] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[149] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.

[150] Mercity.ai, "Guide to Fine-Tuning LLMs with LoRA and QLoRA," [Online]. Available: https://www.mercity.ai/blog-post/guide-to-fine-tuning-llms-with-lora-and-qlora. [Accessed: Nov. 14, 2024].

151] K. S. Kalyan, "A survey of GPT-3 family large language models including ChatGPT and GPT-4,"

Natural Language Processing Journal, p. 100048, 2023, Elsevier.

[152] P. Kumar, "Large language models (LLMs): survey, technical frameworks, and future challenges," *Artificial Intelligence Review*, vol. 57, Aug. 2024, doi: 10.1007/s10462-024-10888-y.

[153] OpenAI, "Models," OpenAI, 2024. [Online]. Available: https://platform.openai.com/docs/models. [Accessed: Nov. 05, 2024].

[154] ResearchGate, "Proposed deep learning model with the Word2vec embedding layer," [Online]. Available:https://www.researchgate.net/figure/Proposed-deep-learning-model-with-the-Word2vec-embedding-layer_fig1_368514020. [Accessed: Nov. 05, 2024].

[155] D. Meyer, "How exactly does Word2vec work," *Uoregon.edu, Brocade.com*, pp. 1–18, 2016.

[156] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," *Neural Computation*, vol. 31, no. 7, pp. 1235–1270, 2019.

[157] M. Navim and R. Pankaja, “Performance analysis of text classification algorithms using confusion matrix,” *International Journal of Engineering and Technology Research,* vol. 6, pp. 75–78, 2016.

[158] J. Karlberg and M. Axén, “Binary classification for predicting customer churn,” Master’s thesis, Umeå University, 2020.

[159] S. Narkhede, “Understanding auc-roc curve,” Towards Data Science, vol. 26, pp.220–227, 2018.

[160] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-Aware Unified Pre-Trained Encoder- Decoder Models for Code Understanding and Generation," *arXiv preprint arXiv:2109.00859*, 2021.

[161] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol.19, no. 4, pp. 2244–2258, 2022, doi: 10.1109/TDSC.2021.3051525.

[162] P. Dessauw, A. Delaitre, H. M. Carvalho, and V. Okun, “A modular and expandable testbed for evaluating ml-based bug finders,” *in 2022 IEEE 29th Annual Software Technology Conference* (STC), IEEE Computer Society, Oct. 2022, pp. 57–58.

[163] M. D. Purba, A. Ghosh, B. J. Radford, and B. Chu, "Software vulnerability detection using large language models," in *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, IEEE, Oct. 2023, pp. 112–119.

[164] D. Noever, "Can large language models find and fix vulnerable software?," *arXiv preprint arXiv:2308.10345*, 2023.

Appendix A

Source Code Repository

All the source code and datasets used in this research can be found in the following repository link. It includes all the necessary files and scripts.

- https://github.com/ravihansalk/Code-Vulnerability-Detection

For research students and institutions this repository provides open access to explore the implementation and gain a deeper understanding of the applied methodologies and the ability to replicate the results to help validate the findings. This also is a launching point for other researchers to extend this work and generate new research on the subject.